Derived Dynamics Meet Learned Control: Omnidirectional Velocity Tracking for a Small Ballbot

Nebiyu Tadesse
Johns Hopkins University
Working draft; updated as experiments complete

Abstract—A ballbot balances on a single ball and moves by leaning, so even "drive left at 0.5 m/s" requires pushing the ball the wrong way first, holding a precise lean while accelerating, and never letting the ball escape. We study this velocity-tracking problem for a small three-omniwheel ballbot from two directions and connect them. From first principles, we derive the planar Euler–Lagrange dynamics including the omniwheel transmission and reflected rotor inertia, prove three structural properties (an unstable pole at $4.79\,\mathrm{s}^{-1}$, a non-minimum-phase zero at $\sqrt{m_a g l/(I_a + m_a l^2 + m_a l r)} = 5.78\,\mathrm{s}^{-1}$, and a lean–acceleration law $\theta = \kappa a$ with $\kappa = 0.176$ rad per $\mathrm{m/s}^2$ that is independent of the drive), and design an LQR controller on the derived model. From data, we train proximal-policy-optimisation (PPO) policies over nine stages and 25 configurations, documenting a ball-ejection reward exploit, a failure of reward shaping, and the discovery that the training command distribution, not the reward form, limited tracking (speed ratio at 1 m/s raised from 0.31 to 0.91 by capping the spin range; 0.95 on a second seed). We then test the theory on both controllers in the full MuJoCo simulator. A torque-free momentum identity of the derived model holds on all trajectories (slope 0.88–0.93, R^2 up to 0.82); the learned policies, trained only from reward, obey the derived lean law (91% and 83% of κ) and exhibit the predicted non-minimum-phase reversal. Head-to-head, the model-based LQR tracks 0.25–0.75 m/s within 4%, is heading-invariant and 50× smoother, while the learned policy survives the nonlinear regime (1 m/s, turning while driving) where the LQR falls. This complementarity motivates a residual policy on top of the LQR. At scale, we move the learning loop onto a GPU physics engine and, on the corrected engine (a late-found error, stale subtree mass under mass randomisation, invalidated all earlier randomised results), compare pure learning, an LQR and a bounded learned residual on an LQR base. On a clean flat world pure learning reaches only about 60 % of the commanded speed while the LQR is 9–15 times more accurate and the residual keeps that accuracy (error 6–12 % of the command up to 0.9 m/s); on a goal task across ten terrain kinds the residual succeeds in 0.58 of worlds against 0.44 for pure learning, and two seeds agree to about 0.02. Training over wider randomisation changes almost nothing, and no training recipe improves success under pushes and pulls (about 0.3) or the tile velocity task. A fall diagnosis shows that the body derails off the ball at about 15° of tilt with torque to spare; a passive ball retainer (arrestors after ETH Zurich's Rezero, our own design, adding no torque about the ball centre) raises goal success with pushes and pulls to about 0.47 and cuts tile velocity falls from 0.53 to about 0.32 (two seeds), and with it the learned authority sets a trade-off between tracking and robustness: we recommend the LQR plus residual at authority 0.25 trained with the retainer, which tracks speed exactly and reaches the goal in 0.42 to 0.44 of worlds with pushes and pulls (0.29 to 0.30 without the retainer) on three seeds. The remaining terrain falls are set mainly by obstacle size and match a traction limit set by ball radius and friction (a larger ball crosses them; a perfect height map given to the policy does not help); on the corrected contact model they also rise with approach speed, but a terrain-aware speed layer that slows the robot before a feature reduces them by only 0.03 to 0.05; routing around what cannot be crossed and a larger ball are what work. Where obstacles can be avoided, terrain sensing is decisive: a planner on top of the frozen recommended controller cuts falls from 0.91 to 0.07 with a perfect map, and smooth sensor-based navigation with a modest forward time-of-flight sensor reaches 0.84 of goals behind unclimbable blocks, within 0.05 of the perfect map; the remaining failures are push recoveries near obstacles, a balance-controller limit, and training with the planner in the loop changes nothing on two seeds. Our final recommendation adds a motion-reference layer between planner and controller, trained together with an adaptive residual authority (0.15 while cruising, 0.3 when manoeuvring) and a goal dead band: over three seeds it reaches 0.77 to 0.79 of terrain goals undisturbed and 0.51 to 0.52 under pushes and pulls, and with the ToF sensor it reaches 0.88 of navigation goals with about a quarter less heading wobble; a higher tracking reward, more training share for hard designs, twice the training and harder training pushes do not improve on it, and the remaining limits are physical (a light ball, a heavy body, weak motors, and sustained pulls above about 3 N). On two unseen robot designs the LQR base carries a policy (0.61 to 0.71 on a larger robot) while pure learning collapses (0.09 to 0.15); a tenth-torque design is motor limited for every controller. Three disturbance-focused variants (higher fall penalty, push curriculum) did not help, and on held-out terrain the goal success drops only slightly. A head-mounted depth sensor gives no gain over a blind control (one seed each). All results are in simulation; we make no hardware claims.

Index Terms—ballbot, underactuated systems, Lagrangian dynamics, LQR, reinforcement learning, velocity tracking, reward hacking, curriculum, residual learning

I. Introduction

Ballbots are omnidirectional, have a footprint the size of their ball and move with a fluid, lifelike quality [3], [4], which makes them attractive for small character-like home robots. The control problem is hard for structural reasons: the system is underactuated (three wheel torques for a body and a ball with five planar degrees of freedom), open-loop unstable, and non-minimum phase, because the ball must first be pushed backward to tip the body

forward. Classical controllers linearise the dynamics and stack an LQR or PID balance loop under a velocity loop [4], [5], [6]; learning-based controllers promise to handle the non-linear, contact-rich regime without hand-derived models [8], [2]. The two are rarely compared on the same robot, and it is rarely asked whether a learned policy behaves the way the physics says it must.

Our earlier report [1] reproduced the PPO pipeline of [2] and stated the Lagrangian form $M(q)\ddot{q}+C(q,\dot{q})\dot{q}+G(q) = \tau$ without deriving it. This paper derives that model for a specific robot, extracts testable predictions from it, builds a model-based controller, and uses both to analyse learned controllers. We focus on the interface every higher layer will use: track a commanded planar velocity and yaw rate, in any direction, smoothly, while staying on the ball.

Contributions.

1) A planar Euler–Lagrange model of a three-omniwheel ballbot with explicit omniwheel transmission and reflected rotor inertia, with parameters extracted from the simulator, and three structural results: instability, a closed-form non-minimum-phase zero, and a drive-independent lean–acceleration law with a torque-free momentum identity (§IV).
2) Validation of the derived model against the full 3-D MuJoCo simulation, and an LQR controller designed purely on the derived model that balances and tracks the simulated robot without simulator tuning (§V-A).
3) A documented learning study: a ball-ejection exploit under relaxed limits, the failure of reward shaping, and evidence that infeasible commands in the training distribution suppress learning of feasible ones (§VII).
4) A theory-versus-learning analysis showing that PPO policies rediscover the derived lean law and the non-minimum-phase reversal, and a head-to-head benchmark that motivates residual learning on top of the model-based controller (§VIII).
5) A GPU-resident training pipeline validated against the CPU engine, with the exact observation, reward, randomisation, terrain, design and evaluation definitions (§IX), the clean flat-world comparison of pure learning, LQR and LQR plus residual, and an account of a late-found engine error (stale subtree mass under mass randomisation) that invalidated the randomised results (§IX-A, §X).
6) A bounded residual on the LQR base that is accurate up to 0.9 m/s on the flat world on two seeds, with its flat-world authority sweep; goal and robustness results are pending on the corrected engine (§X).

II. Related Work

Ballbot modelling and control. The CMU ballbot [3], [4] established planar decoupling of the linearised 3-D dynamics and LQR/PID balance with an outer trajectory loop. Three-omniwheel designs with planar models and LQR follow the same approach [5], [6], [7]; higher-level planners such as topology-informed MPC [9] and constrained path integrals [10] assume an accurate velocity interface. The non-minimum-phase character of wheeled inverted pendulums, and the resulting bandwidth limits, are classical [11]. Learning ballbot control. Zhou et al. [8] learn balancing; Salehi [2] drives over uneven terrain in a single commanded direction with PPO and depth perception, and provides the simulator we build on. Command tracking with RL. Randomised velocity commands with dense tracking rewards are standard for legged robots [12], [13], [14]. Smoothness, robustness and hybrid control. Action-rate regularisation [15], domain randomisation [16], [17] and residual policies on top of model-based controllers [18], [19] are established; we test them on a non-minimum-phase balancer. Specification gaming. Our ball-ejection exploit is an instance of reward misspecification [20].

TABLE I
Model parameters (extracted from the simulator model).

Symbol	Meaning	Value
m_a	body mass (incl. wheels, sensors)	3.853 kg
l	body COM height above ball centre	0.1306 m
I_a	body pitch inertia about its COM	0.0366 kg m^2
$m_k,\ r,\ I_k$	ball mass, radius, inertia	0.168 kg, 0.09 m, 5.44×10^{-4} kg m^2
$J_w,\ d_w$	wheel rotor inertia, damping	0.005 kg m^2, 0.1 N m s
W	omniwheel transmission (Eq. (1))	3×3
$J_{\text{eff}},\ d_{\text{eff}}$	reflected inertia, damping (Eq. (2))	0.1025 kg m^2, 2.05 N m s
$\tau_{\max}$	wheel torque limit	30 N m

III. System and Notation

The MuJoCo [23] robot (Fig. 5) has a cylindrical body with low ballast, a solid ball, and three omniwheels at 120° azimuth spacing whose contact points lie 51.2° from the ball's vertical. Omniwheels are modelled as capsules with anisotropic contact friction (0.001 along the roller direction, 1.0 across it); a contact-frame correction makes the low-friction direction consistent on all three wheels [2]. Actuators are torque motors with rotor inertia (armature) J_w and viscous damping d_w. Physics runs at 500 Hz; controllers act at 100 Hz. Table I lists the parameters, all extracted programmatically from the model file.

IV. First-Principles Model

A. Omniwheel transmission

Let $\boldsymbol{\Omega} \in \mathbb{R}^3$ be the angular velocity of the ball relative to the body, expressed in the body frame. Wheel i has spin axis $\boldsymbol{a}_i$, effective radius r_w at the contact, contact point $\boldsymbol{p}_i$ on the ball (relative to its centre), and rim direction $\boldsymbol{t}_i = \boldsymbol{a}_i \times (\boldsymbol{p}_i - \boldsymbol{c}_i)/r_w$ at the contact, where $\boldsymbol{c}_i$ is the wheel centre. No-slip in the driven direction (the passive rollers absorb the orthogonal component) requires $\omega_{w,i} r_w = (\boldsymbol{\Omega} \times \boldsymbol{p}_i) \cdot \boldsymbol{t}_i$, hence

$$\boldsymbol{\omega}_w = W\boldsymbol{\Omega}, \qquad W_{i,:} = \tfrac{1}{r_w}(\boldsymbol{p}_i \times \boldsymbol{t}_i)^\top. \tag{1}$$

By virtual power, wheel torques $\boldsymbol{\tau}_w$ exert the torque $\boldsymbol{T} = W^\top \boldsymbol{\tau}_w$ on the ball relative to the body. For this robot

W is invertible, so any desired ball torque is realised by the allocation $\boldsymbol{\tau}_w = W^{-\top}\boldsymbol{T}$. In the pitch plane ($\boldsymbol{\Omega} = \Omega_y \hat{\boldsymbol{y}}$) the wheel rates are $\boldsymbol{n}\,\Omega_y$ with $\boldsymbol{n} = W_{:,2} = (3.61, -1.11, -2.50)$, and the rotors contribute kinetic energy $\frac{1}{2} J_w \|\boldsymbol{n}\|^2 \Omega_y^2$. The wheels therefore act as a single reflected inertia and damper,

$$J_{\text{eff}} = J_w \|\boldsymbol{n}\|^2, \qquad d_{\text{eff}} = d_w \|\boldsymbol{n}\|^2, \tag{2}$$

and $J_{\text{eff}} = 0.1025\,\text{kg}\,\text{m}^2$ is 188 times the ball's own inertia: the drive, not the ball, dominates the rotational inertia seen by the controller.

B. Planar Euler–Lagrange dynamics

Following the planar decoupling of [4], [5], consider one vertical plane with generalised coordinates $q = (\varphi, \theta)$: the absolute ball rotation angle and the absolute body tilt. Rolling without slip gives the ball position $x = r\varphi$, and the body COM is at $(x + l\sin\theta,\ l\cos\theta)$. The wheels turn with the relative angle $\varphi - \theta$. The kinetic and potential energies are

$$T = \tfrac{1}{2}(m_k r^2 + I_k)\dot{\varphi}^2 + \tfrac{1}{2} m_a \big[(r\dot{\varphi} + l\cos\theta\,\dot{\theta})^2 + (l\sin\theta\,\dot{\theta})^2\big] + \tfrac{1}{2} I_a \dot{\theta}^2 + \tfrac{1}{2} J_{\text{eff}} (\dot{\varphi} - \dot{\theta})^2, \tag{3}$$

$$V = m_a g l \cos\theta. \tag{4}$$

The motor torque τ and wheel damping act on the relative coordinate, so the generalised forces are $Q = (\tau - d_{\text{eff}}(\dot{\varphi} - \dot{\theta}))\,(1, -1)^\top$. The Euler–Lagrange equations $\frac{d}{dt}\partial_{\dot{q}} L - \partial_q L = Q$ with $L = T - V$ give the form stated in [1],

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + D\dot{q} + G(q) = B\tau, \tag{5}$$

with $B = (1, -1)^\top$, $D = d_{\text{eff}} \left(\begin{smallmatrix} 1 & -1 \\ -1 & 1 \end{smallmatrix}\right)$, $G(q) = (0, -m_a g l \sin\theta)^\top$, Coriolis term $C(q,\dot{q})\dot{q} = (-m_a l r \sin\theta\,\dot{\theta}^2,\ 0)^\top$, and

$$M(q) = \begin{pmatrix} I_k + J_{\text{eff}} + (m_a + m_k) r^2 & m_a l r \cos\theta - J_{\text{eff}} \\ m_a l r \cos\theta - J_{\text{eff}} & I_a + J_{\text{eff}} + m_a l^2 \end{pmatrix}. \tag{6}$$

The derivation is reproduced symbolically by ballbot_model.py. Linearising about the upright equilibrium with state $s = (\varphi, \theta, \dot{\varphi}, \dot{\theta})$ gives $\dot{s} = As + B_s \tau$ with

$$A = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 11.53 & -12.35 & 12.35 \\ 0 & 27.31 & 6.56 & -6.56 \end{pmatrix}, \qquad B_s = \begin{pmatrix} 0 \\ 0 \\ 6.02 \\ -3.20 \end{pmatrix}. \tag{7}$$

The 3-D linearisation decouples into two such planes plus a yaw channel [4].

C. Structural properties

Let $M_0 = M(0)$ with entries M_{11}, M_{12}, M_{22}, and $\Delta = \det M_0$.

Proposition 1 (Open-loop instability). With $d_{\text{eff}} = 0$ the linearised plant has poles $\{0, 0, \pm\lambda\}$ with $\lambda = \sqrt{m_a g l\, M_{11}/\Delta}$.

Proof. The characteristic polynomial of (5) linearised is $\det(M_0 s^2 + K)$ with $K = \text{diag}(0, -m_a g l)$, i.e. $s^2(\Delta s^2 - m_a g l\, M_{11})$. □

Numerically $\lambda = 5.23\,\text{s}^{-1}$; with damping the unstable pole is $4.79\,\text{s}^{-1}$ (time constant 0.21 s), so an unactuated lean grows e-fold every 0.21 s.

Proposition 2 (Non-minimum phase). The transfer function from motor torque to ball position has a real zero pair $\pm z$ with

$$z = \sqrt{\frac{m_a g l}{I_a + m_a l^2 + m_a l r}}, \tag{8}$$

independent of J_{eff} and d_{eff}. The zero at $+z$ makes the system non-minimum phase.

Proof. By Cramer's rule on $(M_0 s^2 + Ds + K)\hat{q} = B\hat{\tau}$, the numerator of $\hat{\varphi}/\hat{\tau}$ is $(M_{22} + M_{12})s^2 - m_a g l$ (the J_{eff} and d_{eff} terms cancel because both enter through the relative coordinate), and $M_{22} + M_{12} = I_a + m_a l^2 + m_a l r$. □

Numerically $z = 5.78\,\text{s}^{-1}$. Physically, to accelerate forward the controller must first tip the body forward, which by reaction drives the ball backward; any controller, learned or not, must show this reversal, and the zero bounds the achievable tracking bandwidth [11].

Proposition 3 (Momentum identity and lean–acceleration law). Adding the two rows of (5) eliminates τ and d_{eff}:

$$(M_{11} + M_{12})\ddot{\varphi} + (M_{12} + M_{22})\ddot{\theta} - m_a l r \sin\theta\,\dot{\theta}^2 = m_a g l \sin\theta. \tag{9}$$

Under quasi-static lean ($\ddot{\theta} = \dot{\theta} = 0$) the ball acceleration $a = r\ddot{\varphi}$ and the lean satisfy $\sin\theta = \kappa a$ with

$$\kappa = \frac{(m_a + m_k) r^2 + I_k + m_a l r}{m_a g l r}. \tag{10}$$

Proof. Row sums of $M(q)$ are $M_{11} + M_{12} = I_k + (m_a + m_k) r^2 + m_a l r \cos\theta$ and $M_{12} + M_{22} = I_a + m_a l^2 + m_a l r \cos\theta$; the J_{eff} terms cancel, as do $B\tau$ and $D\dot{q}$ because $(1,1)B = (1,1)D = 0$. Setting $\ddot{\theta} = \dot{\theta} = 0$ and linearising $\cos\theta$ gives (10). □

Corollary 1 (Acceleration budget). For this robot $\kappa = 0.176\,\text{rad per m/s}^2$ (10.1° per m/s^2). A sustained acceleration a requires a lean of $10.1^\circ a$; the off-ball limit of 30° caps quasi-static acceleration near $3\,\text{m/s}^2$, and a policy that leans 1° accelerates at only $\approx 0.1\,\text{m/s}^2$.

Identity (9) contains no control or drive terms, so it must hold on every trajectory of every controller. It is therefore a controller-independent test of the derived inertia model.

Remark 1 (Constant velocity). At constant velocity v the equilibrium is upright ($\theta = 0$) with torque $\tau = d_{\text{eff}} v / r$ to overcome wheel damping, about 23 N m in the plane at 1 m/s, which the allocation spreads over three wheels well within $\tau_{\max}$. The planar model therefore predicts no torque-imposed speed limit near 1 m/s.

D. Validation against the simulator

We test the model on the full 3-D simulation (contact-frame stepper, implicit integration). (i) The momentum identity (9), evaluated on trajectories of three different controllers, has a regression slope of 0.88–0.93 against its right-hand side (R^2 0.60–0.82; §VIII). (ii) An LQR designed on (A, B_s) alone stabilises the simulated robot (§V-A). (iii) Instantaneous accelerations under a torque step match for the ball (3.17 vs. 3.01 rad/s^2 per 0.5 N m) but not for the body (0.73 vs. 1.60), because MuJoCo's soft contacts allow initial creep at the ball–floor contact (measured ball acceleration 2.5× the rolling value). (iv) Measured wheel rates differ from Eq. (1) by a constant 27–28% across speeds, indicating an error in the effective rolling radius of the capsule contact rather than speed-dependent slip. We report these mismatches as limits of the planar model rather than tuning them away.

V. Controllers

A. Model-based: LQR with reference governor

For each plane we minimise $\int (s^\top Q s + R\tau^2)\, dt$ with $Q = \mathrm{diag}(10^{-6}, 50, 1, 1)$, $R = 1$, giving $\tau = -K(s - s_{\mathrm{ref}})$ with $K = (-0.001, -76.1, -4.33, -11.6)$ (negative entries because B_s has opposite signs on φ and θ; magnitudes $(0, 76.1, 4.33, 11.6)$) from the continuous algebraic Riccati equation and closed-loop poles $\{-20.3, -5.02, -4.50, \approx 0\}$. Velocity tracking uses $s_{\mathrm{ref}} = (\cdot, 0, v_{\mathrm{ref}}/r, 0)$. Because of the non-minimum-phase zero, a step in v_{ref} demands an impulsive torque, so a reference governor moves v_{ref} toward the command at most $a_{\max} = 0.5\,\mathrm{m/s^2}$ (a lean of about 5° by the corollary). A slow outer integral on the measured body velocity removes steady error from contact creep. Yaw uses $T_z = -k_\psi(\omega_{\mathrm{cmd}} - \omega_z)$. Planar torques for the x and y planes and T_z form $\boldsymbol{T}$, mapped to wheels by $\boldsymbol{\tau}_w = W^{-\top} R^\top \boldsymbol{T}$ with the body rotation R. The controller reads body attitude and rates (IMU) and ball rotation (wheel odometry); it was not tuned on the simulator beyond choosing $a_{\max}$, k_ψ and the integral gain.

B. Learned: PPO velocity-tracking policies

MDP. The state observed by the policy is $o_t = (\boldsymbol{\rho}_t, \boldsymbol{\omega}_t, \boldsymbol{v}_t, \boldsymbol{\omega}_{w,t}, a_{t-1}, c_t, \sin\psi_t, \cos\psi_t)$: body orientation as a rotation vector, body angular and linear velocity, wheel speeds, the previous action, the command $c = (v_x, v_y, \omega_z)$ and the heading. The action $a_t \in [-1, 1]^3$ is the normalised wheel-torque vector, $\boldsymbol{\tau}_w = \tau_{\max} a_t$. Commands are resampled every 2–3 s: zero with probability 0.1, otherwise speed $\sim \mathcal{U}[0, v_{\max}]$, direction $\sim \mathcal{U}[0, 2\pi)$, yaw rate $\sim \mathcal{U}[-\omega_{\max}, \omega_{\max}]$. Episodes last 8 s and terminate with a fall penalty when tilt exceeds its limit or the body–ball angle θ_b exceeds $\theta_{\max}$ (§VII-A).

Rewards. With v the planar velocity, $u = c_{xy}/\|c_{xy}\|$, $p = v \cdot u$ and $e = \|v - c_{xy}\|$, the tracking terms compared were

$$r_{\mathrm{dot}} = \min(p, \|c_{xy}\|) - \tfrac{1}{2}\|v - pu\|, \tag{11}$$

$$r_{\mathrm{err}} = \|c_{xy}\| - e + \tfrac{1}{2} e^{-e^2/0.25^2}, \tag{12}$$

$$r_{\mathrm{lean}} = k_\ell\big(1 - \min(p/\|c_{xy}\|, 1)\big)\,\mathrm{clip}(\ell/0.3, -1, 1), \tag{13}$$

plus a yaw analogue at weight 0.5, where ℓ is the body-over-ball offset along u divided by its height. Regularisers: $-10^{-4}\|a_t\|^2$, an action-rate term $k_r\|a_t - a_{t-1}\|^2$, and a soft off-ball term $-k_b(\theta_b/\theta_{\max})^2$.

Optimisation. PPO [21] maximises the clipped surrogate

$$L^{\mathrm{CLIP}}(\phi) = \mathbb{E}_t\big[\min(\rho_t \hat{A}_t,\ \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t)\big], \tag{14}$$

with $\rho_t = \pi_\phi(a_t|o_t)/\pi_{\phi_{\mathrm{old}}}(a_t|o_t)$, generalised advantages $\hat{A}_t = \sum_k (\gamma\lambda)^k \delta_{t+k}$, $\delta_t = r_t + \gamma V(o_{t+1}) - V(o_t)$ [22], a squared-error value loss and an entropy bonus. We use $\epsilon = 0.2$, $\gamma = 0.995$, $\lambda = 0.95$, target KL 0.03, learning rate $3{\times}10^{-4} \to 3{\times}10^{-5}$, 7 environments × 512 steps per update, minibatch 1792, 8 epochs, and separate 2×256 tanh actor and critic MLPs (143k parameters) in Stable-Baselines3 [24], with running observation normalisation.

Simulation throughput. The contact-frame correction must run between collision detection and constraint construction, so the original RK4 step is executed in Python stages (four evaluations per step). Switching the integrator to MuJoCo's implicit-fast scheme (one evaluation) cut the control-step cost from 3.26 to 1.10 ms; RK4-trained policies produced the same speed curves within 0.01 m/s on the new integrator, so later runs use it.

C. Hybrid: residual policy on the LQR

The residual controller applies $a_t = \mathrm{clip}\big(a_t^{\mathrm{LQR}} + \beta\,\pi_\phi(o_t)\big)$ with $\beta = 0.3$, so the LQR supplies a stabilising, heading-invariant prior and the policy learns corrections where the linear model fails [18], [19]. [TBD: results]

VI. Evaluation Protocol

All controllers are evaluated deterministically on the nominal robot with the same scripts:

- Speed grid: 8 directions × speeds 0.25–1.0 m/s (and up to the training $v_{\max}$), 4 s each, 2 repeats; achieved velocity averaged over the last 2 s. Speed ratio is achieved/commanded speed; RMSE is over commands ≤1 m/s; grid falls counts falls within those trials.
- Yaw: spin in place at ±1 rad/s.
- Transients: rise time to 90% of 0.5 m/s; reversal time from +0.5 to −0.5 m/s; a 0.5 m/s arc at 1 rad/s.
- Heading: 0.5 m/s commands in 4 directions from 6 random start headings.
- Smoothness: mean $\|a_t - a_{t-1}\|^2$ over all tests.
- Stability: 30 episodes of random training-distribution commands, recording the command at each fall.

- Path following: a square course (four 1.5 m legs at 0.5 m/s), a 1 m/s leg, a turning leg and a diagonal leg; the planned path integrates the commands, time limits are disabled so the robot is never silently reset, and a fall ends the run.

VII. Learning Results

Table II lists every controller in study order and Fig. 1 the speed ratio at 1 m/s.

A. Relaxing limits invites a ball-ejection exploit

At the original actuator envelope (10 N m, damping 0.8, tilt limit 20°) policies balanced but barely moved (ratio 0.06). Raising torque to 30 N m, lowering damping to 0.1 and the tilt limit to 70° increased reward, but a trace showed the policy driving the ball out from under the body: θ_b reached 59° at 0.2 s and 88° at 0.6 s while the ball rolled away at ≈1 m/s and the body stood on its wheels. Body tilt never reached the limit. A tilt limit is not a ballbot fall detector; the constraint that defines the task, $\theta_b < \theta_{\max}$, must be enforced explicitly.

B. Reward shaping did not fix timid tracking

With $\theta_{\max} = 30°$ enforced, stronger rewards, lower fall penalties, smoothness penalties, the lean bonus and the error reward (S2–S5) all stayed at ratios ≤0.36. The policies used 1–3% of the available torque and leaned about 1°, which by the corollary yields ≈0.1 m/s²; yet given an out-of-distribution 2 m/s command, an S4 policy reached 1.21 m/s. The robot could go faster; the policies chose not to.

C. The command distribution was the bottleneck

Every policy trained with $\omega_{\max} = 3$ rad/s fell in 20–30 of 30 random-command episodes; for trk_a, 23 of 29 falls occurred under yaw commands above 1 rad/s (trk_c: 24 of 29). Capping $\omega_{\max} = 1$ and training from scratch (trk_f) raised the ratio at 1 m/s from 0.31 to 0.91 and at 0.5 m/s to 0.98 (second seed 0.95 and 1.08). Fine-tuning the timid policies instead reached 0.59 or collapsed (trk_e, 30/30 falls). For an unstable balancer, infeasible commands in the training distribution do not merely fail on their own; they suppress learning of the feasible ones, and policies that learned to avoid motion do not easily unlearn it.

D. Smoothness as a second stage

Continuing trk_f (trk_g) cut the action rate 11× (0.147 to 0.013) while keeping the ratio at 0.99 at 0.5 m/s and improving yaw error from 0.40 to 0.12 rad/s; a stronger action-rate penalty (trk_h) gave the fewest falls (3/30). Applied from the start (S3), the same penalties froze motion. Fig. 2 shows the trade-off.

VIII. Theory Versus Learning

A. Do learned policies obey the derived physics?

We apply the three structural results to step, reversal and stop manoeuvres of the LQR and two PPO policies (analyze_theory.py; Fig. 3).

Momentum identity. Regressing the left-hand side of (9) on $m_a g l \sin\theta$ from filtered ball and body rates gives slopes of 0.93 (LQR), 0.88 (trk_g) and 0.89 (trk_h) against an ideal 1, with R^2 of 0.82, 0.60 and 0.60. Because the identity contains no control term, this validates the derived inertia matrix on trajectories of arbitrary controllers.

Non-minimum-phase reversal. After a 0.5 m/s forward step, all three controllers first move the ball backward: by 1.7 cm (LQR), 0.3 cm (trk_g) and 1.1 cm (trk_h), within the first 0.3 s, as Proposition 2 requires.

Lean–acceleration law. On quasi-static samples ($|\ddot{\theta}| < 0.5$ rad/s²) the lean regresses on acceleration with slopes of 0.168 (LQR, 95% of κ, $R^2 = 0.86$), 0.160 (trk_g, 91%, $R^2 = 0.73$) and 0.146 (trk_h, 83%, $R^2 = 0.67$). The learned policies, trained from reward alone, rediscovered the lean law implied by the Lagrangian. They differ in how they use it: PPO leans faster but less deeply (peak ≈ 6° at 0.5 s versus the LQR's 9° at 0.8 s), trading peak acceleration for a smoother approach.

B. Head-to-head: derived versus learned control

Table II (first row) and Fig. 4 compare the LQR with the learned policies. Inside the linear model's validity, the LQR is better on almost every axis: speed ratio 1.00–1.08 at 0.25–0.75 m/s, rise time 0.89 s, heading error 0.02 m/s with no falls (it is heading-invariant by construction, whereas the best policies show 0.31 m/s error and 21–38% falls from random headings), and an action rate of 0.0002, 50× smoother than the smoothest policy. Outside it, the learned policy wins: the LQR falls in every 1 m/s trial and on the square course's turning leg, while trk_h completes the full course including 1 m/s and 0.5 m/s while turning at 1 rad/s. Neither yaw controller is strong (±0.57 rad/s for the LQR at ±1 commanded), which we attribute to the drive: equal wheel torques mostly spin the ball against negligible floor torsional friction.

The 1 m/s edge. Both controller families fail near 1 m/s, which suggests a physical cause. Diagnostics on the LQR rule out the obvious ones: commanded torque stays below 17% of the limit until the final 0.2 s, wheel normal forces are steady at 17–18 N, and the wheel-rate discrepancy is constant across speed (no growing slip). The failure coincides with an unmodelled yaw spin-up (to 0.9 rad/s) and a growing body–ball oscillation, consistent with 3-D coupling absent from the planar model. We tested one candidate, gyroscopic torques of the rotors ($J_w \omega_w \approx 0.2$ N m s per wheel at 40 rad/s), by re-deriving the model and a discrete-time LQR (designed at the 100 Hz control period; a continuous design becomes sampled-data unstable when light rotors push its fastest pole to $-135\,\mathrm{s}^{-1}$) for rotor

TABLE II

Flat-ground velocity-tracking benchmark for every policy, in study order (single seed unless noted, deterministic policy, 2 repeats per condition). Ratio: achieved/commanded speed averaged over 8 directions. RMSE: planar velocity error over all commands $\leq$1 m/s. Yaw err: $|\omega_z - \omega_{cmd}|$ at $\pm$1 rad/s. Act. rate: mean $\|a_t - a_{t-1}\|^2$ (lower is smoother). Grid falls: share of speed-grid trials ($\leq$1 m/s) that fell. Falls/30: falls in 30 episodes of random training-distribution commands. Heading err: 0.5 m/s commands from random start headings. †Policy leaves the ball (invalid).

Stage	Run (change)	Ratio@0.5	Ratio@1.0	RMSE [m/s]	Yaw err [rad/s]	Act. rate	Grid falls	Falls/30	Heading err [m/s]
Model-based	lqr (LQR on the derived planar model)	1.04	1.06	0.05	0.42	0.000	25%	18	0.02
S0 baseline limits	ctrl_base_s0 (torque 10, tilt 20 °)	0.10	0.06	0.67	0.04	–	0%	8	–
S1 free limits (exploit)	ctrl_free_c_s0† (torque 30, tilt 70, no ball check)	0.28	0.21	0.61	0.86	–	3%	–	–
S2 on-ball	ballc_free_s0 (+ off-ball termination 30 °)	0.20	0.21	0.56	0.13	–	0%	28	–
S2 on-ball	ballc_freed_s0 (+ larger reward, low fall penalty)	0.23	0.33	0.50	0.27	0.882	0%	30	–
S3 smoothness	smooth_a_s0 (+ action-rate -0.05)	0.15	0.18	0.58	0.14	0.017	0%	27	–
S3 smoothness	smooth_b_s0 (+ action-rate -0.2, 25 °)	0.12	0.13	0.61	0.04	0.002	0%	28	–
S3 smoothness	smooth_c_s0 (scratch, action-rate -0.05)	0.16	0.16	0.59	0.24	0.109	0%	30	–
S4 lean reward	lean_a_s0 (lean bonus 0.5)	0.07	0.10	0.63	0.03	0.007	0%	23	–
S4 lean reward	lean_b_s0 (lean bonus 1.0)	0.30	0.31	0.49	0.49	0.200	3%	20	–
S4 lean reward	lean_c_s0 (scratch, lean 0.5)	0.32	0.36	0.47	0.05	0.359	0%	29	–
S5 error reward	trk_a_s0 (err reward, vmax 1.5, wmax 3)	0.20	0.22	0.55	0.07	0.008	0%	29	–
S5 error reward	trk_b_s0 (no soft ball penalty)	0.14	0.15	0.59	0.05	0.121	0%	27	–
S5 error reward	trk_c_s0 (dot reward, vmax 2)	0.39	0.31	0.51	0.14	0.031	0%	29	–
S6 spin cap	trk_d_s0 (wmax 1, from trk_a)	0.63	0.59	0.34	0.15	0.011	0%	8	–
S6 spin cap	trk_e_s0 (wmax 1, from trk_c)	0.36	0.40	0.44	0.93	0.015	3%	30	–
S6 spin cap	trk_f_s0 (wmax 1, scratch)	0.98	0.91	0.12	0.40	0.147	0%	16	–
S6 spin cap	trk_f_s1 (wmax 1, scratch, seed 1)	1.09	0.95	0.11	0.50	0.082	0%	14	–
S7 refine	trk_g_s0 (trk_f continued)	0.99	0.87	0.12	0.12	0.013	0%	11	–
S7 refine	trk_h_s0 (trk_f + action-rate -0.08)	0.88	0.81	0.15	0.25	0.011	0%	3	0.31
S8 spin curriculum	spin_a_s0 (wmax 1.5, from trk_g)	0.94	0.87	0.19	0.16	0.021	3%	19	0.31
S8 spin curriculum	spin_b_s0 (wmax 1.5, from trk_h)	0.75	0.70	0.21	0.12	0.009	0%	13	0.33
S8 spin curriculum	spin_c_s1 (wmax 1.5, from trk_f_s1)	0.75	0.75	0.21	0.11	0.009	0%	22	0.34
S9 robustness	rob_a_s0 (DR + pushes 5-20 N, from trk_g)	0.34	0.35	0.50	0.13	0.005	5%	4	0.35
S9 robustness	rob_b_s0 (DR + pushes 10-35 N, from trk_g)	0.28	0.27	0.52	0.24	0.004	0%	8	0.37
S9 robustness	rob_a_h_s0 (DR + pushes 5-20 N, from trk_h)	0.30	0.29	0.51	0.51	0.011	25%	9	0.37
S10 residual	res_a_s0 (LQR + 0.3 PPO, vmax 1.5)	0.84	0.43	0.48	0.12	0.007	59%	24	0.31
S10 residual	res_b_s0 (LQR + 0.3 PPO, vmax 1.5, wmax 1.5)	1.00	0.53	0.46	0.07	0.007	52%	24	0.22
S10 residual	res_c_s0 (LQR + 0.1 PPO, zero-init, vmax 1)	1.03	1.05	0.08	0.04	0.000	28%	3	0.02
S10 residual	res_d_s0 (res_c + beta 0.2 + DR + pushes)	1.33	0.75	0.32	0.16	0.001	50%	15	0.13
S11 DR from scratch	robs_a_s0 (full DR, 20M)	0.35	0.37	0.45	0.10	0.003	0%	5	0.34
S11 DR from scratch	robs_b_s0 (damping 0.07-0.15, 20M)	0.50	0.51	0.37	0.14	0.002	0%	3	0.27
S12 + outer integral	robs_a_ki2 (robs_a + velocity integral ki=2)	1.02	0.56	0.26	0.10	0.004	0%	–	0.05
S12 + outer integral	robs_b_ki1 (robs_b + velocity integral ki=1)	1.01	0.79	0.19	0.13	0.003	3%	8	0.06

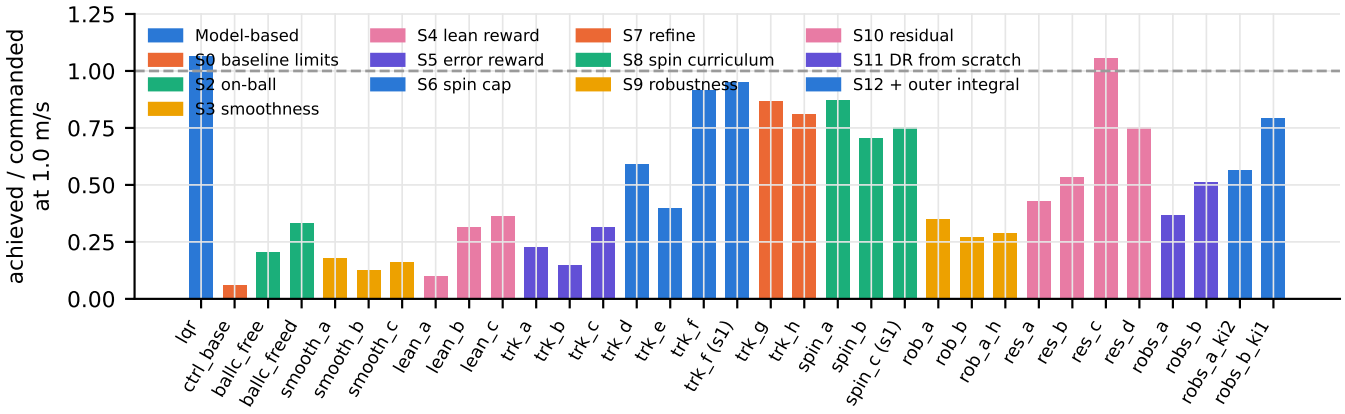


Fig. 1. Achieved/commanded speed at 1 m/s for every valid learned policy, coloured by study stage (dashed: perfect tracking). Reward changes (S2–S5) barely moved the ratio; changing the command distribution (S6) tripled it.

inertias of 0.0005, 0.002 and 0.005 kg m^2 and simulating each. All three track 0.75 m/s without falls and fall in every trial from 1 m/s, reaching 0.84, 0.92–0.95 and 0.99–1.01 m/s respectively before falling. Lighter rotors make the edge slightly earlier, so rotor gyroscopics are not the cause. The remaining candidates are the omniwheel contact model at high sliding speed (the source of the unexplained yaw torque) and ball–floor contact effects; resolving this is the most important open modelling question.

C. Path following

Fig. 5 shows the square course. Velocity tracking does not correct position, so error accumulates (0.35–0.58 m mean distance from the plan). A naive position-feedback outer loop did not help consistently because it pushes commanded speeds toward the stability edge. After a 4 rad turn, the learned policies execute world-frame commands in the wrong direction; training always started at the same heading, so we now randomise it.

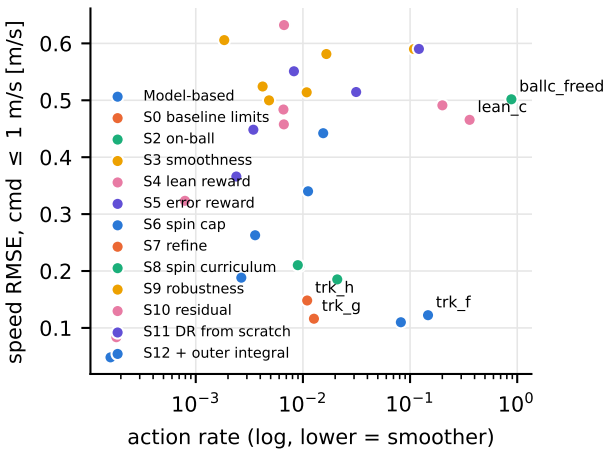


Fig. 2. Smoothness (action rate, log scale) versus tracking error; lower-left is better.

D. Spin-range curriculum

Continuing the S6/S7 policies with $\omega_{\max} = 1.5\,\mathrm{rad/s}$ (S8) improved yaw tracking at $\pm 1\,\mathrm{rad/s}$ (e.g. spin_c: $-0.95/+0.82$ versus $-0.49/+0.51$) but lowered the speed ratio at 1 m/s (0.70–0.87 versus 0.81–0.95) and left the heading error unchanged (0.31–0.34 m/s). Widening the spin range trades translation for yaw, consistent with §VII-C.

E. Robustness: pushes and parameter uncertainty

Table III reports push recovery at 0.5 m/s. On the nominal robot, the learned policies are far more push-robust than the LQR: the LQR falls in 6/10 trials at 40 N and 9/10 at 50 N (a 50 N, 50 ms push is a ≈0.6 m/s velocity disturbance), trk_g in 1/10 and 5/10, trk_h in none. Under randomised physical parameters, the picture reverses: the policies trained on the nominal model fall in 60–80% of runs without any push, while the LQR falls in 40%. A single-parameter ablation on trk_g isolates the cause: randomising wheel damping alone (0.05–0.3 against a nominal 0.1 N m s) produces 5/10 falls, mass 1/10, and friction, torque scale, latency and sensor noise none. In the model, $d_{\mathrm{eff}} = d_w \|\boldsymbol{n}\|^2$ enters the coupling between ball and body directly (Eq. (5)), and it is the parameter hardest to know on hardware (gearbox and motor friction). Fine-tuning trk_g with randomisation and pushes (rob_a) removes the brittleness (0/10 falls up to 30 N under randomised physics, 0/10 up to 50 N nominal) but makes the policy slow again (ratio 0.35 at 1 m/s); stronger randomisation and 10–35 N training pushes (rob_b) give the most robust policy (at most 4/10 falls even at 50 N under randomised physics) at a ratio of 0.27. The timid optimum of §VII-B reappears when a tracking policy is fine-tuned on a harder task. Following Finding 3, we therefore retrain from scratch under randomisation, with the full and with a narrowed damping range (0.07–0.15 N m s, the tolerance a simple bench measurement of the drivetrain would give). Trained from scratch for 20M steps, both policies are robust (0/10 falls at every nominal push up to 50 N; robs_a at most 2/10 at 50 N under randomised physics; 3–5/30 random-command falls) and roughly twice as fast as their fine-tuned counterparts, but their achieved speed is a constant fraction of the command at every speed: 0.35 for robs_a and 0.50 for robs_b. A constant ratio is a steady-state gain below one, not a speed limit.

F. Cascade: an outer velocity integral around the learned policy

A steady-state gain below one is exactly what integral action removes in a classical cascade, and it is how our LQR removes contact creep. We wrap the learned policy in an outer loop that feeds it the corrected command $c'_{xy} = c_{xy} + k_i \int (c_{xy} - v)\,dt$, saturated at the policy's trained range ($\|c'\| \le 1.5\,\mathrm{m/s}$) with the integral frozen while saturated (anti-windup). With $k_i = 1$, robs_b tracks 0.25, 0.5 and 0.75 m/s at 0.25, 0.51 and 0.73 m/s with no falls and 0.79 m/s at a 1 m/s command (12% falls), its heading error drops from 0.27 to 0.06 m/s with no falls (the integral also corrects direction), it recovers from every nominal push up to 50 N, and it remains smooth (action rate 0.003). Under randomised physics with a damping range wider than it was trained on it falls in 2–6 of 10 push trials. With $k_i = 2$, the fully randomised robs_a reaches 0.51 m/s at 0.5 m/s and saturates near 0.57 m/s, with heading error 0.05 m/s and the best randomised-physics push tolerance among fast controllers. Finding 5: a robust but timid learned policy plus a classical outer integral yields a controller that is robust, accurate and heading-invariant in the 0–0.75 m/s range, combining the strengths of both families. Giving the policy the integrated error as an observation (robs_c, two seeds) helps only modestly on its own (ratio 0.54–0.62, heading error 0.19–0.23 m/s): under randomisation, undershooting remains the policy's preferred optimum. Combined with the outer integral, however, robs_c is the most accurate robust controller: 0.25, 0.51 and 0.76 m/s at 0.25, 0.5 and 0.75 m/s with no falls and a heading error of 0.045 m/s.

G. Rough ground: Perlin-noise floors

We evaluate the best controllers, none of which was trained on terrain, on Perlin-noise floors of 4 and 8 cm amplitude. At 0.5 m/s every controller except the nominal-trained PPO policy (trk_h, which slows to 0.32–0.44 m/s with 33–38% heading-test falls) tracks within 0.02 m/s. At 0.75 m/s they separate: the residual policy res_c has no falls on either floor (0.81–0.82 m/s), the LQR falls in 12–25% of trials, and the cascaded robust policies in 12–75%. The residual design, whose LQR prior is heading-invariant and whose learned yaw correction never saw terrain, generalises best to rough ground. Continuing training on random Perlin floors of 0–10 cm did not help:

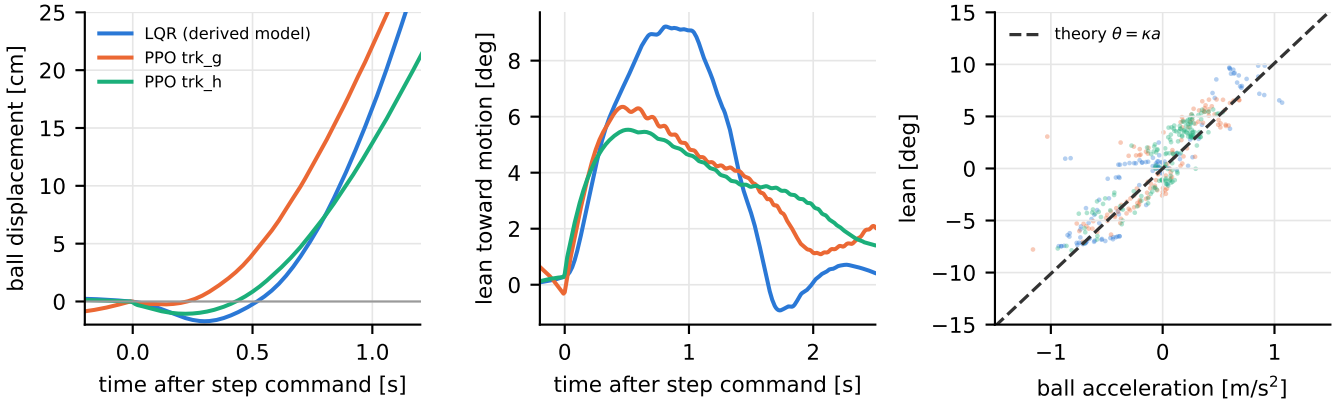


Fig. 3. Theory versus behaviour on a 0.5 m/s forward step. Left: ball displacement, with the non-minimum-phase backward motion before the forward run. Middle: body lean toward the motion. Right: lean versus ball acceleration on step, reversal and stop manoeuvres, with the derived law $\theta = \kappa a$ (dashed).

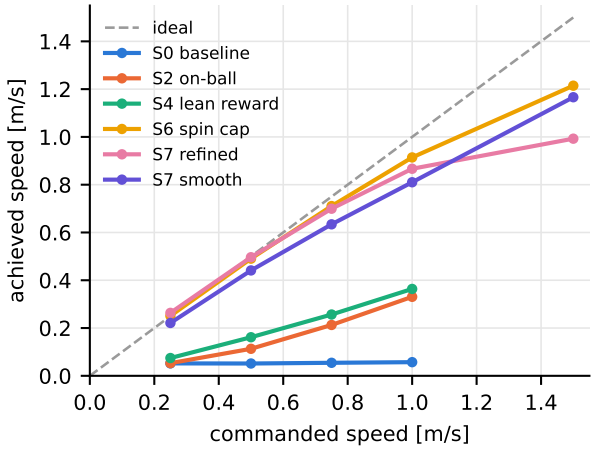


Fig. 4. Achieved versus commanded speed (mean over 8 directions; dashed: ideal).

the terrain-trained residual (res_t) matches res_c (no falls up to 12 cm at ≤0.5 m/s, 17% heading-test falls at 12 cm), and the terrain-trained cascaded policy degrades with amplitude (62–88% falls at 12 cm).

H. Step edges: toward climbing onto a book

We drive each controller straight at a raised floor (a clean step with a 2 cm bevel at $y = 1$ m) from a random start heading, and count a success only if the robot crosses, stops on top and holds its balance for 2 s (10 trials per condition; Fig. 6). None of the controllers was trained on steps, yet all clear 1 cm. At 0.7 m/s the cascaded robs_c climbs 1.5 and 2 cm in 10/10 trials and 3 cm in 3/10; the LQR climbs 2 cm in 10/10 and never 3 cm; the residual res_c degrades from 7/10 at 1.5 cm to 4/10 at 2 cm; the nominal-trained trk_h fails half the trials even on flat ground because of its heading weakness. At 0.4 m/s no controller climbs 2 cm. A speed sweep with the LQR reveals a climb window: the minimum approach speed rises with step height (about 0.3, 0.4, 0.5 and 0.5–0.6 m/s for 1, 1.5, 2 and 2.5 cm), and above ≈0.8 m/s climbing fails again as the 1 m/s instability combines with the impact at the edge. The minimum speed lies below the ballistic bound $\sqrt{2gh}$ because the motors add energy during the climb. A 2–3 cm step, the thickness of a paperback, is therefore within reach of flat-ground controllers if they approach in the window; training for steps explicitly (climb_a, climb_b: 0–4 cm steps, commands toward the step) did not extend this: at 0.7 m/s climb_a clears 2 cm in 8–9/10 and 3 cm in 2/10 trials, no better than the flat-trained robs_c, and climb_b is worse. The limit near 3 cm is therefore physical rather than learned. A static estimate points at traction: the ball's weight share is about 40 N, and climbing a step of height h requires a horizontal force $F \approx W \tan\alpha$ with $\cos\alpha = (r-h)/r$, i.e. 44 N at 3 cm and 60 N at 4 cm, comparable to the total wheel–ball friction available (three wheels carrying 17–18 N each with friction coefficient 1). Scaling the wheel–ball friction supports this: at 0.7 m/s the robs_c cascade climbs 3 cm in 1/4 trials at nominal friction and in 4/4 at 2× and 4×, and 4 cm in 1/4 at 2× (the LQR needs 4× for any 3 cm success at 0.5 m/s). Design implication: a taller step needs a larger ball relative to the step (h/r is 0.33 at 3 cm), more wheel preload, or higher-friction wheels. This contrasts with our earlier book-climbing study at the original actuator envelope, where no policy crossed 1 cm.

I. Residual learning on the LQR

A first residual attempt ($\beta = 0.3$, random initial policy, commands up to 1.5 m/s) was worse than the LQR alone (25–50% falls at 0.5 m/s): the untrained residual destabilised the LQR (initial episodes of ≈1 s against ≈7 s for the LQR alone) and the command range included speeds the LQR cannot survive. The corrected design initialises

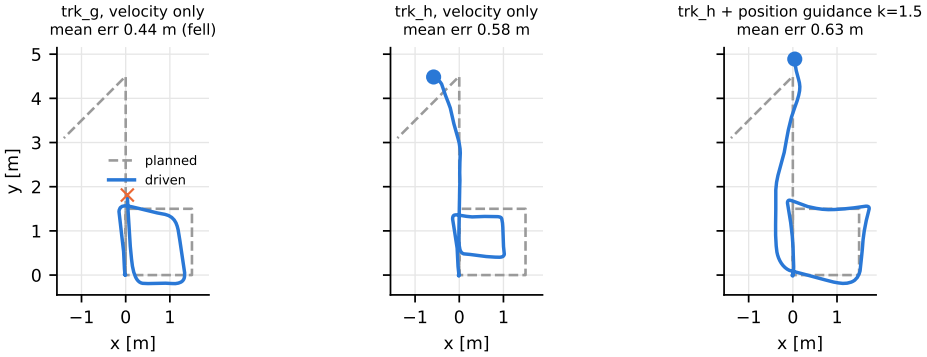


Fig. 5. Top-down path following (dashed: planned; solid: driven; ×: fall). Course: four 1.5 m legs at 0.5 m/s, a 1 m/s leg, a turning leg (0.5 m/s at 1 rad/s) and a diagonal leg.

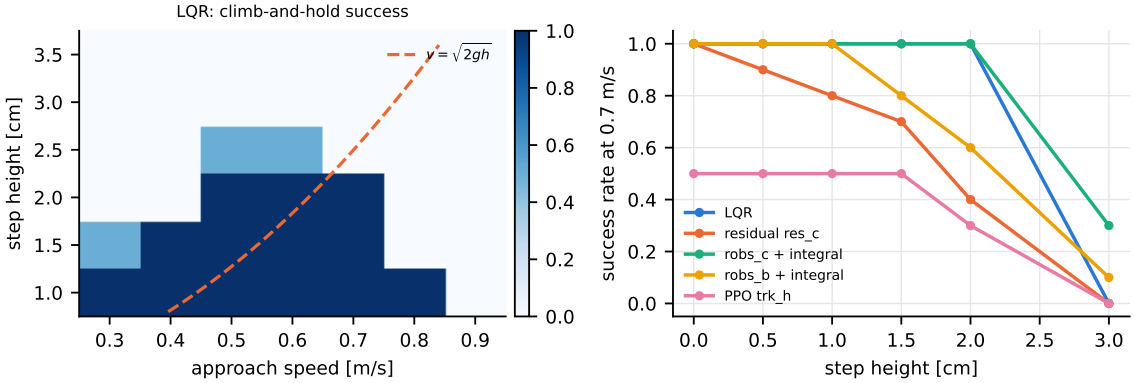


Fig. 6. Step climbing (cross, stop on top, hold 2 s). Left: LQR success over approach speed and step height, with the ballistic reference $v = \sqrt{2gh}$. Right: success at 0.7 m/s from random start headings (10 trials) for each controller.

the policy's output layer to zero (the initial controller is the LQR), uses $\beta = 0.1$ with small exploration noise, and limits commands to 1 m/s. The resulting policy (res_c) keeps every strength of the LQR (heading error 0.02 m/s with no falls, action rate 0.0002, rise time 0.90 s, speed ratio 1.00–1.02 at 0.25–0.5 m/s) and repairs its weakest channel: yaw tracking at ±1 rad/s improves from ±0.57 to $-1.01/+0.93$, the best of any controller, and random-command falls drop to 3/30. It does not, however, extend the LQR's stability envelope: it falls in 19% of 0.75 m/s and 94% of 1 m/s trials, and its push tolerance is lower than the LQR's (6/10 falls at 30 N). With $\beta = 0.1$ the residual has enough authority to shape the yaw channel, where the LQR is merely weak, but not to rescue the balance channel near the stability edge, where the LQR is wrong. Continuing it with $\beta = 0.2$ under randomisation and pushes (res_d) made it worse: it overshoots (0.66 m/s at a 0.5 m/s command), falls in 31% of 0.5 m/s trials and 15/30 random-command episodes, and its push tolerance drops further (8/10 falls at 30 N), although its heading error stays low (0.13 m/s). Larger residual authority lets the policy fight the LQR rather than complement it.

IX. GPU-Resident Training: Setup

The study so far used one simulator process per run. To train one policy across terrains, floor materials and robot designs we moved the whole learning loop onto the GPU and then asked, in order, what limits such a policy and what structure removes the limit.

A. Engines and validation

The reference engine is MuJoCo (CPU, float64, implicit-fast integrator, with the contact-frame correction of §III). The training engine is MuJoCo-Warp 3.17 (GPU, float32) with the same correction as a custom kernel inside a CUDA graph. Before any policy was trained on it, an LQR on 17 designs gave the same fall or no-fall outcome in 94.6 % of speed tests (193 of 204) and 96.1 % of push tests (784 of 816), with speed ratios of non-falling runs within 0.01; one CPU process reaches 1.36k control steps/s and Warp 355k at 4096 worlds, a factor of about 260 (engine report experiments/mjx_port/REPORT.md). Warp is not bit-reproducible and individual trajectories spread more than on the CPU. Observation, reward, resets, command sampling, pushes and PPO all run on the GPU through torch views of the Warp arrays;

TABLE III
Push recovery while driving at 0.5 m/s: falls out of 10 trials with a single 50 ms push from a random direction at $t = 2$ s (any fall counts). Top: nominal robot. Bottom: physical parameters randomised as in training.

Controller	0 N	10 N	20 N	30 N	40 N	50 N
LQR (derived)	0	0	0	0	6	9
PPO trk_g	0	0	0	0	1	5
PPO trk_h	0	0	0	0	0	0
PPO rob_a (DR-trained)	0	0	0	0	0	0
PPO rob_b (strong DR)	0	0	0	0	0	0
PPO robs_a (DR, scratch)	0	0	0	0	0	0
PPO robs_b (narrow DR, scratch)	0	0	0	0	0	0
Residual res_c	0	0	0	6	9	7
Residual res_d	0	0	3	8	9	10
robs_a + integral	0	0	0	0	0	0
robs_b + integral	0	0	0	0	0	0
randomised physics						
LQR (derived)	4	4	4	5	8	8
PPO trk_g	7	8	8	8	8	8
PPO trk_h	6	6	5	6	5	6
PPO rob_a (DR-trained)	0	0	0	0	1	3
PPO rob_b (strong DR)	0	1	0	1	2	4
PPO robs_a (DR, scratch)	0	0	0	0	0	2
PPO robs_b (narrow DR, scratch)	1	2	2	1	4	5
Residual res_c	4	4	6	7	7	10
Residual res_d	0	0	3	4	8	10
robs_a + integral	0	1	0	3	3	5
robs_b + integral	4	3	2	4	5	6

removing host synchronisations (29 to 1 per control step) raised an end-to-end 12M-step run from 52k to 67k environment steps/s with the same learning curve. Policy-transfer checks between the engines were made before the correction described below and are not reported. One quirk of the original environment is kept on purpose: its motor_state input reads qvel[joint.id], which indexes the base free joint, so the policies receive scaled base-velocity components in those three slots and never the wheel rates; a first GPU port that read the true wheel rates failed on the CPU engine until the quirk was copied.

a) Engine pitfall: stale subtree mass.: In a batched MuJoCo-Warp model with per-world masses the derived body_subtreemass must be recomputed too: the engine locates each subtree's centre of mass by dividing mass-weighted positions by it and uses that point as the dynamics reference, so a stale value shifts the reference by a fraction of the distance from the world origin (our tiles lie up to 12 m away) and the error vanishes at the nominal mass. Our code randomised the base and ball masses but not body_subtreemass, so every run and evaluation with randomisation on was affected until the fix; randomisation off was not. Evidence (LQR alone, clean flat world, 64 worlds, falls at 0.3/0.6 m/s): nominal 0.12/0.16; ball-mass randomisation 0.41/0.81 before and 0.17/0.23 after the fix; a ball mass fixed at 0.63 to 2.0 times nominal gave 0.62 to 1.00 before. Planar-model checks and changes of the integral gain, control cost and ball inertia did not remove it, whereas recomputing the constants for one changed mass did. The fix recomputes the subtree masses from the per-world masses at every reset. All randomised results in this paper come from runs made after the fix.

B. The model-based controller in discrete time

A continuous LQR with light rotors has a pole near $-135\,\mathrm{s}^{-1}$ at 100 Hz and is unstable when sampled, so the controllers used with the GPU stack are designed in discrete time at the control period $T = 0.01$ s. With (A_d, B_d) the zero-order-hold discretisation of (A, B_s) and the same weights (QT, RT), the discrete Riccati solution gives $K_d = (RT + B_d^\top X B_d)^{-1} B_d^\top X A_d$. For the nominal robot $K_d = (-0.001,\ -73.87,\ -4.21,\ -11.23)$ and the continuous gain is $K = (-0.001,\ -76.14,\ -4.33,\ -11.58)$ with $\tau = -K(s - s_{\mathrm{ref}})$ (the entries are negative because B_s has a positive torque convention on φ and a negative one on θ; §V-A quotes the magnitudes). The reference governor, the slow integral on the measured velocity and the yaw loop are as in §V-A; a batched version runs on the GPU with one gain per design.

C. Learning problem as coded

Observation. The policy sees 25 numbers: orientation rotation vector (3), body angular velocity (3) and linear velocity (3), the motor_state slots described above (3), the previous action (3), the command (v_x, v_y, ω_z) (3), sine and cosine of the body yaw (2), the velocity-error integral $\xi_v \leftarrow \mathrm{clip}(\xi_v + (c - v)\,0.01, \pm 1)$ (2) and the goal error divided by 2 and clipped (2) with a goal flag (1). Optional extras are a stack of the last $K = 8$ pairs of policy-visible observation and action, a 6-number design vector and a sensor vector. Gaussian noise ($\sigma = 0.01$; 0.25σ, 2σ, σ on orientation, angular and linear velocity) is added to the first nine slots when randomisation is on. With history 8 and the design vector the main runs see 255 numbers. The action is $a \in [-1, 1]^3$ with wheel torque $\mathrm{clip}(\kappa\tau_d a, \pm\tau_d)$, τ_d the design's torque limit and $\kappa \sim \mathcal{U}(0.85, 1.15)$.

Commands and goals. Velocity commands are resampled every 300 control steps: zero with probability 0.1, otherwise speed $\mathcal{U}(0, 1.2)\sqrt{r_{\mathrm{ball}}/0.09}$ m/s, direction $\mathcal{U}(0, 2\pi)$, yaw rate $\mathcal{U}(-1, 1)$ rad/s; episodes last 800 steps. In goal mode the command is the saturating proportional rule

$$c_{xy} = \min\left(k_g\|g-p\|,\ v_{\max} f\right)\frac{g-p}{\|g-p\|}, \quad k_g = 1.5\,\mathrm{s}^{-1}, \quad c_{xy} = 0 \text{ if } \|g- \tag{15}$$

so speed is free up to a cap while the tracking reward still applies. Goals are drawn 0.7 to 1.8 m away (within 1.3 m of the tile centre); a world with waypoints left draws the next goal on arrival within 0.2 m. With cross-focus a goal is, with probability 0.6, placed 1.1 to 1.3 m from the tile centre along a tile axis, so reaching it means crossing the terrain feature.

Reward. With $e = \|v - c_{xy}\|$, $\hat{c} = c_{xy}/\|c_{xy}\|$, $p = v \cdot \hat{c}$, $e_\omega = |\omega_z - c_\omega|$, $\ell = \Delta_{xy} \cdot \hat{c}/\Delta_z$ the lean along the command (Δ the body position minus the ball centre) and $\beta = \mathrm{atan2}(\|\Delta_{xy}\|, \Delta_z)$ the ball offset, the reward of

train_dev.py is

$$\text{raw} = (\|c\| - e) + \tfrac{1}{2}e^{-e^2/0.25^2} + \tfrac{1}{2}(|c_\omega| - e_\omega) + \tfrac{1}{4}e^{-e_\omega^2/0.5^2} + \tag{16}$$

$$r_t = 2.5\, w_{\text{trk}}\, \text{raw} - w_{\text{pen}}\Big[5{\times}10^{-4}\|a_t\|^2 + 0.1\|a_t - a_{t-1}\|^2 + \tag{17}$$

The weight w_{trk} multiplies the whole bracket (1, 2 or 4 in our runs) and w_{pen} the three penalties. A fall is a tilt above 70°, $\beta > 30°$ or a non-finite state; leaving the 2.9 m tile square truncates the episode without penalty, and time-limit truncations bootstrap the value.

PPO. A diagonal Gaussian policy with a state-independent log standard deviation (initial −1) and a tanh MLP (three hidden layers of 512 units in the main runs). The loss is the clipped surrogate plus a value term and no entropy bonus:

$$\mathcal{L} = -\mathbb{E}\big[\min(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 0.8, 1.2)\hat{A}_t)\big] + 0.25\,\mathbb{E}\big[(V_\phi - \hat{R}_t)^2\big], \tag{18}$$

with $\gamma = 0.995$, $\lambda = 0.95$, 4096 worlds × 64 steps per iteration, 5 epochs of 8 minibatches, Adam with learning rate $3{\times}10^{-4} \to 3{\times}10^{-5}$ and gradient-norm clip 0.5, advantage normalisation per minibatch and an early stop at approximate KL 0.03. Rewards are divided by the running standard deviation of the discounted return and clipped at 10; observations are normalised by running statistics. The actor sees only deployable inputs; the asymmetric critic also sees floor friction and softness, base and ball mass offsets, torque scale, wheel damping, a 34-number height scan and the design vector.

D. LQR plus a bounded learned residual

The environment applies $a = \text{clip}(a_{\text{LQR}} + r_s\, a_\pi, \pm 1)$ with a_π the policy output and r_s the residual authority; the LQR runs inside the environment at every control step from the simulator state with one gain set per design, and latency, torque scaling and pushes act on the sum. The output layer starts near zero so training starts from the plain LQR. Fig. 7 sketches the architecture. An interleaved gate $a = (1-g)a_{\text{LQR}} + g\, a_\pi$ was tried on the CPU engine and collapsed to the pure LQR.

E. Terrain, floors, designs and randomisation

MuJoCo-Warp shares one height field across worlds, so terrain variety is a 24 m map of 8×8 tiles of 3 m (3.4 cm cells, $h = B + g(x) + g(y)$ with features at least 0.5 m from the tile centre so that any heading meets them; Fig. 8, Table IV). A world picks a tile and a start within ±0.3 m of its centre; severity is drawn once per tile and the map is generated with a fixed seed, so training and evaluation share the same 64 tiles. Seven floor classes (Table V) set the ball-floor friction and contact time constant (plausible, not measured). Thirteen designs share one topology and their differing model fields are batched per world (Table VI); eleven form the training set, after a per-design fall breakdown showed that two (half scale and very light rotors) were effectively unlearnable. Table VII lists the randomisation.

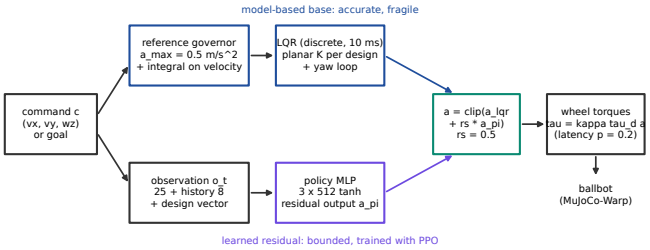


Fig. 7. Architecture of the LQR plus residual controller (schematic, no data). Blue: model-based path using the simulator state; violet: learned path using the noisy observation; teal: the sum with authority r_s. Engine: Warp GPU. Randomisation and noise act as in Table VII.

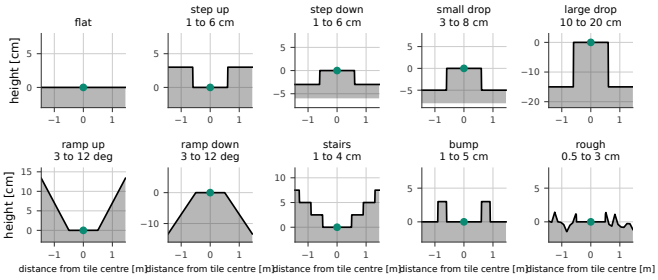


Fig. 8. Terrain tile library: cross-section through the tile centre (dot) at a representative severity; titles give the training range. Geometry only: no engine, randomisation, noise, worlds or seeds apply (map seed 0).

F. Evaluation protocols and metrics

All GPU evaluations use a deterministic policy (the Gaussian mean), a fixed seed and no command resampling; the first fall of a world freezes it. (i) Clean flat benchmark: a perfectly flat 24 m world (interior tiles only, at least 7.5 m from the border: the first version used border tiles, and with tile exits disabled a robot driving outward left the height field and was counted as a fall; a per-fall position record showed that most recorded flat falls were such drop-offs, the benchmark was fixed and the flat rows of every battery were re-run; the error and achieved-speed metrics, computed over surviving worlds, are unaffected by the artefact), nominal design, randomisation (hence noise, latency and pushes) off, a constant command of 0.3, 0.6 or 0.9 m/s at a random heading, 800 steps; 64 worlds per row (two nominal-design copies of 32 random start poses and headings; 32 for the LQR calibration). Steady-state error $\|v - c\|$ and achieved/commanded speed $v \cdot c/\|c\|^2$ are averaged over steps 300 to 799 of worlds still upright; the fall rate is the share of worlds that fell at any time. (ii) Velocity task on the tiles: 0.3 m/s at a random heading,

TABLE IV
Terrain tile library (geometry only; map 24 m, 3 m tiles, truncation outside a 2.9 m square).

Terrain kind	Severity range	Profile	Tiles of 64
flat	none	no feature	7
step up	1 to 6 cm	single raised step 0.6 m from the centre on each axis	7
step down	1 to 6 cm	robot stands on a plateau, floor drops by the severity	7
small drop	3 to 8 cm	drop of 3 to 8 cm	7
large drop	10 to 20 cm	drop of 10 to 20 cm	6
ramp up	3 to 12 deg	slope over 1.0 m starting 0.5 m from the centre	6
ramp down	3 to 12 deg	descending slope over 1.0 m	6
stairs	1 to 4 cm	2 to 3 treads of 0.4 m, riser = severity	6
bump	1 to 5 cm	slab 0.3 m wide, 0.6 m from the centre	6
rough	0.5 to 3 cm	smooth two-octave noise outside a flat 0.5 m start zone	6

TABLE V
Floor material classes (plausible, not measured).

Floor class	Friction μ	Contact time constant [s]
concrete	0.9	0.020
hardwood	0.6	0.020
carpet	1.1	0.045
grass	0.8	0.050
gravel	0.7	0.030
wet tile	0.3	0.020
rubber	1.3	0.020

11 designs × 10 terrain kinds × 16 repeats (1760 worlds), randomisation and floor classes on; undisturbed means pushes and pulls off, disturbed means pushes of 5 to 20 N (scaled per design) and pulls of 3 to 12 N at full strength. (iii) Goal task: a goal 1.1 m away at a random heading, the same layout; success is reaching within 0.2 m before any fall, with the median time over successful worlds and success per terrain kind (176 worlds). (iv) Disturbance scaling: pushes and pulls multiplied together by 0 to 1. With 1760 worlds a fall rate near 0.5 has a binomial standard error of about 0.012, a terrain kind about 0.04 and a 64-world row about 0.06; the seed-to-seed spread of a recipe is about 0.03 to 0.06 (Fig. 12). Every table and figure states engine, randomisation, noise, worlds per cell, seeds and protocol.

X. GPU Generalist: Results

All randomised results in this section come from runs trained and evaluated on the corrected engine (§IX-A); earlier randomised results are superseded and omitted. Evaluations with randomisation off were never affected. Recipes (all 11 designs, 200M steps unless stated): pure learning (no LQR, 300M steps); LQR + residual with authority 0.5 at the standard randomisation ranges (A, two seeds); the same with ranges ramped to 1.75 times over the first half of training (B, two seeds); a soft LQR base (control cost ×3, reference acceleration 0.3 m/s^2) with the 1.75 ramp (C, two seeds) and with authority 0.75 (one seed).

A. Baselines on the CPU engine

Table VIII repeats the CPU results that carried forward (the full list is Table II): the LQR is accurate and fragile, the randomised-from-scratch PPO policy is robust and timid, and a small-authority residual on the LQR keeps the LQR's precision.

B. Tracking accuracy and the LQR calibration (clean flat world)

On the clean flat benchmark (Table IX, Fig. 9) the pure-learning policy reaches 0.60, 0.58 and 0.55 of the commanded speed at 0.3, 0.6 and 0.9 m/s (error 0.130 to 0.406 m/s). The batched GPU LQR on the same worlds (re-measured on interior tiles after the map-edge correction, 64 worlds; S92 addendum) tracks exactly up to 0.6 m/s (error 0.014 and 0.017 m/s, ratio 1.00, no falls), 9.29 to 15.63 times more accurate than pure learning, so the shortfall is a policy deficiency and not a limit of the robot; at 0.9 m/s the LQR alone falls in 0.91 of worlds, its speed wall near 1 m/s, which the learned residual removes (the residual controllers fall 0.00 to 0.02 there at the same accuracy). The residual keeps the LQR's accuracy (error 0.031 to 0.064 m/s on seed 0 and 0.025 to 0.048 on seed 1; 5 to 10 % of the command) with fewer falls at 0.9 m/s; the flat 0.9 m/s fall rate differs between seeds (0.02 and 0.00; 64 worlds, standard error about 0.05). The fall column of Table IX is the re-run on interior tiles after the map-edge correction (§IX-F); the recommended controller does not fall at any speed in it. The LQR-alone row is the interior-tile re-measurement of the same addendum.

C. Goal task and velocity task on the tiles

On the goal task (Table X, Fig. 10) the residual (A, two seeds) succeeds in 0.58 of undisturbed worlds in a median 2.52 s; pure learning in 0.48 in 3.02 s, and the soft base in 0.63 in 3.09 s. By kind the residual reaches the goal on ramps up in 0.72 of worlds, on flat tiles 0.83, stairs 0.38, step ups 0.31 and bumps 0.18. With pushes and pulls success drops to 0.25 to 0.34 for every recipe, pure learning included (0.27). On the velocity task at 0.3 m/s on the tiles (Table XI) the residual falls in 0.55 of worlds undisturbed and 0.73 with pushes and pulls, and so do the ramped and soft recipes; pure learning has fewer falls (0.35, 0.55) but tracks at about 0.63 of the command on the flat world, so this is not a robustness advantage.

D. Soft base, authority and seed noise

The soft base gives a small, consistent gain in goal success (+0.05 undisturbed, +0.08 with pushes and pulls at scale 1) at the cost of speed (3.09 s against 2.52 s) and a little flat accuracy at 0.9 m/s (error 0.110 against 0.056 m/s). Authority 0.75 on the soft base is no better (goal success 0.58) and tracks worse on the flat floor (error 0.188, falls 0.02 at 0.9 m/s; Fig. 11). Seed replicates agree:

TABLE VI
The 13 compiled designs (eleven used in training).

Design	What it changes	Body mass [kg]	Ball mass [kg]	COM height l [m]	Rotor inertia [kg m^2]	Torque limit [N m]	Contact zenith [deg]	In training set
nom	reference robot	3.85	0.168	0.131	0.005	30	51	yes
zhigh	ballast raised: COM height about 1.9 times nominal	3.85	0.168	0.254	0.005	30	51	yes
zlow	ballast lowered: COM height about 0.67 times nominal	3.85	0.168	0.087	0.005	30	51	yes
kb4	ball mass x4	3.85	0.672	0.131	0.005	30	51	yes
bal4	body ballast x4 (body mass 3.5 times nominal, pushes scaled)	13.45	0.168	0.124	0.005	30	51	yes
J02	wheel rotor inertia x4	3.85	0.168	0.131	0.02	30	51	yes
tau6	torque limit 6 N m (one fifth)	3.85	0.168	0.131	0.005	6	51	yes
bal025	ballast x0.25 (body mass 0.38 times nominal)	1.45	0.168	0.145	0.005	30	51	yes
kb025	ball mass x0.25	3.85	0.042	0.131	0.005	30	51	yes
zeta41	wheel contact zenith 41 deg (nominal 51)	3.85	0.168	0.132	0.005	30	41	yes
zeta66	wheel contact zenith 66 deg	3.85	0.168	0.129	0.005	30	66	yes
s0.5	half scale (all lengths x0.5, masses x0.125)	0.48	0.021	0.065	0.0001563	1.875	51	no (dropped)
J0005	rotor inertia x0.1	3.85	0.168	0.131	0.0005	30	51	no (dropped)

TABLE VII
Domain randomisation ranges (configs/env/dz_nom.yaml and train_dev.py).

Quantity	Range	When	Notes
Robot design	one of 11 designs, uniform	per episode	Table C
Base mass and ball mass	each times (1 plus uniform plus or minus 0.15)	per episode	independent draws
Wheel-ball transverse friction	times (1 plus uniform plus or minus 0.20)	per episode	the cross-roller direction of the anisotropic pair
Wheel damping	uniform 0.07 to 0.15 N m s times the design's nominal damping over 0.1	per episode, each wheel	master scale widens the spread around 0.1
Torque scale	1 plus uniform plus or minus 0.15	per episode	applied to the commanded torque
Action latency	with probability 0.2 the previous action is applied instead	per control step	probability capped at 0.6 when scaled
Observation noise	Gaussian, sigma 0.01 (orientation 0.25 sigma, angular velocity 2 sigma, linear velocity 1 sigma)	per control step	slots 0 to 8 only
Pushes	horizontal force 5 to 20 N (times the design push factor), random direction, 5 control steps (50 ms)	every 2 to 4 s	factor = push range of the design over 20 N
Pulls	horizontal force 3 to 12 N, random direction, 0.5 to 2 s	every 4 to 10 s	only in runs with pulls on; not scaled per design
Floor material	one of 7 classes; friction times (1 plus uniform plus or minus 0.15)	per episode	Table B
Terrain tile and start	uniform over 64 tiles; start offset uniform plus or minus 0.3 m; start yaw uniform over a full turn	per episode	Table A
Master scale (wide runs)	multiplies mass, friction, torque, damping spread, latency probability and noise ranges; 1 to 1.75 over the first half of training	per iteration	does not scale pushes or pulls

goal success differs by at most 0.02 at every randomisation scale and with or without disturbances (Fig. 12).

E. Robustness to randomisation and disturbances

Evaluated at randomisation scales 1, 1.75 and 2.5 (Table XII, Fig. 13), goal success falls from 0.58 to 0.45 for the standard-range policy; training over the 1.75 times ranges changes it by at most 0.02 undisturbed and 0.03 with pushes and pulls, within seed noise, so the standard-range policy is already robust to wider parameter ranges, and pure learning degrades alike (0.48 to 0.38). What no recipe changes is the success under pushes and pulls (about 0.3) and the tile velocity falls. Three disturbance-focused one-variable changes (fall penalty 5 on the ramped recipe, fall penalty 5 on the soft base, push and pull strength ramped from 0.2 to 1 in training) were also evaluated: none improved robustness under pushes and pulls (goal success 0.27, 0.33 and 0.25 at scale 1 against 0.29 for the ramped recipe and 0.34 for the soft base); the push curriculum is slightly worse. On a held-out terrain layout (tile seed 1, Table XIII) the ramped recipe reaches 0.52 and 0.53 undisturbed and 0.26 and 0.26 disturbed on two seeds, a small drop from the trained map. [TBD: privileged height scan on the corrected engine and second seeds of the perception arms.]

F. Why it falls, and a ball retainer

a) Fall diagnosis.: The simulated robot is two free bodies, the base (with the wheels) and the ball, held together only by gravity and the three wheel-ball contacts. Logging every world at every step for the ramped policy (Table XIV) shows that the motors do not run out of

TABLE VIII

Baseline controllers on the CPU engine (selected rows of Table II). Engine: CPU (MuJoCo 3.13, float64, implicit-fast, contact-frame fix). Domain randomisation: off (nominal robot). Observation noise: off. Worlds per cell: 1 robot per condition, 2 repeats per speed and direction (8 directions); 30 random-command episodes. Seeds: 1 per row unless two are listed. Protocol: scripts/study/eval_control.py and eval_falls.py, deterministic policies. The 1.0 m/s ratio of the LQR averages trials that mostly fall; the LQR falls in every 1 m/s trial of the study log.

Controller	Achieved/commanded at 0.5 m/s	at 1.0 m/s	Velocity RMSE, commands up to 1 m/s [m/s]	Heading-test error [m/s]	Falls in 30 random-command episodes
LQR on the derived model (lqr)	1.04	1.06	0.05	0.02	18/30
PPO, spin cap 1 rad/s, from scratch (trk_f_s0)	0.98	0.91	0.12	n/a	16/30
same, seed 1 (trk_f_s1)	1.09	0.95	0.11	n/a	14/30
trk_f continued, smoother (trk_g_s0)	0.99	0.87	0.12	n/a	11/30
PPO fine-tuned under DR, pushes 10 to 35 N (rob_b_s0)	0.28	0.27	0.52	0.37	8/30
PPO from scratch under DR (narrow damping) (robs_b_s0)	0.50	0.51	0.37	0.27	3/30
robs_b + outer velocity integral (robs_b_ki1)	1.01	0.79	0.19	0.06	8/30
LQR + residual (beta 0.1, zero-init) (res_c_s0)	1.03	1.05	0.08	0.02	3/30
res_c continued, beta 0.2, DR + pushes (res_d_s0)	1.33	0.75	0.32	0.13	15/30

TABLE IX

Clean flat-world velocity-tracking benchmark, corrected engine. Cells: steady-state error [m/s] / achieved over commanded speed / fall rate. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: off. Observation noise: off. Worlds per cell: 64 per cell. Seeds: one per row as listed (seeds 0 and 1 where both are shown). Protocol: perfectly flat 24 m world on interior tiles (at least 7.5 m from the map border, the S92 correction), nominal design, constant velocity command at a random heading, 800 control steps, error and achieved/commanded speed averaged over steps 300 to 799 of worlds still upright, falls = share of worlds that fell (eval_battery.py; every flat row re-run after S92). Error is the Euclidean velocity error in m/s. All learned rows are corrected-engine runs. The LQR-alone row is the interior-tile re-measurement of the S92 addendum (source: lqr_flat_interior.log); a dagger marks values taken from the log.

Controller	0.3 m/s: err / achieved / falls	0.6 m/s	0.9 m/s	Seed
LQR alone	0.014 / 1.00 / 0.00	0.017 / 1.00 / 0.00	0.026 / 1.00 / 0.91	n/a (no learning)
pure learning (no LQR), 300M steps (seed 0)	0.130 / 0.60 / 0.00	0.260 / 0.58 / 0.00	0.406 / 0.55 / 0.00	seed 0
LQR + residual 0.5, standard randomisation (seed 0)	0.031 / 0.93 / 0.00	0.056 / 0.94 / 0.00	0.064 / 0.98 / 0.02	seed 0
LQR + residual 0.5, standard randomisation (seed 1)	0.025 / 0.95 / 0.00	0.038 / 0.97 / 0.00	0.048 / 0.99 / 0.00	seed 1
LQR + residual 0.5, randomisation ramped to 1.75x (seed 0)	0.027 / 0.95 / 0.00	0.046 / 0.97 / 0.00	0.057 / 0.99 / 0.02	seed 0
LQR + residual 0.5, randomisation ramped to 1.75x (seed 1)	0.013 / 1.00 / 0.00	0.021 / 1.00 / 0.00	0.041 / 1.02 / 0.00	seed 1
soft LQR base (R x3, a_max 0.3), ramped to 1.75x (seed 0)	0.032 / 0.91 / 0.00	0.068 / 0.93 / 0.00	0.098 / 0.95 / 0.00	seed 0
soft LQR base (R x3, a_max 0.3), ramped to 1.75x (seed 1)	0.039 / 0.90 / 0.00	0.084 / 0.92 / 0.00	0.123 / 0.94 / 0.00	seed 1
soft LQR base, authority 0.75 (seed 0)	0.050 / 0.86 / 0.00	0.136 / 0.81 / 0.00	0.188 / 0.84 / 0.02	seed 0
ramped 1.75x, fall penalty 5 (was 2) (seed 0)	0.027 / 0.95 / 0.00	0.044 / 0.98 / 0.02	0.062 / 1.01 / 0.00	seed 0
soft base, fall penalty 5 (seed 0)	0.025 / 0.95 / 0.00	0.053 / 0.96 / 0.00	0.102 / 0.96 / 0.02	seed 0
ramped 1.75x, push and pull strength ramped 0.2 to 1 (seed 0)	0.026 / 0.96 / 0.00	0.045 / 0.97 / 0.00	0.061 / 1.00 / 0.02	seed 0
ramped 1.75x + edge starts + depth sensor (seed 0)	0.014 / 1.00 / 0.00	0.027 / 0.99 / 0.00	0.044 / 1.00 / 0.02	seed 0
ramped 1.75x + edge starts, blind control (seed 0)	0.019 / 0.99 / 0.02	0.033 / 0.99 / 0.00	0.046 / 1.00 / 0.00	seed 0
pure learning (no LQR), 300M steps (seed 1)	0.103 / 0.67 / 0.11	0.246 / 0.59 / 0.00	0.421 / 0.54 / 0.00	seed 1
ramped 1.75x, trained with the ball retainer (arrestors at 100 deg) (seed 0)	0.049 / 0.86 / 0.00	0.140 / 0.81 / 0.00	0.188 / 0.86 / 0.00	seed 0
ramped 1.75x, trained with the ball retainer (arrestors at 100 deg) (seed 1)	0.048 / 0.87 / 0.00	0.096 / 0.87 / 0.00	0.115 / 0.91 / 0.00	seed 1
pure learning, trained with the retainer (seed 0)	0.087 / 0.73 / 0.00	0.196 / 0.69 / 0.00	0.335 / 0.64 / 0.00	seed 0
pure learning, trained with the retainer (seed 1)	0.060 / 0.82 / 0.02	0.172 / 0.72 / 0.00	0.325 / 0.65 / 0.02	seed 1
retainer, authority 0.5, tracking weight x3 (seed 0)	0.039 / 0.88 / 0.00	0.177 / 0.74 / 0.00	0.201 / 0.84 / 0.00	seed 0
retainer, authority 0.15 (seed 0)	0.016 / 0.99 / 0.00	0.023 / 1.00 / 0.00	0.042 / 1.01 / 0.02	seed 0
retainer, authority 0.25 (recommended) (seed 0)	0.014 / 0.99 / 0.00	0.027 / 1.00 / 0.00	0.036 / 1.01 / 0.00	seed 0
retainer, authority 0.25 (recommended) (seed 1)	0.024 / 0.98 / 0.00	0.033 / 0.98 / 0.00	0.034 / 1.00 / 0.00	seed 1
retainer, authority 0.35 (seed 0)	0.034 / 0.91 / 0.00	0.052 / 0.94 / 0.00	0.060 / 0.98 / 0.00	seed 0
retainer, soft LQR base, authority 0.5 (seed 0)	0.074 / 0.76 / 0.00	0.196 / 0.69 / 0.00	0.266 / 0.72 / 0.00	seed 0
retainer, authority 0.25 (recommended) (seed 2)	0.039 / 0.90 / 0.00	0.043 / 0.97 / 0.00	0.049 / 0.99 / 0.00	seed 2
retainer, authority 0.25, 400M steps (seed 0)	0.065 / 0.92 / 0.02	0.059 / 1.00 / 0.02	0.058 / 1.01 / 0.00	seed 0
retainer, authority 0.25, depth sensor + edge starts (seed 0)	0.021 / 0.96 / 0.02	0.040 / 0.96 / 0.02	0.051 / 0.99 / 0.02	seed 0
retainer, authority 0.25, 12 training designs (adds tau3; 1920 worlds per goal row) (seed 0)	0.027 / 0.95 / 0.00	0.036 / 0.98 / 0.00	0.033 / 1.00 / 0.02	seed 0
retainer, authority 0.25, 12 training designs (adds tau3; 1920 worlds per goal row) (seed 1)	0.033 / 0.94 / 0.00	0.039 / 0.99 / 0.00	0.041 / 1.01 / 0.00	seed 1
retainer, authority 0.25, privileged height scan input (seed 0)	0.030 / 0.92 / 0.00	0.037 / 0.97 / 0.00	0.054 / 1.00 / 0.00	seed 0
retainer, authority 0.5, privileged height scan input (seed 0)	0.036 / 0.90 / 0.00	0.079 / 0.87 / 0.00	0.104 / 0.89 / 0.00	seed 0
pure learning, retainer, privileged height scan input (seed 0)	0.078 / 0.76 / 0.00	0.195 / 0.68 / 0.00	0.356 / 0.61 / 0.02	seed 0
recommended recipe on the corrected contact model (free-rolling friction 0.01; recommended checkpoint) (seed 0)	0.041 / 0.90 / 0.00	0.046 / 0.96 / 0.00	0.043 / 0.99 / 0.00	seed 0
recommended recipe on the corrected contact model (free-rolling friction 0.01; recommended checkpoint) (seed 1)	0.054 / 0.85 / 0.00	0.047 / 0.95 / 0.00	0.048 / 0.99 / 0.00	seed 1
recommended recipe on the corrected contact model (free-rolling friction 0.01; recommended checkpoint) (seed 2)	0.034 / 0.93 / 0.00	0.040 / 0.98 / 0.00	0.044 / 1.00 / 0.00	seed 2
recommended recipe, trained with the planner in the loop (30 % of worlds on blocks tiles) (seed 0)	0.053 / 0.84 / 0.00	0.055 / 0.95 / 0.00	0.050 / 0.99 / 0.00	seed 0
recommended recipe, trained with the planner in the loop (30 % of worlds on blocks tiles) (seed 1)	0.030 / 0.92 / 0.00	0.036 / 0.98 / 0.00	0.046 / 1.01 / 0.02	seed 1

torque (wheels at the limit for 0.74 % of the last second before an undisturbed fall, 0.41 % with pushes and pulls) and that the robot rarely tips past 70° (0.95 of undisturbed falls end at the ball-offset limit). In 0.78 of undisturbed falls the body first derails off the ball (the base axis turns more than 5° away from the ball-centre-to-base direction, against 0.1 to 0.3° when seated) at a median tilt of only 14.65°; survivors reach about 11.57° without derailing.

b) Retainer design.: The idea is taken from the arrestors of ETH Zurich's Rezero ballbot, which according to its project coverage hold the ball against the wheels; no design details were published, so the design is our own. Three passive spherical arrestors of radius $r_a = 8\,\mathrm{mm}$ are fixed to the base, one per wheel on the azimuth opposite that wheel, at polar angle ϕ from the top of the ball, with centres $\boldsymbol{a}_k = \boldsymbol{c} + (R + r_a)\,\boldsymbol{u}_k(\phi)$ computed at the loaded seat (the model settled under gravity, about 1 cm above the reset pose), so they touch the ball without preload, carry no force while the robot is seated, and act only when the body starts to lift or slide (Fig. 14). A frictionless sphere-sphere contact force acts along the line of centres, $\boldsymbol{F}_k = \lambda_k \hat{\boldsymbol{r}}_k$ with $\boldsymbol{r}_k$ from the ball centre to the contact point and $\lambda_k \geq 0$, so

$$\boldsymbol{M}_c = \sum_k \boldsymbol{r}_k \times \lambda_k \hat{\boldsymbol{r}}_k = \boldsymbol{0}. \tag{19}$$

Such a force does no virtual work on the ball rotation φ of the planar model, and $\lambda_k = 0$ while seated; when an arrestor engages it only enforces the constraint the model already assumes (the body stays on the ball).

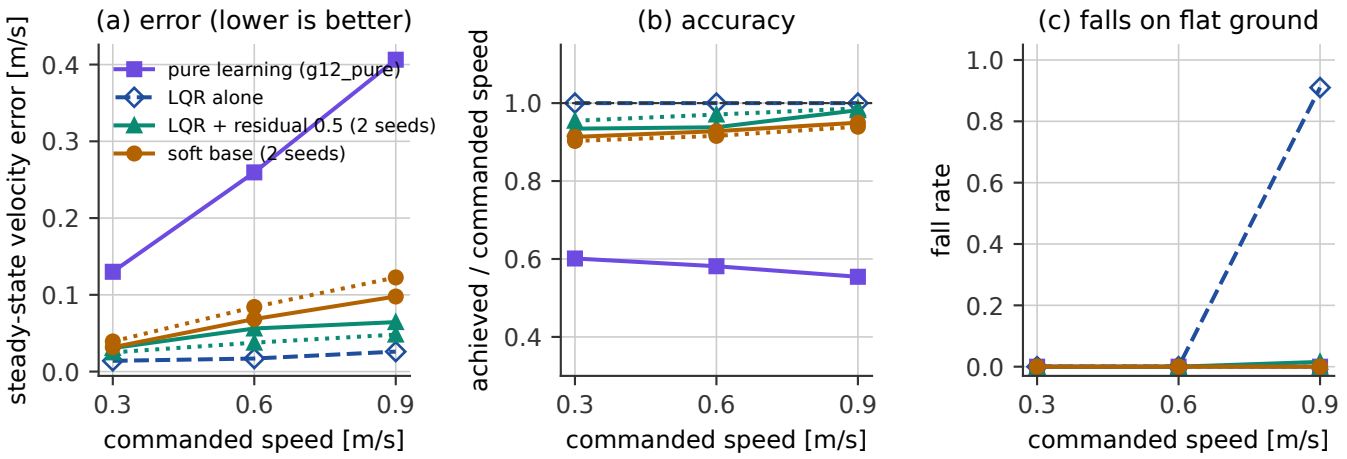


Fig. 9. Tracking error, achieved/commanded speed and falls versus commanded speed. Engine: Warp GPU, corrected. Randomisation: off. Noise: off. Worlds per point: 64 (LQR alone: hollow markers; its error and ratio at 0.9 m/s are over the few surviving worlds). Seeds: one for pure learning, two for the residual and the soft base (dotted). Protocol: clean flat benchmark of Table IX. Data files store means only, so no error bars.

TABLE X

Goal task on the tiled terrain, corrected engine. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1 (mass, friction, damping, torque, latency, floor classes, 11 designs). Observation noise: on. Worlds per cell: 16 per (design, terrain kind): 176 per terrain kind, 1760 per row. Seeds: one per row as listed. Protocol: goal 1.1 m from a random start on a tile, success = within 0.2 m before any fall within 800 steps (eval_battery.py); terrain columns are undisturbed success per kind; undisturbed = pushes and pulls off, disturbed = pushes 5 to 20 N and pulls 3 to 12 N. g14 rows are one-variable changes of the ramped (B) or soft (C) recipe.

Policy	Undisturbed: falls / success	median time [s]	step up	stairs	bump	ramp up	flat	Pushes + pulls: falls / success	Seed
pure learning (no LQR), 300M steps g12_pure_s0, seed 0	0.32 / 0.44	3.08	0.19	0.14	0.19	0.23	0.73	0.62 / 0.26	seed 0
LQR + residual 0.5, standard randomisation g12_res_s0, seed 0	0.42 / 0.58	2.61	0.33	0.38	0.17	0.74	0.84	0.75 / 0.25	seed 0
LQR + residual 0.5, standard randomisation g12_res_s1, seed 1	0.42 / 0.58	2.43	0.28	0.39	0.20	0.69	0.82	0.73 / 0.27	seed 1
LQR + residual 0.5, randomisation ramped to 1.75x g12_dr175_s0, seed 0	0.42 / 0.58	2.44	0.31	0.44	0.26	0.65	0.80	0.71 / 0.29	seed 0
LQR + residual 0.5, randomisation ramped to 1.75x g12_dr175_s1, seed 1	0.40 / 0.60	2.33	0.39	0.47	0.20	0.69	0.81	0.70 / 0.30	seed 1
soft LQR base (R x3, a_max 0.3), ramped to 1.75x g12_soft175_s0, seed 0	0.37 / 0.63	3.06	0.35	0.39	0.16	0.75	0.87	0.67 / 0.33	seed 0
soft LQR base (R x3, a_max 0.3), ramped to 1.75x g12_soft175_s1, seed 1	0.38 / 0.62	3.12	0.28	0.34	0.18	0.74	0.89	0.66 / 0.34	seed 1
soft LQR base, authority 0.75 g13_softrs075_s0, seed 0	0.42 / 0.58	3.37	0.22	0.26	0.15	0.67	0.84	0.68 / 0.32	seed 0
ramped 1.75x, fall penalty 5 (was 2) g14_fp5_s0, seed 0	0.43 / 0.57	2.41	0.32	0.38	0.19	0.70	0.80	0.73 / 0.27	seed 0
soft base, fall penalty 5 g14_softfp5_s0, seed 0	0.37 / 0.63	2.98	0.35	0.32	0.18	0.77	0.89	0.67 / 0.33	seed 0
ramped 1.75x, push and pull strength ramped 0.2 to 1 g14_pramp_s0, seed 0	0.43 / 0.57	2.29	0.34	0.37	0.18	0.66	0.79	0.75 / 0.25	seed 0
ramped 1.75x + edge starts + depth sensor g15_edge_depth_s0, seed 0	0.40 / 0.60	2.36	0.36	0.44	0.27	0.66	0.80	0.71 / 0.29	seed 0
ramped 1.75x + edge starts, blind control g15_edge_blind_s0, seed 0	0.40 / 0.60	2.30	0.38	0.41	0.28	0.65	0.80	0.72 / 0.28	seed 0
pure learning (no LQR), 300M steps g12_pure_s1, seed 1	0.36 / 0.51	2.97	0.26	0.27	0.26	0.28	0.80	0.66 / 0.27	seed 1
ramped 1.75x, trained with the ball retainer (arrestors at 100 deg) g16_ret100_s0, seed 0	0.28 / 0.71	2.81	0.47	0.52	0.40	0.76	0.88	0.53 / 0.46	seed 0
ramped 1.75x, trained with the ball retainer (arrestors at 100 deg) g16_ret100_s1, seed 1	0.24 / 0.76	2.75	0.55	0.64	0.51	0.86	0.89	0.51 / 0.49	seed 1
pure learning, trained with the retainer g16_pure_ret100_s0, seed 0	0.14 / 0.70	2.96	0.42	0.35	0.44	0.56	0.95	0.40 / 0.50	seed 0
pure learning, trained with the retainer g16_pure_ret100_s1, seed 1	0.15 / 0.74	2.62	0.58	0.55	0.53	0.65	0.88	0.44 / 0.50	seed 1
retainer, authority 0.5, tracking weight x3 g17_ret100_trk3_s0, seed 0	0.28 / 0.71	3.10	0.47	0.60	0.35	0.81	0.89	0.51 / 0.49	seed 0
retainer, authority 0.15 g18_ret_rs015_s0, seed 0	0.31 / 0.69	2.24	0.48	0.57	0.44	0.75	0.81	0.65 / 0.35	seed 0
retainer, authority 0.25 (recommended) g18_ret_rs025_s0, seed 0	0.28 / 0.71	2.44	0.52	0.57	0.49	0.83	0.84	0.58 / 0.42	seed 0
retainer, authority 0.25 (recommended) g18_ret_rs025_s1, seed 1	0.24 / 0.76	2.50	0.56	0.62	0.54	0.84	0.88	0.57 / 0.43	seed 1
retainer, authority 0.35 g18_ret_rs035_s0, seed 0	0.28 / 0.72	2.73	0.50	0.60	0.45	0.81	0.88	0.55 / 0.45	seed 0
retainer, soft LQR base, authority 0.5 g18_ret_soft_s0, seed 0	0.24 / 0.75	2.96	0.51	0.55	0.39	0.84	0.93	0.49 / 0.51	seed 0
retainer, authority 0.25 (recommended) g18_ret_rs025_s2, seed 2	0.24 / 0.76	2.68	0.56	0.65	0.49	0.86	0.87	0.56 / 0.44	seed 2
retainer, authority 0.25, 400M steps g19_ret_rs025_400m_s0, seed 0	0.25 / 0.75	2.61	0.52	0.63	0.49	0.86	0.87	0.57 / 0.43	seed 0
retainer, authority 0.25, depth sensor + edge starts g20_ret_rs025_depth_s0, seed 0	0.26 / 0.74	2.52	0.56	0.61	0.50	0.85	0.86	0.57 / 0.43	seed 0
retainer, authority 0.25, 12 training designs (adds tau3; 1920 worlds per goal row) g21_ret_rs025_tau3_s0, seed 0	0.29 / 0.71	2.51	0.51	0.64	0.52	0.83	0.82	0.59 / 0.41	seed 0
retainer, authority 0.25, 12 training designs (adds tau3; 1920 worlds per goal row) g21_ret_rs025_tau3_s1, seed 1	0.33 / 0.67	2.54	0.47	0.54	0.43	0.79	0.80	0.60 / 0.40	seed 1
retainer, authority 0.25, privileged height scan input g22_ret_rs025_scan_s0, seed 0	0.32 / 0.68	2.56	0.49	0.55	0.45	0.72	0.81	0.59 / 0.41	seed 0
retainer, authority 0.5, privileged height scan input g22_ret_rs05_scan_s0, seed 0	0.26 / 0.74	2.86	0.50	0.61	0.41	0.84	0.87	0.47 / 0.53	seed 0
pure learning, retainer, privileged height scan input g22_pure_ret_scan_s0, seed 0	0.17 / 0.69	2.96	0.47	0.42	0.32	0.44	0.93	0.40 / 0.50	seed 0
recommended recipe on the corrected contact model (free-rolling friction 0.01; recommended checkpoint) g26_ret_rs025_mu01_s0, seed 0	0.26 / 0.74	2.62	0.52	0.65	0.40	0.85	0.89	0.55 / 0.45	seed 0
recommended recipe on the corrected contact model (free-rolling friction 0.01; recommended checkpoint) g26_ret_rs025_mu01_s1, seed 1	0.26 / 0.74	2.67	0.52	0.61	0.42	0.86	0.88	0.54 / 0.46	seed 1
recommended recipe on the corrected contact model (free-rolling friction 0.01; recommended checkpoint) g26_ret_rs025_mu01_s2, seed 2	0.27 / 0.73	2.58	0.53	0.64	0.41	0.86	0.88	0.55 / 0.45	seed 2
recommended recipe, trained with the planner in the loop (30 % of worlds on blocks tiles) g23_nav_rs025_s0, seed 0	0.29 / 0.71	2.82	0.47	0.57	0.41	0.85	0.85	0.57 / 0.43	seed 0
recommended recipe, trained with the planner in the loop (30 % of worlds on blocks tiles) g23_nav_rs025_s1, seed 1	0.30 / 0.70	2.55	0.52	0.56	0.44	0.81	0.84	0.57 / 0.43	seed 1

The equations of motion, (A, B_s) and the LQR gain are therefore unchanged. Roller friction μ adds a moment of at most $\mu\lambda_k R$, which is zero while seated (Table XVI). With the robot upright the bottom of an arrestor sits $R + (R + r_a)\cos\phi - r_a$ above the floor: 6.50 cm at $\phi = 100°$ and only 1.27 cm at 135° (opposite the wheel), which would touch the floor at about 21° of tilt and is not buildable on this ball; arrestor-floor contact is not simulated.[1]

[1]Two software mistakes in the first retainer implementation (the settling procedure and a contact artefact between spheres and the Warp height field) were fixed before any result was kept.

c) Results.: Added at evaluation to the policy trained without them, arrestors at 100° raise goal success by 0.17 undisturbed and 0.17 with pushes and pulls (Table XV), the first change in this study that moves disturbance robustness; mounts between 90 and 100° are equal within noise, and 100° is chosen because only a mount below the equator resists lift-off. Trained with the arrestors (two seeds, Table XVII, Fig. 17), the recipe improves every robustness measure at once: goal success 0.74 undisturbed and 0.47 with pushes and pulls against 0.59 and 0.29 without, and tile velocity falls 0.32 against 0.53 undisturbed and 0.49 against 0.71 disturbed; the gain grows with randomisation (0.64 against 0.46 at scale 2.5). Most

TABLE XI

Velocity task at 0.3 m/s on the tiled terrain, corrected engine. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 16 per (design, terrain kind): 1760 per cell. Seeds: one per row as listed. Protocol: velocity task at 0.3 m/s at a random heading on the tiles with floor classes, 800 steps (eval_battery.py); tracking error is the mean over upright steps.

Policy	Falls, undisturbed	Falls, pushes + pulls	Tracking error undisturbed / disturbed [m/s]	Seed
pure learning (no LQR), 300M steps g12_pure_s0, seed 0	0.32	0.54	0.194 / 0.292	seed 0
LQR + residual 0.5, standard randomisation g12_res_s0, seed 0	0.55	0.72	0.112 / 0.218	seed 0
LQR + residual 0.5, standard randomisation g12_res_s1, seed 1	0.55	0.74	0.104 / 0.205	seed 1
LQR + residual 0.5, randomisation ramped to 1.75x g12_dr175_s0, seed 0	0.54	0.71	0.109 / 0.217	seed 0
LQR + residual 0.5, randomisation ramped to 1.75x g12_dr175_s1, seed 1	0.53	0.71	0.102 / 0.226	seed 1
soft LQR base (R x3, a_max 0.3), ramped to 1.75x g12_soft175_s0, seed 0	0.55	0.71	0.109 / 0.208	seed 0
soft LQR base (R x3, a_max 0.3), ramped to 1.75x g12_soft175_s1, seed 1	0.53	0.73	0.118 / 0.212	seed 1
soft LQR base, authority 0.75 g13_soft075_s0, seed 0	0.54	0.70	0.123 / 0.217	seed 0
ramped 1.75x, fall penalty 5 (was 2) g14_fp5_s0, seed 0	0.54	0.73	0.110 / 0.219	seed 0
soft base, fall penalty 5 g14_soft5p5_s0, seed 0	0.54	0.71	0.113 / 0.223	seed 0
ramped 1.75x, push and pull strength ramped 0.2 to 1 g14_pramp_s0, seed 0	0.54	0.73	0.111 / 0.222	seed 0
ramped 1.75x + edge starts, blind control g15_edge_blind_s0, seed 0	0.54	0.73	0.105 / 0.220	seed 0
ramped 1.75x + edge starts, blind control g15_edge_blind_s0, seed 0	0.53	0.73	0.105 / 0.220	seed 0
pure learning (no LQR), 300M steps g12_pure_s1, seed 1	0.38	0.57	0.183 / 0.274	seed 1
ramped 1.75x, trained with the ball retainer (arrestors at 100 deg) g16_ret100_s0, seed 0	0.31	0.49	0.141 / 0.229	seed 0
ramped 1.75x, trained with the ball retainer (arrestors at 100 deg) g16_ret100_s1, seed 1	0.32	0.50	0.138 / 0.230	seed 1
pure learning, trained with the retainer g16_pure_ret100_s0, seed 0	0.16	0.34	0.171 / 0.251	seed 0
pure learning, trained with the retainer g16_pure_ret100_s1, seed 1	0.19	0.38	0.155 / 0.235	seed 1
retainer, authority 0.5, tracking weight x3 g17_ret100_trk3_s0, seed 0	0.32	0.46	0.141 / 0.219	seed 0
retainer, authority 0.15 g18_ret_ra015_s0, seed 0	0.38	0.58	0.126 / 0.246	seed 0
retainer, authority 0.25 (recommended) g18_ret_ra025_s0, seed 0	0.34	0.55	0.125 / 0.235	seed 0
retainer, authority 0.25 (recommended) g18_ret_ra025_s1, seed 1	0.35	0.56	0.131 / 0.236	seed 1
retainer, authority 0.35 g18_ret_ra035_s0, seed 0	0.34	0.52	0.132 / 0.223	seed 0
retainer, soft LQR base, authority 0.5 g18_ret_soft_s0, seed 0	0.29	0.49	0.150 / 0.217	seed 0
retainer, authority 0.25 (recommended) g18_ret_ra025_s2, seed 2	0.36	0.55	0.137 / 0.234	seed 2
retainer, authority 0.25, 400M steps g19_ret_ra025_400m_s0, seed 0	0.38	0.55	0.140 / 0.221	seed 0
retainer, authority 0.25, depth sensor + edge starts g20_ret_ra025_depth_s0, seed 0	0.35	0.54	0.137 / 0.224	seed 0
retainer, authority 0.25, 12 training designs (adds tau), 1920 worlds per goal row) g21_ret_ra025_tau3_s0, seed 0	0.35	0.56	0.127 / 0.232	seed 0
retainer, authority 0.25, 12 training designs (adds tau), 1920 worlds per goal row) g21_ret_ra025_tau3_s1, seed 1	0.37	0.57	0.132 / 0.230	seed 1
retainer, authority 0.25, privileged height scan input g22_ret_ra025_scan_s0, seed 0	0.36	0.55	0.132 / 0.226	seed 0
retainer, authority 0.5, privileged height scan input g22_ret_ra05_scan_s0, seed 0	0.32	0.48	0.133 / 0.213	seed 0
pure learning, retainer, privileged height scan input g24_pure_ret_scan_s0, seed 0	0.15	0.35	0.165 / 0.231	seed 0
recommended recipe on the corrected contact model (free-rolling friction 0.01; recommended checkpoint) g26_ret_ra025_ms01_s0, seed 0	0.34	0.53	0.134 / 0.230	seed 0
recommended recipe on the corrected contact model (free-rolling friction 0.01; recommended checkpoint) g26_ret_ra025_ms01_s1, seed 1	0.34	0.51	0.140 / 0.229	seed 1
recommended recipe on the corrected contact model (free-rolling friction 0.01; recommended checkpoint) g26_ret_ra025_ms01_s2, seed 2	0.34	0.53	0.138 / 0.227	seed 2
recommended recipe, trained with the planner in the loop (30 % of worlds on blocks tiles) g23_nav_ra025_s0, seed 0	0.36	0.55	0.132 / 0.224	seed 0
recommended recipe, trained with the planner in the loop (30 % of worlds on blocks tiles) g23_nav_ra025_s1, seed 1	0.36	0.55	0.126 / 0.213	seed 1

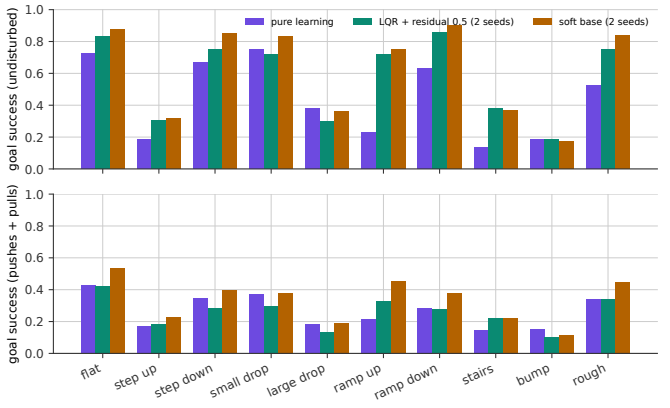


Fig. 10. Goal success per terrain kind, undisturbed (top) and with pushes and pulls (bottom). Engine: Warp GPU, corrected. Randomisation: on at scale 1. Noise: on. Worlds per bar: 176. Seeds: one for pure learning, two averaged otherwise. Protocol: goal task of Table X.

of it is mechanical: zero-shot arrestors give about the same goal success at scale 1. Training with the arrestors costs tracking accuracy: on the flat world the trained policies reach only 0.81 to 0.91 of the command against 0.95 to 1.02 without arrestors, on both seeds, while the arrestors themselves do not (0.96 to 0.99 zero-shot). A second pure-learning seed confirms the first (0.54 to 0.67 of the command, goal success 0.51 undisturbed).

d) The authority trade-off and the recommended controller.: Pure learning trained with the arrestors (two seeds) is the most robust controller (goal success 0.72 undisturbed and 0.50 with pushes and pulls, tile velocity falls 0.18 and 0.36) but tracks only 0.64 to 0.82 of the command; a tracking weight of 3 on the residual trained with arrestors does not recover tracking (0.74 to 0.88). With the arrestors the learned authority sets the trade-off (Table XX, Fig. 16): above 0.25 tracking falls off while robustness rises and saturates, and below 0.25 robustness drops (0.35 with pushes and pulls at 0.15) with no gain in tracking. We therefore recommend the LQR plus residual at authority 0.25 trained with the retainer (numbers in this paragraph are the g18 seeds at the training friction 0.001; the checkpoints retrained on the corrected contact model are in §X-F, correction paragraph): achieved 0.90 to 1.01 of the command over three seeds (error at most 0.049 m/s up to 0.9 m/s), goal success 0.71, 0.76 and 0.76 undisturbed and 0.42, 0.43 and 0.44 with pushes and pulls (without arrestors 0.59 and 0.29). Training it for 400M instead of 200M steps gives no gain (0.75 and 0.43). A single rendered clip shows a known weakness: at 0.6 m/s up an 11° ramp it lags the command (0.40 m/s at 2.8 s) and falls at 5.6 s. On held-out terrain (tile seed 1, Table XXVII) the retainer gain holds (residual 0.5: 0.42 and 0.43 with pushes and pulls; pure learning 0.46 and 0.47; without arrestors 0.26 and 0.26). On held-out terrain the recommended controller keeps its gain (0.66 and 0.69 undisturbed, 0.38 and 0.39 with pushes and pulls). Its remaining falls are on discrete obstacles (Table XXI): in the velocity task at 0.3 m/s it falls on bumps in 0.76 of worlds, large drops 0.65, step ups 0.62 and stairs 0.50, but on flat tiles only 0.08 and on ramps up 0.13; a raw depth sensor does not reduce them. With 20 s episodes, so that every robot reaches its feature, slowing the command from 0.3 to 0.1 m/s did not reduce falls on the original contact model (0.37 and 0.42 overall; Table XXII), and a rule-based speed governor on the privileged height map barely helped (0.39 to 0.37); both readings were partly the standing blow-up described below and are corrected in the next paragraph. Feature size decides (Table XXIII, re-checked on the corrected model): step ups above about 2.35 cm fail in 0.95 of worlds against 0.40 below, and bumps above 2.41 cm in 0.90; up-steps and bumps above about a quarter of the ball radius are nearly always fatal at any speed.

e) Correction: speed does matter on the corrected contact model.: Re-run at free-rolling friction 0.01 (Table XVIII), the speed test gives falls that rise monotonically with commanded speed, 0.28 at 0.1 m/s to 0.51 at 0.9 m/s; the old reading was inflated most at 0.1 m/s (0.42 against 0.28) because a creeping robot is nearly stationary, the blow-up condition. Size still dominates. The terrain-aware speed layer, dropped after the old measurement, was re-tested on a recommended checkpoint with 40 s episodes so that a slowed robot still reaches its feature (Table XIX): falls are 0.38 with no layer, 0.33 with a governor on the perfect height map and 0.35 with the head ToF sensor through a learned height estimator (precision 0.73, recall 0.85), both slowing to 0.05 m/s near a discontinuity; the real effect is 0.03 to 0.05, mostly on stairs (0.56 to 0.43) and bumps (0.79 to 0.72), at a cost of 0.04 to 0.08 in tile exits reached, and the goal task with 30 s episodes agrees (0.24 falls with no layer, 0.23 with the ToF layer). An earlier 15 s version of this test appeared to halve obstacle falls; it was an exposure artefact, the slowed robots not having reached their feature. Obstacle falls are therefore

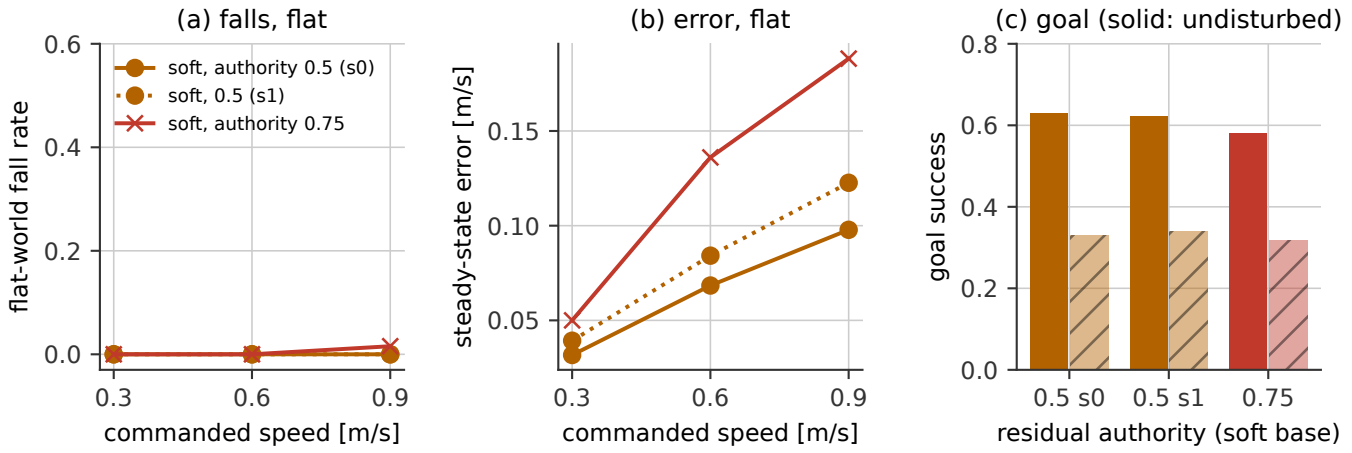


Fig. 11. Authority 0.5 against 0.75 on the soft base. Engine: Warp GPU, corrected. Randomisation: off in (a, b), on at scale 1 in (c). Noise: off (a, b), on (c). Worlds: 64 per point (a, b), 1760 per bar (c; solid undisturbed, hatched with pushes and pulls). Seeds: two at 0.5, one at 0.75. Protocol: Tables IX and X.

TABLE XII

Goal success against randomisation scale, corrected engine. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on; evaluated at the scale in the column (policies trained at scale 1 or ramped to 1.75). Observation noise: on. Worlds per cell: 1760 per cell. Seeds: one per row as listed. Protocol: goal task of Table 2 at DR scales 1, 1.75 and 2.5 (the master scale multiplies mass, friction, torque, damping, latency and noise ranges, not push or pull strength).

Policy	Goal success, DR 1: undisturbed / disturbed	DR 1.75	DR 2.5	Tile velocity falls, DR 2.5 undisturbed	Seed
pure learning (no LQR), 300M steps g12_pure_s0, seed 0	0.44 / 0.26	0.43 / 0.23	0.37 / 0.18	0.52	seed 0
LQR + residual 0.5, standard randomisation g12_res_s0, seed 0	0.58 / 0.25	0.52 / 0.23	0.45 / 0.18	0.66	seed 0
LQR + residual 0.5, standard randomisation g12_res_s1, seed 1	0.58 / 0.27	0.54 / 0.25	0.44 / 0.19	0.65	seed 1
LQR + residual 0.5, randomisation ramped to 1.75x g12_dr175_s0, seed 0	0.58 / 0.29	0.54 / 0.25	0.45 / 0.19	0.62	seed 0
LQR + residual 0.5, randomisation ramped to 1.75x g12_dr175_s1, seed 1	0.60 / 0.30	0.56 / 0.25	0.47 / 0.19	0.62	seed 1
soft LQR base (R x3, a_max 0.3), ramped to 1.75x g12_soft175_s0, seed 0	0.63 / 0.33	0.57 / 0.29	0.49 / 0.23	0.65	seed 0
soft LQR base (R x3, a_max 0.3), ramped to 1.75x g12_soft175_s1, seed 1	0.62 / 0.34	0.58 / 0.29	0.52 / 0.24	0.63	seed 1
soft LQR base, authority 0.75 g13_softrs075_s0, seed 0	0.58 / 0.32	0.53 / 0.26	0.45 / 0.21	0.66	seed 0
ramped 1.75x, fall penalty 5 (was 2) g14_fp5_s0, seed 0	0.57 / 0.27	0.53 / 0.25	0.46 / 0.18	0.63	seed 0
soft base, fall penalty 5 g14_softfp5_s0, seed 0	0.63 / 0.33	0.59 / 0.27	0.50 / 0.22	0.65	seed 0
ramped 1.75x, push and pull strength ramped 0.2 to 1 g14_pramp_s0, seed 0	0.57 / 0.25	0.52 / 0.21	0.47 / 0.16	0.62	seed 0
ramped 1.75x + edge starts + depth sensor g15_edge_depth_s0, seed 0	0.60 / 0.29	0.53 / 0.26	0.45 / 0.18	0.64	seed 0
ramped 1.75x + edge starts, blind control g15_edge_blind_s0, seed 0	0.60 / 0.28	0.54 / 0.23	0.47 / 0.19	0.61	seed 0
pure learning (no LQR), 300M steps g12_pure_s1, seed 1	0.51 / 0.27	0.44 / 0.23	0.39 / 0.19	0.58	seed 1
ramped 1.75x, trained with the ball retainer (arrestors at 100 deg) g16_ret100_s0, seed 0	0.71 / 0.46	0.70 / 0.42	0.62 / 0.35	0.42	seed 0
ramped 1.75x, trained with the ball retainer (arrestors at 100 deg) g16_ret100_s1, seed 1	0.76 / 0.49	0.73 / 0.44	0.67 / 0.37	0.44	seed 1
pure learning, trained with the retainer g16_pure_ret100_s0, seed 0	0.70 / 0.50	0.67 / 0.45	0.64 / 0.40	0.29	seed 0
pure learning, trained with the retainer g16_pure_ret100_s1, seed 1	0.74 / 0.50	0.73 / 0.46	0.67 / 0.40	0.33	seed 1
retainer, authority 0.5, tracking weight x3 g17_ret100_trk3_s0, seed 0	0.71 / 0.49	0.68 / 0.43	0.62 / 0.36	0.44	seed 0
retainer, authority 0.15 g18_ret_rs015_s0, seed 0	0.69 / 0.35	0.65 / 0.31	0.56 / 0.22	0.45	seed 0
retainer, authority 0.25 (recommended) g18_ret_rs025_s0, seed 0	0.71 / 0.42	0.68 / 0.36	0.60 / 0.29	0.44	seed 0
retainer, authority 0.25 (recommended) g18_ret_rs025_s1, seed 1	0.76 / 0.43	0.72 / 0.39	0.66 / 0.32	0.46	seed 1
retainer, authority 0.35 g18_ret_rs035_s0, seed 0	0.72 / 0.45	0.71 / 0.40	0.63 / 0.32	0.44	seed 0
retainer, soft LQR base, authority 0.5 g18_ret_soft_s0, seed 0	0.75 / 0.51	0.75 / 0.47	0.66 / 0.39	0.44	seed 0
retainer, authority 0.25 (recommended) g18_ret_rs025_s2, seed 2	0.76 / 0.44	0.71 / 0.41	0.63 / 0.32	0.46	seed 2
retainer, authority 0.25, 400M steps g19_ret_rs025_400m_s0, seed 0	0.75 / 0.43	0.70 / 0.36	0.64 / 0.30	0.48	seed 0
retainer, authority 0.25, depth sensor + edge starts g20_ret_rs025_depth_s0, seed 0	0.74 / 0.43	0.69 / 0.38	0.63 / 0.31	0.46	seed 0
retainer, authority 0.25, 12 training designs (adds tau3; 1920 worlds per goal row) g21_ret_rs025_tau3_s0, seed 0	0.71 / 0.41	0.66 / 0.36	0.58 / 0.29	0.45	seed 0
retainer, authority 0.25, 12 training designs (adds tau3; 1920 worlds per goal row) g21_ret_rs025_tau3_s1, seed 1	0.67 / 0.40	0.63 / 0.34	0.56 / 0.26	0.49	seed 1
retainer, authority 0.25, privileged height scan input g22_ret_rs025_scan_s0, seed 0	0.68 / 0.41	0.64 / 0.35	0.56 / 0.28	0.47	seed 0
retainer, authority 0.5, privileged height scan input g22_ret_rs05_scan_s0, seed 0	0.74 / 0.53	0.71 / 0.48	0.65 / 0.36	0.46	seed 0
pure learning, retainer, privileged height scan input g22_pure_ret_scan_s0, seed 0	0.69 / 0.50	0.67 / 0.46	0.64 / 0.40	0.31	seed 0
recommended recipe on the corrected contact model (free-rolling friction 0.01; recommended checkpoint) g26_ret_rs025_mu01_s0, seed 0	0.74 / 0.45	0.72 / 0.41	0.68 / 0.33	0.38	seed 0
recommended recipe on the corrected contact model (free-rolling friction 0.01; recommended checkpoint) g26_ret_rs025_mu01_s1, seed 1	0.74 / 0.46	0.72 / 0.42	0.69 / 0.36	0.39	seed 1
recommended recipe on the corrected contact model (free-rolling friction 0.01; recommended checkpoint) g26_ret_rs025_mu01_s2, seed 2	0.73 / 0.45	0.72 / 0.41	0.68 / 0.34	0.39	seed 2
recommended recipe, trained with the planner in the loop (30 % of worlds on blocks tiles) g23_nav_rs025_s0, seed 0	0.71 / 0.43	0.67 / 0.39	0.61 / 0.31	0.46	seed 0
recommended recipe, trained with the planner in the loop (30 % of worlds on blocks tiles) g23_nav_rs025_s1, seed 1	0.70 / 0.43	0.67 / 0.40	0.59 / 0.30	0.46	seed 1

set by obstacle size against ball size, and the tools that work are routing around what cannot be crossed (§X-H) and a larger ball (§X-G). A separate gap between the two checkpoint families, reported briefly in our log, was an evaluation bug, since fixed.

f) A traction limit, not missing foresight.: A driven ball of radius r pressed against a step edge of height h must lift off the floor and pivot on the edge; the contact normal makes an angle θ with the vertical, $\cos\theta = 1 - h/r$, and rolling over needs a tangential-to-normal force ratio $\tan\theta$, which friction supplies only if $\tan\theta < \mu$:

$$h < h_{\max} = r\big(1-\cos(\arctan\mu)\big), \qquad \mu \approx 1 : \; h_{\max} = r(1-\cos 45^\circ) \approx 0.29\,r, \tag{20}$$

2.64 cm for the 9 cm ball and 3.95 cm for the 13.5 cm ball (Fig. 15); floor friction varies by surface class, so the limit varies per world. Run on the unseen 13.5 cm-ball design, the recommended controller crosses up-steps and bumps the small ball cannot (Table XXIV: falls on the larger half of the step-up range 0.98 to 0.80, large bumps 0.94 to 0.62), while drops change little. A privileged height scan as policy input, the upper bound for any terrain sensor, does not reduce obstacle falls (Table XXV). Pure learning falls less on large obstacles because it stalls in front of them and keeps balancing, not because it climbs (crossing 0.02 on large step ups, against 0.02 for the residual; Table XXVI). On this map the falls above the traction limit are therefore a physical limit of ball size and friction that foresight does not change; below it, slowing before a feature buys

TABLE XIII

Goal task on held-out terrain (tile seed 1), corrected engine. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 16 per (design, terrain kind): 176 per terrain kind, 1760 per cell. Seeds: seeds 0 and 1 of the recipe. Protocol: goal task of Table 2 on a freshly generated tile map (tile seed 1: new severities and positions, same ten kinds and ranges) while the policy was trained on tile seed 0; undisturbed = pushes and pulls off, disturbed = on.

Policy	Undisturbed success, tile seed 1 (held out)	tile seed 0 (trained)	Disturbed success, tile seed 1	tile seed 0	Median time [s], undisturbed / disturbed	Seed
ramped to 1.75x (g12_dr175_s0, seed 0)	0.52	0.58	0.26	0.29	2.44 / 2.35	seed 0
ramped to 1.75x (g12_dr175_s1, seed 1)	0.53	0.60	0.26	0.30	2.32 / 2.24	seed 1

TABLE XIV

Fall diagnosis, corrected engine. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 1760 (16 per design and terrain kind). Seeds: one policy (g12_dr175_s0, seed 0). Protocol: goal task of Table 2 with per-step logging (fall_cause.py); derailed = the base axis is misaligned with the ball-centre-to-base direction by more than 5 deg (0.1 to 0.3 deg while the wheels sit on the ball).

Condition	Fall rate	Falls ended by the ball-offset limit (tilt below 70 deg)	Body derailed before tipping	Median tilt when derailing [deg]	Wheels at the torque limit, last 1 s before the fall	Falls with a pull active
undisturbed	0.43	0.95	0.78	14.7	0.7 %	0.00
pushes + pulls	0.72	0.97	0.49	15.4	0.4 %	0.58
pushes + pulls, LQR alone (no residual)	0.85	0.98	0.98	not recorded	0.4 %	0.71

TABLE XV

Arrestor mount-angle sweep (zero-shot), corrected engine. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 1760 per cell (176 per terrain kind). Seeds: one policy (g12_dr175_s0, seed 0), trained without arrestors (zero-shot). Protocol: goal task of Table 2 with frictionless arrestors (radius 8 mm) added at evaluation; floor clearance from the geometry R + (R + r_a) cos(phi) - r_a; the floor-touch tilts marked with a dagger are from the study log (S79), and arrestor-floor contact is not simulated.

Arrestor mount (polar angle from the top of the ball, deg)	Goal success undisturbed	Goal success pushes + pulls	Median misalignment at fall, undisturbed / disturbed [deg]	Arrestor bottom above the floor, upright [cm]	Tilt at which it touches the floor [deg]
none	0.57	0.28	20.6 / 11.4	n/a	n/a
135 (opposite the wheel)	0.66	0.37	7.2 / 4.3	1.3	21†
120	0.69	0.39	5.4 / 3.2	3.3	n/a
110	0.72	0.40	4.1 / 2.7	4.8	n/a
100	0.75	0.45	3.4 / 1.8	6.5	57†
95	0.75	0.43	2.6 / 1.8	7.3	n/a
90 (equator)	0.76	0.44	2.7 / 1.7	8.2	n/a

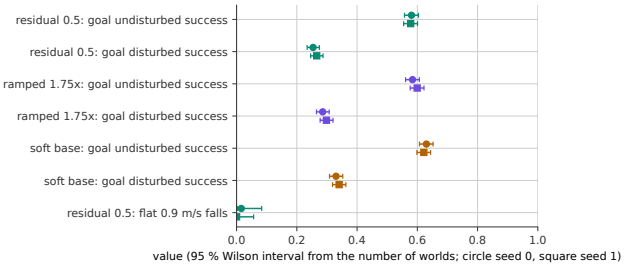


Fig. 12. Seed noise. Engine: Warp GPU, corrected. Randomisation: on at scale 1 (flat row: off). Noise: on (flat row: off). Worlds: 1760 per point (flat row: 64). Seeds: two per recipe, circle seed 0, square seed 1. Bars are 95 % Wilson intervals for sampling of worlds only.

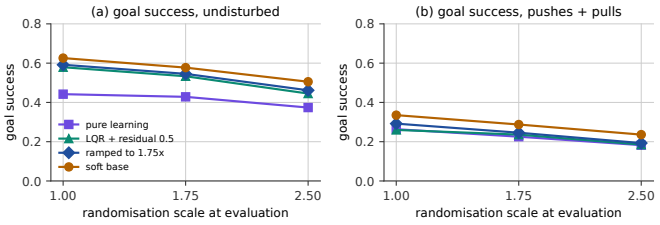


Fig. 13. Goal success against randomisation scale. Engine: Warp GPU, corrected. Randomisation: on, at the scale on the axis. Noise: on. Worlds: 1760 per point. Seeds: one for pure learning, two averaged for the others. Protocol: goal task, undisturbed (a) and with pushes and pulls (b).

TABLE XVI

Arrestor roller friction (zero-shot). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 1760 per cell. Seeds: one policy (g12_dr175_s0, seed 0), zero-shot. Protocol: goal task of Table 2, arrestors at 100 deg with the stated sliding friction (0 = ideal ball-transfer unit).

Arrestor sliding friction	Goal success undisturbed	Goal success pushes + pulls
0	0.75	0.45
0.05	0.74	0.42
0.2	0.76	0.42

only 0.03 to 0.05 once the robot is given time to reach it (Table XIX), and terrain sensing matters most for routing around obstacles, which these enclosed tiles do not allow (tested in §X-H).

g) Correction: the 20 s standing falls were a simulator blow-up.: Earlier drafts reported a 20 s standing fall rate per design for the recommended controller, an LQR gain sweep on that test and a seed comparison; all are withdrawn. The wheel-ball contact pairs use anisotropic friction, 0.001 along the roller axis and 1.0 across it, and in the float32 engine that 1000 to 1 elliptic cone is ill-conditioned when the contact is nearly stationary: a standing robot at 1 to 2° of tilt can have its state turn non-finite in one 10 ms step, preferentially where the ball is heavy relative to the body (kb4, bal025, zeta66). The termination rule counts a non-finite state as a fall,

TABLE XVII

Training with the ball retainer, corrected engine. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on (flat columns: off). Observation noise: on (flat columns: off). Worlds per cell: 1760 per goal and tile cell (176 per terrain kind); 64 per flat cell. Seeds: as listed (two seeds for the trained-with-retainer, ramped and pure rows). Protocol: eval_battery.py; the retainer rows use the 100 deg arrestors in training and evaluation (RETAINER=0,0,0,100), the zero-shot row adds them only at evaluation; its flat row was not part of the S92 re-run, so its flat falls are measured on the old tile positions and overstated (error and achieved speed unaffected).

Policy	Arrestors	Goal success undisturbed, DR 1 / 1.75 / 2.5	Goal success pushes + pulls, DR 1 / 1.75 / 2.5	Tile velocity falls undisturbed / disturbed (DR 1)	Flat error at 0.3 / 0.6 / 0.9 [m/s]	Flat achieved / command
ramped recipe, seed 0 g12_dr175_s0	none	0.58 / 0.54 / 0.45	0.29 / 0.25 / 0.19	0.54 / 0.71	0.027 / 0.046 / 0.057	0.95 to 0.99
ramped recipe, seed 1 g12_dr175_s1	none	0.60 / 0.56 / 0.47	0.30 / 0.25 / 0.19	0.53 / 0.71	0.013 / 0.021 / 0.041	1.00 to 1.02
ramped recipe, seed 0 g12_dr175_s0	added at evaluation (zero-shot)	0.75 (scale 1 only)	0.45 (scale 1 only)	0.36 / 0.53	0.030 / 0.046 / 0.067	0.96 to 0.99
trained with the retainer, seed 0 g16_ret100_s0	trained with	0.71 / 0.70 / 0.62	0.46 / 0.42 / 0.35	0.31 / 0.49	0.049 / 0.140 / 0.188	0.81 to 0.86
trained with the retainer, seed 1 g16_ret100_s1	trained with	0.76 / 0.73 / 0.67	0.49 / 0.44 / 0.37	0.32 / 0.50	0.048 / 0.096 / 0.115	0.87 to 0.91
pure learning, seed 0 g12_pure_s0	none	0.44 / 0.43 / 0.37	0.26 / 0.23 / 0.18	0.32 / 0.54	0.130 / 0.260 / 0.406	0.55 to 0.60
pure learning, seed 1 g12_pure_s1	none	0.51 / 0.44 / 0.39	0.27 / 0.23 / 0.19	0.38 / 0.57	0.103 / 0.246 / 0.421	0.54 to 0.67

TABLE XVIII

Falls against commanded speed on the corrected contact model (friction 0.01; 20 s episodes). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; wheel-ball free-rolling friction 0.01 (S98), policy trained at 0.001. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 176 per terrain kind (1760 per row). Seeds: one policy (g18_ret_rs025_s1, seed 1). Protocol: the Table 14 protocol: velocity task on the tiles, undisturbed, arrestors on, 20 s episodes so that every robot reaches its feature (eval_governor.py with no governor; files speed_mu01_*.json); fall rate per kind; the last column repeats Table 14 at 0.001.

Commanded speed	All tiles	step up	bump	large drop	stairs	flat	All tiles at 0.001 (Table 14, superseded)
0.1 m/s	0.28	0.46	0.54	0.74	0.43	0.06	0.42
0.2 m/s	0.34	0.61	0.73	0.72	0.48	0.05	0.41
0.3 m/s	0.35	0.62	0.74	0.66	0.49	0.06	0.37
0.6 m/s	0.40	0.54	0.66	0.77	0.51	0.11	not run
0.9 m/s	0.51	0.62	0.66	0.83	0.62	0.23	not run

TABLE XIX

A terrain-aware speed layer with 40 s episodes. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; wheel-ball free-rolling friction 0.01 (the policy's own training value, recorded in each file). Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 176 per terrain kind (1760 per row). Seeds: one policy (g26_ret_rs025_mu01_s1, a recommended checkpoint), one evaluation seed. Protocol: velocity task at 0.3 m/s on the tiles, undisturbed, arrestors on, 40 s episodes so that a slowed robot still reaches its feature, tile seed 0; governor.py slow mode, which lowers the commanded speed to 0.05 m/s within 0.25 m of a rise above 2.4 cm or a drop above 8 cm, read from the perfect height map or from the ToF estimate; files gov_vel03_ps0_none,oracle_v005,tof18_v005_40s.json. An earlier 15 s version of this test (S100 to S102) let slowed robots run out of time before their feature and is superseded.

Speed layer (g26 seed 1)	Falls, all tiles	step up	bump	stairs	large drop	flat	Tile exit reached	Progress
no speed layer	0.38	0.62	0.79	0.56	0.72	0.11	0.62	1.35 m
perfect height map, slow to 0.05 m/s near a discontinuity (upper bound)	0.33	0.61	0.72	0.43	0.74	0.07	0.58	1.31 m
ToF sensor (18 x 14 zones) through a learned height estimator, slow to 0.05 m/s	0.35	0.61	0.78	0.47	0.77	0.09	0.54	1.28 m

so these entered every 20 s standing number (91 % of the standing "falls" of the worst seed were neither tips nor unseatings), while driving and the 8 s battery were almost untouched. Contact buffers, solver, warm start, iteration counts, arrestors, randomisation and the height field were ruled out in a diagnostic of 880 standing worlds; raising the free-rolling coefficient to 0.01 removes the blow-up (0 of 880 non-finite against 0.127 at 0.001) with the real falls unchanged, and 0.01 is the model default from here (the coefficient of real rollers was never measured). Re-measured at 0.01 (Table XXVIII; policies trained at 0.001), the recommended controller stands for 20 s with 0.02 and 0.04 falls over 11 designs on two seeds, kb4 and bal025 at 0.00 and 0.00, and drives at 0.3 m/s with 0.06 to 0.07. What remains is design specific and real: the quarter-mass ball unseats within the first second (0.24 to 0.29 standing, 0.48 to 0.54 driving) and the four-times base mass falls in 0.18 to 0.23 of driving worlds. The earlier claim that the velocity integral is essential for standing is unverified. The recipe retrained at 0.01 (three seeds, g26_ret_rs025_mu01) reproduces the picture: standing 0.025 to 0.030, driving 0.062 to 0.068, no non-finite world, so the two weak designs are properties of the recipe at these mass extremes. These three seeds replace the g18 seeds as the recommended checkpoints; the recipe is unchanged and only the contact model is corrected, they are the only family with a reproducible standing number, and they equal the g18 seeds on a 15 s obstacle course on the same contact model. Their standard battery: goal success 0.73 to 0.74 undisturbed and 0.45 to 0.46 with pushes and pulls at scale 1 (0.72 / 0.41 to 0.42 at 1.75, 0.68 to 0.69 / 0.33 to 0.36 at 2.5), tile velocity falls 0.34

TABLE XX

Tracking accuracy against robustness with the retainer, corrected engine. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on (flat columns: off). Observation noise: on (flat columns: off). Worlds per cell: 1760 per goal and tile cell (176 per terrain kind); 64 per flat cell. Seeds: one per row as listed; authority 0.25 has three seeds (plus a 400M-step run), authority 0.5 and pure learning two. Protocol: eval_battery.py; every row except the two references is trained and evaluated with the 100 deg arrestors; the zero-shot row adds them only at evaluation; its flat row was not part of the S92 re-run, so its flat falls are measured on the old tile positions and overstated (error and achieved speed unaffected).

Controller	Flat achieved / command (0.3 to 0.9 m/s)	Flat error at 0.9 m/s [m/s]	Goal success undisturbed, DR 1 / 1.75 / 2.5	Goal success pushes + pulls, DR 1 / 1.75 / 2.5	Tile velocity falls undisturbed / disturbed (DR 1)	Median goal time [s]
no arrestors (reference) g12_dr175_s0, seed 0	0.95 to 0.99	0.057	0.58 / 0.54 / 0.45	0.29 / 0.25 / 0.19	0.54 / 0.71	2.44
no arrestors (reference) g12_dr175_s1, seed 1	1.00 to 1.02	0.041	0.60 / 0.56 / 0.47	0.30 / 0.25 / 0.19	0.53 / 0.71	2.33
authority 0.5, arrestors added at evaluation only g12_dr175_s0, seed 0	0.96 to 0.99	0.067	0.75 (scale 1 only)	0.45 (scale 1 only)	0.36 / 0.53	n/a
authority 0.15 g18_ret_rs015_s0, seed 0	0.99 to 1.01	0.042	0.69 / 0.65 / 0.56	0.35 / 0.31 / 0.22	0.38 / 0.58	2.24
authority 0.25 (recommended) g18_ret_rs025_s0, seed 0	0.99 to 1.01	0.036	0.71 / 0.68 / 0.60	0.42 / 0.36 / 0.29	0.34 / 0.55	2.44
authority 0.25 (recommended) g18_ret_rs025_s1, seed 1	0.98 to 1.00	0.034	0.76 / 0.72 / 0.66	0.43 / 0.39 / 0.32	0.35 / 0.56	2.50
authority 0.25 (recommended) g18_ret_rs025_s2, seed 2	0.90 to 0.99	0.049	0.76 / 0.71 / 0.63	0.44 / 0.41 / 0.32	0.36 / 0.55	2.68
authority 0.25, 400M steps g19_ret_rs025_400m_s0, seed 0	0.92 to 1.01	0.058	0.75 / 0.70 / 0.64	0.43 / 0.36 / 0.30	0.38 / 0.55	2.61
authority 0.35 g18_ret_rs035_s0, seed 0	0.91 to 0.98	0.060	0.72 / 0.71 / 0.63	0.45 / 0.40 / 0.32	0.34 / 0.52	2.73
authority 0.5 g16_ret100_s0, seed 0	0.81 to 0.86	0.188	0.71 / 0.70 / 0.62	0.46 / 0.42 / 0.35	0.31 / 0.49	2.81
authority 0.5 g16_ret100_s1, seed 1	0.87 to 0.91	0.115	0.76 / 0.73 / 0.67	0.49 / 0.44 / 0.37	0.32 / 0.50	2.75
authority 0.5, tracking weight x3 g17_ret100_trk3_s0, seed 0	0.74 to 0.88	0.201	0.71 / 0.68 / 0.62	0.49 / 0.43 / 0.36	0.32 / 0.46	3.10
soft LQR base, authority 0.5 g18_ret_soft_s0, seed 0	0.69 to 0.76	0.266	0.75 / 0.75 / 0.66	0.51 / 0.47 / 0.39	0.29 / 0.49	2.96
pure learning (no LQR) g16_pure_ret100_s0, seed 0	0.64 to 0.73	0.335	0.70 / 0.67 / 0.64	0.50 / 0.45 / 0.40	0.16 / 0.34	2.96
pure learning (no LQR) g16_pure_ret100_s1, seed 1	0.65 to 0.82	0.325	0.74 / 0.73 / 0.67	0.50 / 0.46 / 0.40	0.19 / 0.38	2.62

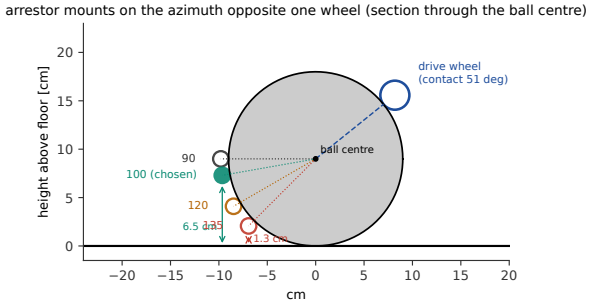


Fig. 14. Arrestor geometry: section through the ball centre on the azimuth opposite one wheel, mounts at 90, 100 (chosen), 120 and 135° with the upright floor clearance. Geometry only: no engine, randomisation, noise, worlds or seeds apply.

TABLE XXI

Where the recommended controller still falls (velocity task, undisturbed). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 176 per terrain kind. Seeds: as in the column headers. Protocol: velocity task at 0.3 m/s on the tiles, undisturbed (pushes and pulls off), 100 deg arrestors; fall rate per terrain kind, sorted by the seed-0 value.

Terrain kind	recommended, seed 0 (g18_ret_rs025_s0)	recommended, seed 1 (g18_ret_rs025_s1)	recommended + depth sensor, seed 0 (g20_ret_rs025_depth_s0)
flat	0.08	0.07	0.08
ramp up	0.12	0.14	0.11
step down	0.16	0.16	0.19
rough	0.17	0.16	0.23
small drop	0.18	0.17	0.22
ramp down	0.20	0.26	0.24
stairs	0.48	0.52	0.51
step up	0.61	0.62	0.55
large drop	0.64	0.66	0.68
bump	0.77	0.74	0.75

TABLE XXII

Falls against commanded speed (20 s episodes). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 176 per terrain kind (1760 per row). Seeds: one policy (g18_ret_rs025_s1, seed 1). Protocol: velocity task on the tiles, undisturbed, arrestors on, 20 s episodes so that every robot reaches the terrain feature (eval_governor.py with no governor); fall rate per kind.

Commanded speed	All tiles	step up	bump	large drop	stairs	flat
0.1 m/s	0.42	0.64	0.64	0.80	0.55	0.15
0.2 m/s	0.41	0.66	0.80	0.73	0.56	0.08
0.3 m/s	0.37	0.64	0.74	0.66	0.52	0.10

TABLE XXIII

Falls against feature size (velocity task, 0.3 m/s, 20 s). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 352 per kind (two tile maps' worth of worlds, split into halves by severity). Seeds: one policy (g18_ret_rs025_s1, seed 1). Protocol: velocity task at 0.3 m/s on the tiles, undisturbed, arrestors on, 20 s episodes; worlds split at the median drawn severity of their tile.

Feature	Severity range on the map	Split	Falls, smaller half	Falls, larger half
step up	1.1 to 5.9 cm	2.3 cm	0.40	0.95
bump	1.4 to 3.6 cm	2.4 cm	0.59	0.90
stairs (riser)	1.2 to 3.2 cm	1.4 cm	0.27	0.70
large drop	10.9 to 18.3 cm	15.1 cm	0.61	0.77
small drop	3.1 to 5.8 cm	4.4 cm	0.23	0.34
step down	1.2 to 5.5 cm	2.6 cm	0.12	0.21
ramp up	3.5 to 11.2 deg	9.6 deg	0.15	0.12
ramp down	5.2 to 11.9 deg	8.5 deg	0.28	0.29

undisturbed and 0.51 to 0.53 disturbed, flat achieved 0.85 to 0.93 at 0.3 m/s, 0.95 to 0.98 at 0.6 and 0.99 to 1.00 at 0.9 with no fall; every seed within 0.03 of the others (Tables IX and X, rows "corrected contact model"). The g18 seeds remain valid comparisons for every 8 s result and for driving.

G. Ball size against step size

The traction limit of §X-F0f predicts that the climbable step scales with the ball radius. To test it with one controller, the recommended recipe was trained (two seeds) on the 11 designs plus the robot scaled uniformly to 0.75, 1.5 and 2 times nominal (ball radius 6.75, 13.5 and 18 cm; masses scaled with the cube of the size; Froude-scaled commands; arrestors re-fitted per size; its own planar model per size for the LQR), 14 designs in all, and evaluated at five sizes including a held-out 0.5 scale (4.5 cm ball) on the same tile map with the same absolute feature sizes (Table XXIX; cells are the mean of the two seeds, which agree within 0.1 on every large-feature cell). The four trained sizes are evaluated on the corrected contact model (wheel-ball free-rolling friction 0.01, see the correction above; policies trained at 0.001), the held-out 0.5 scale only at 0.001. Falls on up-steps and bumps fall monotonically with ball radius, in step with the limit: the taller step band (2.3 to 5.9 cm) is fatal for the 6.75 cm ball (1.00), nearly fatal for 9 cm (0.94) and survivable for 13.5 and 18 cm (0.61, 0.58), whose limits (4.0 and 5.3 cm) cover most of the band. Drops improve with size beyond

TABLE XXIV

Ball size against step height (traction-limit test). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 640 per terrain kind (64 per kind and design copy, two copies), split into halves by severity. Seeds: one policy (g18_ret_rs025_s1, seed 1), trained on 9 cm-ball designs. Protocol: velocity task at 0.3 m/s on the tiles, undisturbed, arrestors on, 20 s episodes (eval_governor.py with –eval-designs nom,nom against s1.5,s1.5).

Feature	Smaller half / larger half of the severity range	Falls, nominal (9 cm ball): smaller / larger	Falls, s1.5 (13.5 cm ball, unseen design): smaller / larger
step up	1.1 to 2.3 / 2.3 to 5.9 cm	0.40 / 0.98	0.26 / 0.80
bump	1.4 to 2.4 / 2.4 to 3.6 cm	0.52 / 0.94	0.33 / 0.62
stairs (riser)	1.2 to 1.4 / 1.4 to 3.2 cm	0.17 / 0.67	0.19 / 0.64
large drop	10.9 to 15.1 / 15.1 to 18.3 cm	0.64 / 0.83	0.62 / 0.72

TABLE XXV

Privileged height scan as policy input against blind counterparts (retainer on). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 1760 per goal and tile cell, 176 per terrain kind. Seeds: one per row as listed (scan arms one seed each). Protocol: eval_battery.py; the scan arms add the privileged 7 x 5 height scan ahead as a policy input (the upper bound for any terrain sensor); per-kind columns are falls in the 8 s velocity task at 0.3 m/s, undisturbed.

Policy (all with the retainer)	Goal success undisturbed / disturbed (DR 1)	Tile velocity falls, undisturbed	step up falls	bump falls	stairs falls	large drop falls
residual 0.25 + scan g22_ret_rs025_scan_s0, seed 0	0.68 / 0.41	0.36	0.59	0.78	0.51	0.70
residual 0.25, blind g18_ret_rs025_s0, seed 0	0.71 / 0.42	0.34	0.61	0.77	0.48	0.64
residual 0.5 + scan g22_ret_rs05_scan_s0, seed 0	0.74 / 0.53	0.32	0.55	0.71	0.45	0.63
residual 0.5, blind g16_ret100_s0, seed 0	0.71 / 0.46	0.31	0.53	0.56	0.42	0.68
residual 0.5, blind g16_ret100_s1, seed 1	0.76 / 0.49	0.32	0.54	0.65	0.43	0.68
pure learning + scan g22_pure_ret_scan_s0, seed 0	0.69 / 0.50	0.15	0.29	0.24	0.19	0.38
pure learning, blind g16_pure_ret100_s0, seed 0	0.70 / 0.50	0.16	0.24	0.20	0.20	0.43
pure learning, blind g16_pure_ret100_s1, seed 1	0.74 / 0.50	0.19	0.31	0.30	0.25	0.45

TABLE XXVI

Falls and crossings on the larger half of each obstacle range. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 640 per terrain kind, halved by severity. Seeds: one policy each (seed 1). Protocol: velocity task at 0.3 m/s on the tiles, undisturbed, arrestors on, 20 s episodes; crossed = left the tile upright past the feature.

Feature (larger half of the range)	Pure learning, retainer (g16_pure_ret100_s1): falls / crossed	LQR + residual 0.25, retainer (g18_ret_rs025_s1): falls / crossed
step up, larger half (2.3 to 5.9 cm)	0.69 / 0.02	0.98 / 0.02
bump, larger half (2.4 to 3.6 cm)	0.30 / 0.02	0.94 / 0.06
stairs (riser), larger half (1.4 to 3.2 cm)	0.38 / 0.03	0.67 / 0.33
large drop, larger half (15.1 to 18.3 cm)	0.58 / 0.39	0.83 / 0.17

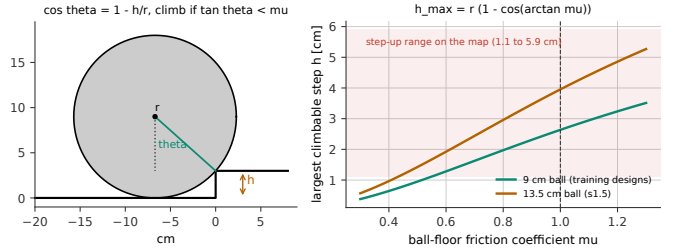


Fig. 15. Traction limit for climbing a step: geometry (left, $r = 9\,\text{cm}$, $h = 3\,\text{cm}$) and the largest climbable step against friction for the two ball sizes (right), with the map's step-up range shaded. Analytical: no engine, randomisation, noise, worlds or seeds apply.

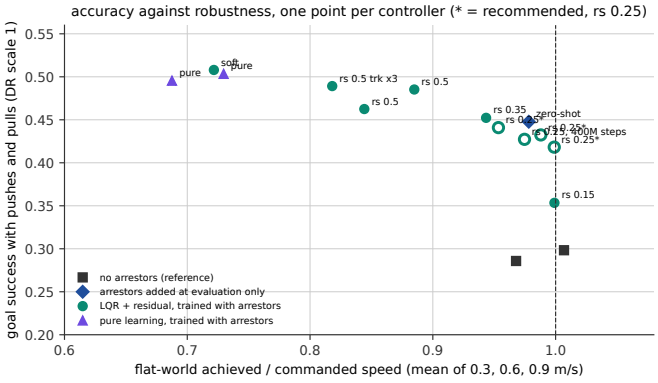


Fig. 16. Accuracy against robustness, one point per controller. Engine: Warp GPU, corrected. Randomisation: off on the horizontal axis (flat benchmark, 64 worlds per speed), on at scale 1 on the vertical axis (goal task with pushes and pulls, 1760 worlds). Noise: off (horizontal), on (vertical). Seeds: one point per seed. Protocol: Tables IX and X; all points except the references use the 100° arrestors in training and evaluation.

what traction explains (larger drops 0.89 to 0.27): in its own Froude time a bigger robot has longer to recover from a ledge that is a smaller fraction of its size. The held-out 4.5 cm-ball robot fails almost everywhere, on flat ground too (0.90, at 0.001): the policy does not transfer that far down, and small ballbots are the hardest case. Against the first-pass evaluation at 0.001 the two small robots are unchanged (within 0.03 overall) because their falls are traction, while the two large robots lose 0.05 to 0.06 of overall falls and up to 0.15 at the large steps, where a stalled robot sits almost still at the riser (the blow-up condition); the size ordering and the design rule did not change. Binning the recorded height of every world's feature, both seeds pooled (Fig. 18, Table XXX) gives a fall curve per ball that rises from a floor of about 0.3 (the general 20 s fall rate on terrain) to 1 and moves right with the radius. Its 50 % point sits at 0.17 to $0.29\,r$ across the four sizes and both features (mean $0.24\,r$), below the nominal $0.29\,r$, because floor friction is randomised per world (0.4 to 1.2 by surface class; at $\mu = 0.4$ the limit is only $0.07\,r$), so the measured curve is the limit

TABLE XXVII

The retainer on held-out terrain (tile seed 1), corrected engine. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 176 per terrain kind, 1760 per cell. Seeds: as listed. Protocol: goal task of Table 2 on a freshly generated tile map (tile seed 1); policies were trained on tile seed 0; the retainer rows run with the 100 deg arrestors.

Policy	Undisturbed success, held-out map (tile seed 1)	trained map (tile seed 0)	Pushes + pulls success, held-out map	trained map
recommended: LQR + residual 0.25, trained with the retainer g18_ret_rs025_s0, seed 0	0.66	0.71	0.38	0.42
recommended: LQR + residual 0.25, trained with the retainer g18_ret_rs025_s1, seed 1	0.69	0.76	0.39	0.43
LQR + residual 0.5, trained with the retainer g16_ret100_s0, seed 0	0.66	0.71	0.42	0.46
LQR + residual 0.5, trained with the retainer g16_ret100_s1, seed 1	0.68	0.76	0.43	0.49
pure learning, trained with the retainer g16_pure_ret100_s0, seed 0	0.66	0.70	0.46	0.50
pure learning, trained with the retainer g16_pure_ret100_s1, seed 1	0.71	0.74	0.47	0.50
no arrestors (reference) g12_dr175_s0, seed 0	0.52	0.58	0.26	0.29
no arrestors (reference) g12_dr175_s1, seed 1	0.53	0.60	0.26	0.30

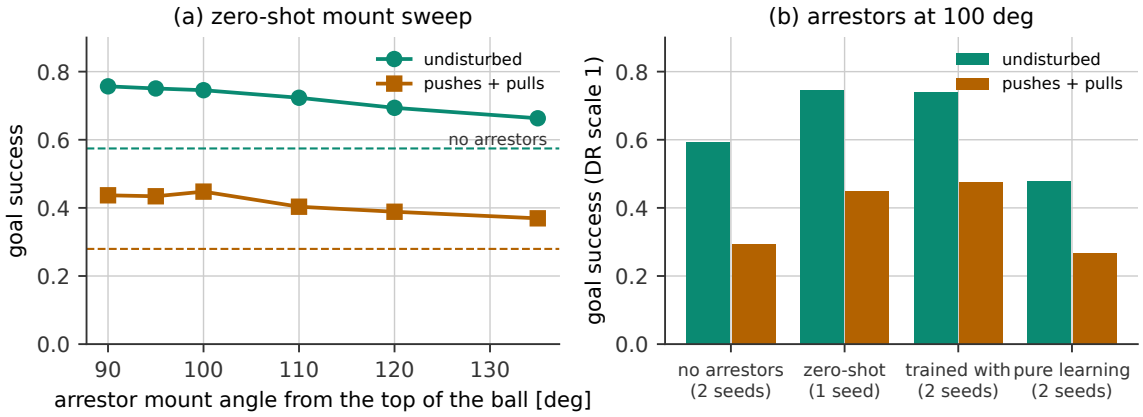


Fig. 17. The ball retainer. Engine: Warp GPU, corrected. Randomisation: on at scale 1. Noise: on. Worlds: 1760 per point or bar. Seeds: (a) one policy (g12_dr175_s0), zero-shot mount sweep, dashed lines without arrestors; (b) two seeds per bar except zero-shot (one). Protocol: goal task of Table X.

TABLE XXVIII

Standing and driving for 20 s on flat interior tiles at wheel-ball free-rolling friction 0.01 (the earlier 20 s standing tables are withdrawn, see text). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; wheel-ball free-rolling friction 0.01 for every row (the first two policies were trained at 0.001 and are evaluated at 0.01 through PAIR_MU1; the retrained ones carry 0.01 in args.json; the value is recorded in each file's env_settings). Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 32 per design (352 per cell). Seeds: seeds 1 and 2 of the recommended recipe (seed 0 not re-measured) and the seeds of the recipe retrained at 0.01 that are evaluated so far, one evaluation seed each. Protocol: flat interior tiles (at least 7.5 m from the map border), 20 s episodes, arrestors on, no pushes or pulls; zero command (standing) or 0.3 m/s at a random heading (driving); no fall was at the map edge; a non-finite state is counted separately where the file records it (new evaluations only) and is 0.000 in every such row.

Policy	Standing still for 20 s at friction 0.01, falls: all 11 designs / kb025 / bal4 / kb4 / bal025 / zeta66 / nominal	Driving at 0.3 m/s for 20 s at 0.01, falls: all / kb025 / bal4 / kb4 / bal025 / zeta66 / nominal	Non-finite worlds (fraction)
recommended controller, seed 1 (trained at 0.001) g18_ret_rs025_s1	0.03 / 0.24 / 0.05 / 0.00 / 0.00 / 0.00 / 0.00	0.06 / 0.50 / 0.18 / 0.00 / 0.00 / 0.00 / 0.00	not recorded
recommended controller, seed 2 (trained at 0.001) g18_ret_rs025_s2	0.04 / 0.29 / 0.13 / 0.00 / 0.00 / 0.00 / 0.00	0.07 / 0.54 / 0.23 / 0.00 / 0.00 / 0.00 / 0.02	not recorded
same recipe retrained at 0.01, seed 0 g26_ret_rs025_mu01_s0	0.03 / 0.27 / 0.04 / 0.00 / 0.00 / 0.00 / 0.00	0.06 / 0.48 / 0.20 / 0.00 / 0.00 / 0.00 / 0.00	0.000
same recipe retrained at 0.01, seed 1 g26_ret_rs025_mu01_s1	0.03 / 0.28 / 0.06 / 0.00 / 0.00 / 0.00 / 0.00	0.07 / 0.53 / 0.22 / 0.00 / 0.00 / 0.00 / 0.00	0.000
same recipe retrained at 0.01, seed 2 g26_ret_rs025_mu01_s2	0.02 / 0.24 / 0.04 / 0.00 / 0.00 / 0.00 / 0.00	0.06 / 0.49 / 0.21 / 0.00 / 0.00 / 0.00 / 0.00	0.000

averaged over the friction distribution. The scale policy keeps the recommended controller's standard results on both seeds (Table XXXI, scale seeds evaluated at friction 0.01 next to the recommended checkpoints: goal 0.70 to 0.74 undisturbed and 0.44 to 0.47 with pushes and pulls, flat achieved 0.89 to 0.99). Design rule (simulation only): choose a ball radius of about 3.4 to 4.2 times the tallest step to be climbed (from $0.29\,r$ at $\mu \approx 1$ and from the mean measured 50 % point when friction is unknown), then add arrestors for the falls that are not traction.

H. Terrain-aware navigation and sensors

To test what terrain sensing does when obstacles can be avoided, every tile of a new map is a blocks tile: flat ground with four 8 cm blocks (three times the traction limit of the 9 cm ball) across the tile axes 0.65 to 0.85 m from the centre, 0.3 to 0.5 m wide, plus up to two 24 cm square blocks; the goal lies 1.2 m away behind a block, and gaps between blocks are at least 0.7 m. The balance controller is frozen; nothing is retrained. The results in this paragraph and the next were measured with the earlier checkpoint (g18, friction 0.001); the recommended checkpoint at its own friction is in Table XXXII. A wavefront planner on

TABLE XXIX

Five robot sizes against the same obstacles (falls on the smaller / larger half of each feature range, mean of two training seeds; 0.5 scale one seed). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1 (floor friction 0.4 to 1.2 by surface class). Observation noise: on. Worlds per cell: 128 per terrain kind, robot scale and seed (1280 per scale and seed), split into halves at the midpoint of each feature's drawn range. Seeds: two training seeds of one recipe (g24_scale_rs025_s0, g24_scale_rs025_s1), the recommended recipe trained on the 11 designs plus the robot at 0.75, 1.5 and 2 times nominal size (14 designs); cells are the mean of the two seeds (the last column gives each seed); the 0.5 scale was never trained on and was evaluated with seed 0 only, at friction 0.001. Protocol: velocity task at 0.3 m/s Froude-scaled (times the square root of the scale) on the tiles, undisturbed, arrestors re-fitted per scale, 20 s episodes, tile seed 0; feature sizes are the same absolute sizes for every robot (eval_governor.py –source none –eval-designs); evaluated at wheel-ball free-rolling friction 0.01 (S98) with policies trained at 0.001 (files stepsize_mu01_*.json).

Robot scale	Ball radius	Predicted limit 0.29 r	Step up, falls: 1.1 to 2.3 / 2.3 to 5.9 cm	Bump: 1.4 to 2.4 / 2.4 to 3.6 cm	Stairs riser: below / above 1.4 cm	Large drop: 10.9 to 15.1 / 15.1 to 18.3 cm	Flat	All ten tile kinds	All ten tile kinds, per seed
0.5 (held out, not trained); at friction 0.001 only	4.5 cm	1.3 cm	0.90 / 1.00	0.98 / 1.00	0.94 / 1.00	0.98 / 0.98	0.90	0.95	0.95 (seed 0 only)
0.75	6.75 cm	2.0 cm	0.49 / 1.00	0.77 / 0.99	0.34 / 0.82	0.84 / 0.89	0.13	0.46	0.46 | 0.46
1 (nominal)	9 cm	2.6 cm	0.33 / 0.94	0.68 / 0.95	0.16 / 0.68	0.59 / 0.84	0.03	0.32	0.31 | 0.34
1.5	13.5 cm	4.0 cm	0.25 / 0.61	0.37 / 0.50	0.12 / 0.45	0.35 / 0.48	0.00	0.21	0.20 | 0.21
2	18 cm	5.3 cm	0.17 / 0.58	0.26 / 0.41	0.08 / 0.29	0.23 / 0.27	0.00	0.13	0.13 | 0.13

TABLE XXX

Falls against step and bump height in 1 cm bins, by ball radius (both seeds pooled). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1 (floor friction 0.4 to 1.2). Observation noise: on. Worlds per cell: 36 to 128 per bin (the per-world feature heights of Table 30, binned, both seeds pooled). Seeds: 2 training seeds of the scale recipe (g24_scale_rs025_s0, g24_scale_rs025_s1). Protocol: as Table 30, evaluated at wheel-ball free-rolling friction 0.01 (S98); fall probability per 1 cm band of feature height.

Ball radius (predicted limit)	step up 1 to 2 cm	step up 2 to 3 cm	step up 3 to 4 cm	step up 4 to 5 cm	step up 5 to 6 cm	bump 1 to 2 cm	bump 2 to 3 cm	bump 3 to 4 cm
6.75 cm (limit 2.0 cm)	0.36	0.86	1.00	1.00	1.00	0.57	0.87	0.99
9 cm (limit 2.6 cm)	0.25	0.56	0.81	1.00	1.00	0.36	0.84	0.95
13.5 cm (limit 4.0 cm)	0.21	0.36	0.33	0.72	0.76	0.33	0.38	0.50
18 cm (limit 5.3 cm)	0.16	0.19	0.36	0.53	0.84	0.21	0.28	0.41

TABLE XXXI

The scale policy on the standard battery at friction 0.01, against the recommended checkpoints. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; wheel-ball free-rolling friction 0.01 in every row (the scale seeds were trained at 0.001 and evaluated at 0.01 through PAIR_MU1, files battery_mu0.01.json; the recommended checkpoints were trained at 0.01). Domain randomisation: on at scale 1 for the goal columns (the 11 training designs of the goal task), off for the flat columns (nominal design). Observation noise: on (goal), off (flat). Worlds per cell: 1760 per goal cell, 64 per flat cell. Seeds: one row per training seed, as listed. Protocol: the standard battery (eval_battery.py): goal task of Table 2 and the clean flat benchmark of Table 1 on interior tiles.

Policy	Goal success, undisturbed / pushes and pulls (DR scale 1)	Flat benchmark, achieved over commanded speed at 0.3 / 0.6 / 0.9 m/s	Flat benchmark falls at 0.3 / 0.6 / 0.9 m/s
scale policy, 14 designs, trained at 0.001 g24_scale_rs025_s0, seed 0, evaluated at 0.01	0.74 / 0.47	0.91 / 0.92 / 0.98	0.00 / 0.00 / 0.00
scale policy, 14 designs, trained at 0.001 g24_scale_rs025_s1, seed 1, evaluated at 0.01	0.70 / 0.44	0.89 / 0.93 / 0.99	0.00 / 0.00 / 0.00
recommended checkpoint, 11 designs, trained and evaluated at 0.01 g26_ret_rs025_mu01_s0, seed 0	0.74 / 0.45	0.90 / 0.96 / 0.99	0.00 / 0.00 / 0.00
recommended checkpoint, 11 designs, trained and evaluated at 0.01 g26_ret_rs025_mu01_s1, seed 1	0.74 / 0.46	0.85 / 0.95 / 0.99	0.00 / 0.00 / 0.00
recommended checkpoint, 11 designs, trained and evaluated at 0.01 g26_ret_rs025_mu01_s2, seed 2	0.73 / 0.45	0.93 / 0.98 / 1.00	0.00 / 0.00 / 0.00

top marks ground more than 3 cm above the start as obstacle, inflates obstacles by the robot half-width plus margin (0.22 m), computes a distance-to-goal field on a 64×64 grid of 5 cm cells every 0.2 s, and steers toward the lowest-cost of 24 nearby points at up to 0.3 m/s. With a sensor, each noisy range and the known sensor pose give a 3-D hit point; points more than 3 cm above the ground under the ball are kept in a memory of the 600 most recent points that the planner rasterises. With the same controller (Table XXXIII, Fig. 19) the perfect height map cuts falls from 0.91 to 0.09 and raises success from 0.09 to 0.91; a head-mounted ToF of 18×14 zones reaches 0.85, a denser depth camera at the same mount 0.83. Mount and pitch matter more than resolution: pitched to −40° the camera leaves 0.34 of worlds stuck, an 8×8 sensor or a low mount loses 7 to 12 points, and two body-cone sensors are worst (0.58). Under pushes and pulls even the perfect map falls 0.41 (Table XXXIV); a wider margin (0.30 or 0.38 m) closes the 0.7 m gaps and leaves worlds stuck instead, so disturbance rejection near obstacles is the next limit.

a) Smooth motion to goals.: The wavefront planner picks a new neighbour every step, so its commanded heading jumps (RMS heading rate 14.40 rad/s). A smooth follower on the same distance field (carrot point 0.3 m ahead, heading turned at most 2 rad/s, speed lowered with heading error and changed at most 0.6 m/s^2) cuts the rate to 1.73 rad/s for a few points of success (0.86 against 0.91 with the perfect map; Table XXXV); two faults found while tuning (a robot inside the inflated margin stopping for good, and a wider margin closing the 0.7 m gaps) were fixed before any result was kept. With a real sensor the smooth follower first did worse (0.72; 0.16 of worlds rolled onto a block) because the gimbal yawed to the slowly turning command and saw side blocks late; aiming the

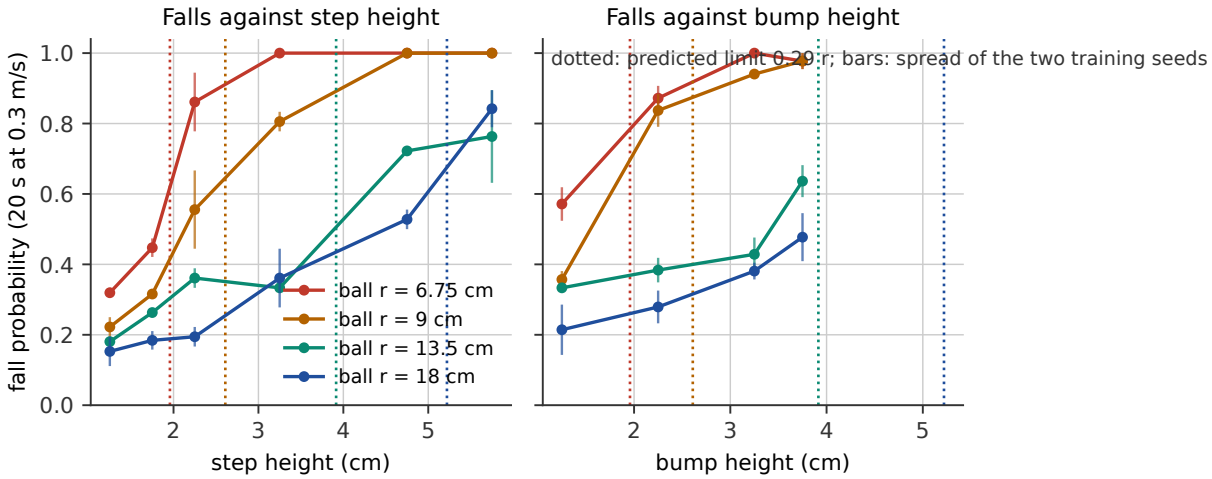


Fig. 18. Fall probability against step height (left) and bump height (right) for four ball radii, 0.5 cm bins with at least 8 worlds, worlds of both seeds pooled; vertical bars span the two seeds' values; dotted lines at the predicted limit $0.29\,r$; evaluated at wheel-ball free-rolling friction 0.01 (policies trained at 0.001). Engine: Warp GPU, corrected. Randomisation: on at scale 1 (floor friction 0.4 to 1.2). Noise: on. Worlds: 128 per terrain kind, size and seed, binned by drawn feature height. Seeds: two training seeds (g24_scale_rs025_s0, s1). Protocol: velocity task at 0.3 m/s Froude-scaled, undisturbed, arrestors on, 20 s.

sensor at the planner's target heading restores 0.78 with block contacts 0.03 (Table XXXVI); a proximity slow-down, faster turning and faster replanning were tried and rejected. Training with the planner in the loop (30 % of training worlds on blocks tiles following the smooth planner, two seeds) is a null result: within seed noise of the frozen controller on every condition, with about 0.05 fewer block contacts under pushes that do not turn into fewer falls (Table XXXVII). The remaining failures are push recoveries near obstacles, a balance-controller limit, not a perception or training-data problem.

b) The recommended checkpoint on the corrected contact model.: Repeated with a recommended checkpoint (g26 seed 1) at friction 0.01 (Table XXXII; smooth follower, sensor aimed at the target), ToF routing reaches 0.84 of goals behind a block, within 0.05 of the perfect map (0.89) and as smooth (heading-rate RMS 1.74 against 1.69 rad/s); blind it reaches 0.09. Under pushes and pulls about half the goals are reached (0.57 with the perfect map, 0.51 with the ToF), and a push near or onto a block remains the hard case.

I. Smoother motion, adaptive authority and the current recommendation

a) Where the wobble came from.: The navigation paths above oscillate and robots sometimes hunt around a reached goal. In a recorded ToF run the planner's command turned smoothly (heading-rate RMS 1.6 rad/s) while the ball's velocity heading swung at 8.1 rad/s RMS and the robot made 66 % of the commanded speed: the oscillation lies in how the balance controller follows a continuously changing command, not in the planner. The policy had been trained on piecewise-constant commands, a ballbot must lean before it accelerates, and at the

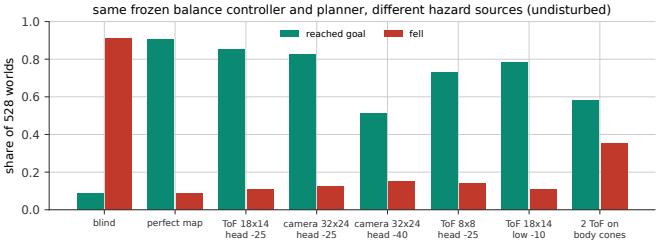


Fig. 19. Navigation success and falls per hazard source. Engine: Warp GPU, corrected. Randomisation: on at scale 1. Noise: on (sensor ranges noisy, with dropout). Worlds: 528 per bar (48 per design, 11 designs). Seeds: one controller seed (g18_ret_rs025_s1, frozen). Protocol: blocks map, undisturbed, 20 s, wavefront planner with a 0.22 m margin.

goal nothing held position. We therefore place a motion-reference layer between planner and controller (acceleration, braking and turn-rate limits, a braking profile $v = \sqrt{2ad}$ into the goal, and a position hold on arrival). Added to an unchanged policy it doubled the goal-test falls (0.15 to 0.31); trained with it (three seeds) it costs nothing on the battery and improves arrival and hold. Two further changes give the recommended recipe: adaptive authority (0.15 while cruising, rising to 0.3 for about 0.5 s when the command changes faster than 0.3 m/s per second, nears zero, or the body tilts beyond 4°) and a goal dead band (near the goal the higher authority stays on only while the ball moves faster than 0.05 m/s).

b) The recommended controller and the variants that did not help.: Table XXXVIII compares every variant on a flat-tile goal test and on the standard battery. The recommended recipe (g34, three seeds) falls in 0.09 to 0.11 of goal tests against 0.15 for the previous checkpoint,

TABLE XXXII

Navigation with the recommended checkpoint at friction 0.01 (smooth follower). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; wheel-ball free-rolling friction 0.01, the controller's own training value. Domain randomisation: on at scale 1 (11 designs, floor classes). Observation noise: on (sensor ranges: noise 0.5 percent of range, 2 percent dropout). Worlds per cell: 528 per row (48 per design, 11 designs). Seeds: one controller seed (g26_ret_rs025_mu01_s1, a recommended checkpoint, frozen). Protocol: blocks map, 20 s, smooth follower (carrot 0.3 m, turn rate at most 2 rad/s; the blind row heads straight for the goal), sensor gimbal aimed at the planner's target heading; pushes 5 to 20 N and pulls 3 to 12 N in the last two rows; files g26_ret_rs025_mu01_s1/navS_blind,oracle,tof18.json and navS_oracle,tof18_pp.json.

Hazard source (recommended checkpoint)	Reached goal	Fell	Stuck	Rolled onto a block	Median time to goal	Commanded heading-rate RMS [rad/s]
blind (straight to the goal)	0.09	0.91	0.00	0.75	6.5 s	2.21
perfect height map (upper bound)	0.89	0.07	0.04	0.01	9.5 s	1.69
ToF 18 x 14, head, pitched down 25 deg, aimed at the target heading	0.84	0.07	0.08	0.04	10.8 s	1.74
perfect height map, pushes and pulls	0.57	0.39	0.04	0.16	8.6 s	1.69
ToF 18 x 14, pushes and pulls	0.51	0.43	0.06	0.17	9.4 s	1.74

TABLE XXXIII

Navigation around unclimbable blocks (undisturbed), frozen recommended controller with a planner. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; earlier measurement: controller g18_ret_rs025_s1 trained and evaluated at wheel-ball free-rolling friction 0.001 (the recommended checkpoint at 0.01 is in Table 37). Domain randomisation: on at scale 1 (11 designs, floor classes). Observation noise: on (sensor ranges: noise 0.5 percent of range, 2 percent dropout). Worlds per cell: 528 per row (48 per design, 11 designs). Seeds: one controller seed (g18_ret_rs025_s1, frozen). Protocol: blocks map, undisturbed, 20 s episodes, goal 1.2 m away behind a block; wavefront planner (eval_nav.py) with obstacles inflated by 0.22 m on a 5 cm grid; nothing is retrained.

Hazard source	Falls	Reached goal	Stuck	Rolled onto a block
blind (straight to the goal)	0.91	0.09	0.00	0.74
privileged height map (upper bound)	0.09	0.91	0.00	0.00
ToF 18 x 14, 55 x 42 deg, head, pitch -25 deg	0.11	0.85	0.04	0.02
depth camera 32 x 24, 87 x 58 deg, head, pitch -25 deg	0.12	0.83	0.05	0.02
depth camera 32 x 24, head, pitch -40 deg	0.15	0.51	0.34	0.00
ToF 8 x 8, 45 x 45 deg, head, pitch -25 deg	0.14	0.73	0.13	0.02
ToF 18 x 14 at the base origin, pitch -10 deg	0.11	0.78	0.11	0.02
two ToF 18 x 14 on the body cones	0.35	0.58	0.07	0.25

holds at 0.052 to 0.065 m/s with 0.48 to 0.86 command reversals per world, and reaches 0.77 to 0.79 of battery goals undisturbed and 0.51 to 0.52 with pushes and pulls (g26: 0.73 to 0.74 and 0.45 to 0.46). Four variants did not improve on it: a tracking reward weight of 3 (two seeds each on fixed and adaptive authority; curve tracking inside the seed spread); twice the training share for the three hard designs (three seeds; every battery cell within the spread, the hard designs still falling 0.28 to 0.29); 400M instead of 200M steps (three seeds; fewer flat goal-test falls, 0.064 to 0.072, but worse terrain goals in every battery cell and goal hunting back at 2.52 to 3.73 reversals per world); and training on 1.5 or 2 times stronger pushes and pulls (one seed each; pushed-battery success 0.02 to 0.04 lower). Three readings were corrected on the way: the goal hunting that motivated the dead band belonged to one seed, so the claim that adaptive authority needs a dead band was withdrawn (the dead band is kept because it removes that seed-dependent behaviour at no cost); the zigzag falls of 0.28 to 0.33 measured terrain exposure, not turning (0.051 on a fully flat map); and solver constraint-buffer overflow warnings changed no battery cell by more than 0.02 when the buffers were quadrupled.

c) Navigation with the motion layer.: With the motion layer in the loop (Table XXXIX) ToF navigation reaches 0.877 of goals behind a block against 0.845 for the previous checkpoint, with the commanded heading-rate RMS cut from 1.74 to 1.31 rad/s, about a quarter. Running the same checkpoint with raw planner commands splits the gain: the success belongs to the checkpoint (0.881 without the layer) and the whole wobble reduction to the layer (1.79 rad/s without it). With a perfect map 0.938 arrive, so perception now limits ToF navigation; under pushes and pulls 0.557. The layer costs about 0.3 s of median time per route; faster acceleration and braking limits were no better undisturbed and worse under pushes, and were rejected. One seed per recipe: success differences of 0.01 to 0.05 are within the seed spread, the wobble reduction is not. A contact escape layer for a robot with no terrain sensing (stall detection, obstacle memory, back up and re-plan; Table XL) reaches 0.634 of goals on the new controller against 0.547 on the previous one, with half as many robots stuck (0.10 against 0.22).

d) What the controller cannot absorb.: On a fully flat map (Table XLI) the learned residual is what makes goal arrival work: 0.07 of robots fall with it, 0.32 with the LQR alone. The three hard designs are the light ball kb025 (0.25 times the ball mass), the heavy body bal4 (4 times the ballast) and the weak motors of tau6 (a 6 N m torque limit against 30). The light ball is nearly impossible for the linear controller (0.96) and the residual saves part of it (0.38); the heavy body and the weak motors gain

TABLE XXXIV

Navigation with pushes and pulls, and a wider planner margin. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; earlier measurement: controller g18_ret_rs025_s1 trained and evaluated at wheel-ball free-rolling friction 0.001 (the recommended checkpoint at 0.01 is in Table 37). Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 528 per row. Seeds: one controller seed (g18_ret_rs025_s1, frozen). Protocol: as Table 19, with pushes 5 to 20 N and pulls 3 to 12 N (first three rows), and with the obstacle margin widened from 0.22 m to 0.30 or 0.38 m (last five rows; their disturbance setting is in the label).

Hazard source	Falls	Reached goal	Stuck	Rolled onto a block
blind	0.86	0.14	0.00	0.44
privileged height map	0.41	0.58	0.01	0.13
ToF 18 x 14, head, pitch -25 deg	0.52	0.45	0.03	0.15
ToF 18 x 14, margin 0.30 m (undisturbed)	0.19	0.11	0.70	0.00
ToF 18 x 14, margin 0.38 m (undisturbed)	0.28	0.00	0.71	0.00
ToF 18 x 14, margin 0.30 m, pushes + pulls	0.62	0.14	0.24	0.17
ToF 18 x 14, margin 0.38 m, pushes + pulls	0.62	0.01	0.38	0.21
privileged map, margin 0.38 m, pushes + pulls	0.57	0.00	0.42	0.21

TABLE XXXV

Smooth follower against the jerky wavefront planner (privileged map). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; earlier measurement: controller g18_ret_rs025_s1 trained and evaluated at wheel-ball free-rolling friction 0.001 (the recommended checkpoint at 0.01 is in Table 37). Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 528 per row (48 per design, 11 designs). Seeds: one controller seed (g18_ret_rs025_s1, frozen). Protocol: blocks map, undisturbed, 20 s, privileged height map; the smooth follower (nav.py) tracks a carrot point 0.3 m down the same distance field, turns the commanded heading at most 2 rad/s, lowers speed with heading error and changes it at most 0.6 m/s^2. A dagger marks a value from the study log (the S89 file has no heading-rate field).

Planner (privileged height map)	Reached goal	Falls	Stuck	Rolled onto a block	Commanded heading-rate RMS [rad/s]
S89 wavefront, best neighbour every step	0.91	0.09	0.00	0.00	14.4†
smooth follower (carrot 0.3 m, turn rate at most 2 rad/s)	0.86	0.14	0.00	0.04	1.7

TABLE XXXVI

Smooth following with the ToF sensor: variants. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; earlier measurement: controller g18_ret_rs025_s1 trained and evaluated at wheel-ball free-rolling friction 0.001 (the recommended checkpoint at 0.01 is in Table 37). Domain randomisation: on at scale 1. Observation noise: on (sensor ranges noisy, with dropout). Worlds per cell: 528 per row. Seeds: one controller seed (g18_ret_rs025_s1, frozen). Protocol: blocks map, undisturbed, 20 s; the head gimbal is yawed either to the command direction or to the planner's target heading (eval hook; on hardware the gimbal would be commanded by the navigation layer).

Smooth follower with the ToF 18 x 14 sensor (head)	Reached goal	Falls	Stuck	Rolled onto a block	Commanded heading-rate RMS [rad/s]
smooth follower, sensor yawed along the command	0.72	0.22	0.05	0.16	1.8
smooth follower, sensor aimed at the target heading, 2 rad/s (adopted)	0.78	0.16	0.05	0.03	1.8
sensor aimed at the target heading, 4 rad/s	0.76	0.17	0.07	0.04	3.3
rejected: faster turning, 4 rad/s, sensor along the command	0.74	0.20	0.05	0.07	3.4
rejected: replanning every 0.04 s	0.69	0.25	0.06	0.26	1.9
rejected: slow-down within 0.3 m of seen obstacles	0.44	0.28	0.28	0.05	2.1
rejected: slow-down within 0.5 m	0.26	0.29	0.46	0.03	2.2
S89 wavefront (jerky), for reference	0.85	0.11	0.04	0.02	14.4†

much less (0.23 against 0.38, 0.12 against 0.15), which points to a design-space boundary rather than a training gap. Short pushes are absorbed fully (falls 0.078 against 0.074 undisturbed), sustained pulls up to about 3 N as well (0.078), and at the standard 3 to 12 N 0.36 fall on flat ground; nearly all of the gap between undisturbed and pushed battery results is these sustained pulls. A steady pull needs a steady lean against an unseen force, which may need integral action that the LQR base lacks and an 80 ms observation history cannot supply; this is being tested. All of this is simulation; there is no physical robot yet.

J. Generalisation to unseen robot designs

Two designs outside the 11 training designs were held out (Table XLII, Fig. 20): s1.5 (the robot scaled 1.5 times, ball radius 13.5 cm, with its own planar model for the LQR base) and tau3 (a wheel torque limit of one tenth of nominal).[2] The LQR base carries a policy to an unseen robot: the LQR-based controllers keep 0.61 to 0.71 on the larger robot, while pure learning with the retainer, the most robust controller on its training designs, collapses (0.09 to 0.15). The tenth-torque design is motor limited

[2]An earlier version of this evaluation computed the design vector relative to the first loaded design instead of the nominal one; it was fixed and re-run, and only the re-run is reported.

TABLE XXXVII

Training with the planner in the loop against the frozen controller (null result). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; earlier measurement: controller g18_ret_rs025_s1 trained and evaluated at wheel-ball free-rolling friction 0.001 (the recommended checkpoint at 0.01 is in Table 37). Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 528 per cell. Seeds: one frozen controller seed against two planner-trained seeds. Protocol: blocks map, smooth follower (sensor along the command), 20 s; the planner-trained runs spent 30 percent of training worlds on blocks tiles following the smooth planner on the privileged map, otherwise the recommended recipe.

Hazard source	frozen recommended controller (g18_ret_rs025_s1): reached / fell / on block	planner-trained, seed 0 (g23_nav_rs025_s0)	planner-trained, seed 1 (g23_nav_rs025_s1)
perfect map, undisturbed	0.86 / 0.14 / 0.04	0.85 / 0.13 / 0.01	0.85 / 0.14 / 0.02
perfect map, pushes + pulls	0.55 / 0.44 / 0.20	0.55 / 0.43 / 0.13	0.57 / 0.42 / 0.13
ToF 18 x 14, undisturbed	0.72 / 0.22 / 0.16	0.68 / 0.25 / 0.15	0.71 / 0.23 / 0.17
ToF 18 x 14, pushes + pulls	0.41 / 0.56 / 0.22	0.43 / 0.53 / 0.18	0.46 / 0.51 / 0.21

TABLE XXXVIII

The motion-layer recipe and its variants (S110 to S116). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; wheel-ball free-rolling friction 0.01. Domain randomisation: on at scale 1 unless a column says 1.75 or 2.5 (training: ramped to 1.75). Observation noise: on. Worlds per cell: goal test 528 per row (48 per design, 11 designs); battery 1760 per cell (16 per design and terrain kind). Seeds: one per row; three seeds of the recommended recipe g34 (s0, s1, s2). Protocol: goal test: eval_motion.py, goal 1.2 m away on flat interior tiles, 15 s, no pushes, commands through the motion layer for every run trained with it (the g26 row uses raw commands, as it was trained); hard designs = kb025 (light ball, 0.25 x ball mass), bal4 (heavy body, 4 x ballast) and tau6 (weak motors, 6 N m torque limit against 30). Battery: eval_battery.py, goal task 1.1 m and velocity task 0.3 m/s on the tiled map, 8 s, surfaces on, arrestors on; pushes 5 to 20 N for 50 ms every 2 to 4 s and pulls 3 to 12 N for 0.5 to 2 s every 4 to 10 s. All recipes 200M steps except g36 (400M); files outputs/study/warp_ppo/RUN/motion_goal_ref.json (g26: motion_goal_raw.json) and RUN/battery.json.

Recipe (run)	Goal test: falls	Command reversals per world	Hold speed RMS [m/s]	Falls of the three hard designs (mean)	Battery goal success DR 1: undisturbed / pushes + pulls	DR 1.75	DR 2.5	Tile velocity falls DR 1: undisturbed / pushes + pulls
previous recommendation, raw commands, no motion layer (g26 s1)	0.146	1.19	0.077	0.42	0.74 / 0.46	0.72 / 0.42	0.69 / 0.36	0.34 / 0.51
motion layer, fixed authority 0.25 (g29 s0)	0.108	0.38	0.047	0.33	0.76 / 0.49	0.73 / 0.44	0.67 / 0.39	0.34 / 0.50
motion layer, fixed authority 0.25 (g29 s1)	0.121	0.39	0.049	0.37	0.78 / 0.49	0.73 / 0.45	0.68 / 0.39	0.32 / 0.52
motion layer, fixed authority 0.25 (g29 s2)	0.108	0.71	0.050	0.32	0.78 / 0.51	0.75 / 0.46	0.70 / 0.40	0.33 / 0.49
adaptive authority 0.15 / 0.3 (g32 s0)	0.083	4.94	0.096	0.29	0.76 / 0.50	0.73 / 0.44	0.68 / 0.39	0.32 / 0.48
adaptive authority 0.15 / 0.3 (g32 s1)	0.104	0.57	0.063	0.35	0.75 / 0.52	0.72 / 0.46	0.67 / 0.38	0.32 / 0.48
recommended: adaptive authority + goal dead band (g34 s0)	0.104	0.67	0.058	0.32	0.79 / 0.51	0.77 / 0.47	0.69 / 0.39	0.32 / 0.48
recommended: adaptive authority + goal dead band (g34 s1)	0.091	0.86	0.065	0.31	0.77 / 0.51	0.73 / 0.45	0.68 / 0.39	0.32 / 0.48
recommended: adaptive authority + goal dead band (g34 s2)	0.110	0.48	0.052	0.30	0.78 / 0.52	0.75 / 0.45	0.69 / 0.39	0.31 / 0.48
adaptive + tracking reward 3 (g33 s0)	0.100	0.99	0.068	0.33	0.77 / 0.53	0.75 / 0.45	0.70 / 0.41	0.32 / 0.48
adaptive + tracking reward 3 (g33 s1)	0.081	1.38	0.060	0.29	0.77 / 0.51	0.73 / 0.45	0.67 / 0.38	0.30 / 0.48
adaptive + hard-design weighting (g35 s0)	0.091	0.91	0.063	0.28	0.81 / 0.53	0.78 / 0.45	0.72 / 0.39	0.32 / 0.49
adaptive + hard-design weighting (g35 s1)	0.108	0.40	0.059	0.29	0.79 / 0.52	0.77 / 0.45	0.72 / 0.39	0.31 / 0.49
adaptive + hard-design weighting (g35 s2)	0.081	0.69	0.044	0.28	0.77 / 0.52	0.73 / 0.49	0.67 / 0.40	0.31 / 0.46
recommended + weighting, 400M steps (g36 s0)	0.068	3.73	0.076	0.23	0.74 / 0.47	0.71 / 0.42	0.65 / 0.35	0.32 / 0.49
recommended + weighting, 400M steps (g36 s1)	0.072	2.52	0.067	0.25	0.73 / 0.48	0.69 / 0.46	0.65 / 0.38	0.32 / 0.49
recommended + weighting, 400M steps (g36 s2)	0.064	3.36	0.073	0.22	0.72 / 0.45	0.69 / 0.41	0.64 / 0.34	0.32 / 0.49
recommended, trained on 1.5 x pushes and pulls (g37 s0)	0.136	1.58	0.082	0.37	0.79 / 0.49	0.76 / 0.43	0.70 / 0.36	0.34 / 0.50
recommended, trained on 2 x pushes and pulls (g37 s0)	0.119	1.84	0.073	0.37	0.79 / 0.49	0.76 / 0.45	0.72 / 0.36	0.33 / 0.52

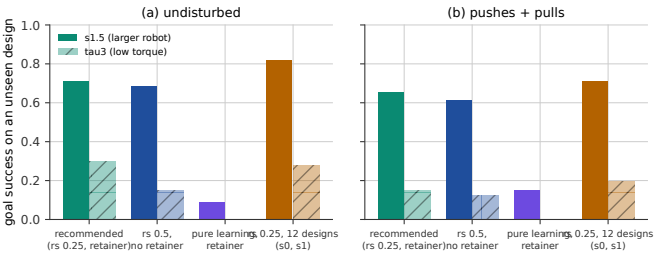


Fig. 20. Goal success on unseen robot designs. Engine: Warp GPU, corrected. Randomisation: on at scale 1. Noise: on. Worlds: 160 per bar (16 per terrain kind). Seeds: one per bar (seed 1); the 12-design bar averages two seeds. Protocol: goal task of Table X on s1.5 (solid) and tau3 (hatched); tau3 is a training design for the 12-design runs.

for every controller (at most 0.30), and training on it does not help (0.29 and 0.26); training on 12 designs raises the unseen larger robot to 0.89 and 0.75 undisturbed (two seeds), with a seed-dependent gain under pushes and pulls.

a) Implications for a real robot (simulation only).: Four hypotheses follow for a hardware study, none tested on hardware: keep a model-based base controller with a small learned residual, since it transferred to a body unlike the training set while pure learning did not; add passive ball arrestors, which in simulation removed most falls without changing the controller's model; give the motors torque headroom, since with a tenth of the nominal torque every controller failed; and choose the ball radius for the steps that must be climbed, about 3.4 to 4.2 times the tallest step (§X-G).

K. Perception

The test is a controlled pair on the corrected engine: the ramped recipe with edge-focused starts, trained once with the 18×14 head-mounted depth sensor (noisy, with dropout, learned end to end) and once blind, otherwise identically (200M steps, one seed each; Table XLIII). The depth sensor gives no gain over the blind control: goal success is 0.60 with depth against 0.60 blind undisturbed and 0.29 against 0.28 with pushes and pulls, the two differ

TABLE XXXIX

Navigation with the motion layer (S114). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; wheel-ball free-rolling friction 0.01. Domain randomisation: on at scale 1 (11 designs, floor classes). Observation noise: on (sensor ranges: noise 0.5 percent of range, 2 percent dropout). Worlds per cell: 528 (48 per design, 11 designs). Seeds: one controller seed per row, frozen. Protocol: eval_nav.py on the blocks map (8 cm blocks, goal 1.2 m away behind a block), 20 s, smooth wavefront follower (carrot 0.3 m, turn limit 2 rad/s, v_max 0.3 m/s), ToF on the head pitched down 25 deg and aimed at the planner target; for runs trained with the motion layer the planner command passes through motion_ref.py (arrival braking, goal hold) unless NO_MREF=1; pushes 5 to 20 N and pulls 3 to 12 N in the pushes column; files outputs/study/warp_ppo/RUN/navS_tof18,tof18_pp,oracle[_nomref].json.

Controller (motion layer in the loop or not)	ToF 18 x 14: reached / fell / stuck	Commanded heading-rate RMS [rad/s]	Command change per step	Median time to goal	ToF, pushes and pulls: reached / fell	Perfect map: reached / fell / stuck	Perfect map heading-rate RMS [rad/s]
previous recommendation, no motion layer (g26 s1; Table 37)	0.845 / 0.072 / 0.083	1.74	0.50	10.8 s	0.511 / 0.432	0.886 / 0.072 / 0.042	1.69
adaptive authority, with the motion layer (g32 s1)	0.852 / 0.078 / 0.070	1.29	0.40	9.7 s	0.544 / 0.400	0.926 / 0.064 / 0.009	1.23
recommended, with the motion layer (g34 s0)	0.877 / 0.080 / 0.044	1.31	0.40	9.2 s	0.557 / 0.413	0.938 / 0.061 / 0.002	1.23
recommended checkpoint, planner commands raw (g34 s0, NO_MREF=1)	0.881 / 0.070 / 0.049	1.79	0.51	8.9 s	not run	0.915 / 0.083 / 0.002	1.74

TABLE XL

Blind navigation with a contact escape layer (S109, S115). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; wheel-ball free-rolling friction 0.01. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 528 per row (48 per design, 11 designs). Seeds: one controller seed per pair of rows, frozen. Protocol: blocks map as Table 39 but no terrain sensing at all, 40 s episodes; escape layer experiments/warp_ppo/escape.py through eval_nav.py –source bump: stall detection (under 30 percent of the commanded speed for 1 s, no 0.1 m approach in 4 s, or tilt above 15 deg), a 0.3 m obstacle segment marked 0.2 m ahead, 1 s back-up at 0.15 m/s, re-plan, give up after 6 failures; files outputs/study/warp_ppo/RUN/navS_blind,bump_v15_40s.json.

Controller	Reached goal	Fell	Stuck	Rolled onto a block	Median time to goal
blind, straight at the goal previous recommendation g26 s1, no layer	0.078	0.922	0.000	0.75	6.5 s
blind + contact escape (stall detection, obstacle memory, back up, re-plan), 0.15 m/s previous recommendation g26 s1, no layer	0.547	0.237	0.216	0.55	14.7 s
blind, straight at the goal recommended g34 s0, motion layer	0.017	0.983	0.000	0.81	6.0 s
blind + contact escape (stall detection, obstacle memory, back up, re-plan), 0.15 m/s recommended g34 s0, motion layer	0.634	0.261	0.104	0.44	12.1 s

by at most 0.03 at any randomisation scale (about the seed and evaluation spread), and step up, stairs and bump are equal within noise; the tile velocity task falls slightly more with depth (0.56 against 0.53). Edge-focused starts alone do not move the baseline (blind with edge starts 0.60 against 0.58 and 0.60 without). Each arm is a single seed, one sensor and an unchanged reward that does not value foresight; this shows that plain PPO did not learn to use this sensor, not that perception cannot help. A policy-shaped encoder pretrained to predict the height grid ahead reaches R^2 0.60 (Table XLV), weakest in the closest row because the 25° downward pitch of the head mount leaves the nearest floor outside the zone grid; its frames came from a policy on the affected engine and no policy here uses it. The same sensor added on top of the recommended controller with the retainer (one seed, Table XLIV) also adds nothing: goal success 0.74 undisturbed and 0.43 with pushes and pulls, against 0.74 and 0.43 blind (three seeds), and obstacle falls unchanged within the seed spread.

XI. Discussion

What the theory buys. The derivation explains every qualitative failure we saw in learning: timid policies are timid because a 1° lean buys only $0.1\,\mathrm{m/s^2}$ (corollary); fast responses are bounded by the zero at $5.8\,\mathrm{s^{-1}}$; and the drive inertia J_{eff}, invisible in a naive pendulum-on-a-ball picture, dominates the rotational dynamics. It also yields a controller that is exact where the model is.

What learning buys. PPO found the same lean law without a model and remained stable in regimes the linear model does not cover. Its weaknesses (jitter, heading generalisation, sensitivity to the command distribution) are exactly where the model-based controller is strong, which is the case for residual learning.

What the GPU study adds. Model-based structure beats pure learning on accuracy and on the goal task; no training recipe improves robustness to pushes, pulls and rough terrain, and wider training randomisation does not help, because the falls are mechanical (the body derails off the ball); a passive ball retainer that leaves the LQR model unchanged improves every robustness measure, at a cost in tracking when trained with it. Smoothness at the command level has to be trained with, not bolted on: a motion-reference layer added to a finished policy made goal arrival worse, while the same layer used in training, with an authority that rises only when the robot

TABLE XLI

Limits on flat ground: hard designs and sustained pulls (S114, S116). Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error; wheel-ball free-rolling friction 0.01. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 528 per row in the first two rows (48 per design); 352 per row in the others (32 per design). Seeds: one controller seed (g34_mref_rsad_db_s0, recommended recipe), frozen. Protocol: TILES_FLAT=1, so no terrain is involved. First two rows: eval_motion.py goal test (1.2 m, 15 s, motion layer in the loop), files motion_goal_ref_allflat[_lqronly].json. Other rows: eval_tiles2.py goal task (1.1 m, 8 s) with PUSH_SCALE multiplying both the pushes and the sustained pulls, files eval2_goal_goal_dr_surf_*_flat_noexit.json.

Test on a fully flat map (g34 s0)	Falls, all 11 designs	kb025 light ball (0.25 x ball mass)	bal4 heavy body (4 x ballast)	tau6 weak motors (6 N m)	the other eight designs
goal arrival, no disturbances: full controller (LQR + residual + motion layer)	0.068	0.38	0.23	0.12	at most 0.02
goal arrival, no disturbances: LQR alone (residual zeroed), same motion layer	0.324	0.96	0.38	0.15	0.17 to 0.35
goal task 1.1 m, no pushes, no pulls	0.074	0.42	0.17	0.13	0.00 to 0.07
goal task 1.1 m, short pushes only (5 to 20 N, 50 ms)	0.078	0.40	0.19	0.13	0.00 to 0.08
goal task 1.1 m, pushes and pulls at 0.25 x (pulls 0.75 to 3 N)	0.078	0.41	0.18	0.08	0.00 to 0.07
goal task 1.1 m, 0.5 x (pulls 1.5 to 6 N)	0.139	0.47	0.18	0.11	0.01 to 0.58
goal task 1.1 m, 0.75 x (pulls 2.25 to 9 N)	0.221	0.52	0.18	0.21	0.06 to 0.83
goal task 1.1 m, 1 x, the standard battery (pulls 3 to 12 N)	0.361	0.59	0.18	0.47	0.17 to 0.93

TABLE XLII

Generalisation to unseen robot designs, corrected engine. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on at scale 1. Observation noise: on. Worlds per cell: 160 per design (16 per terrain kind). Seeds: one per row as listed (two for the 12-design runs). Protocol: goal task of Table 2 on two designs outside the 11 training designs: s1.5 (robot scaled 1.5 times, ball radius 13.5 cm) and tau3 (wheel torque limit one tenth of nominal); the 12-design runs include tau3 in training, so for them only s1.5 is unseen.

Policy	Trained on tau3	Undisturbed: s1.5 (larger robot, unseen)	tau3 (one tenth torque)	Pushes + pulls: s1.5	tau3
recommended: LQR + residual 0.25, retainer g18_ret_rs025_s1, seed 1	no	0.71	0.30	0.66	0.15
LQR + residual 0.5, no retainer g12_dr175_s1, seed 1	no	0.69	0.15	0.61	0.12
pure learning, retainer g16_pure_ret100_s1, seed 1	no	0.09	0.00	0.15	0.00
LQR + residual 0.25, retainer, 12 designs g21_ret_rs025_tau3_s0, seed 0	yes	0.89	0.29	0.76	0.17
LQR + residual 0.25, retainer, 12 designs g21_ret_rs025_tau3_s1, seed 1	yes	0.75	0.26	0.66	0.21

TABLE XLIII

Depth sensor against blind control, both with edge-focused starts, corrected engine. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on (flat error column: off). Observation noise: on (flat error column: off). Worlds per cell: 176 per terrain kind (1760 per cell); 64 for the flat column. Seeds: one seed per arm (seed 0), the reference rows are the two seeds of the ramped recipe. Protocol: goal task of Table 2 at DR scales 1, 1.75 and 2.5; the per-kind columns are undisturbed success at scale 1; tile velocity task at 0.3 m/s; both arms are the ramped recipe plus –edge-focus 0.5 and differ only by the depth sensor.

Policy	Goal success DR 1: undisturbed / disturbed	DR 1.75	DR 2.5	Median time [s]	step up	stairs	bump	Tile velocity falls: undisturbed / disturbed	Flat error at 0.9 m/s
depth sensor (18x14, head mount) + edge starts g15_edge_depth_s0, seed 0	0.60 / 0.29	0.53 / 0.26	0.45 / 0.18	2.36	0.36	0.44	0.27	0.56 / 0.73	0.044
blind control + edge starts g15_edge_blind_s0, seed 0	0.60 / 0.28	0.54 / 0.23	0.47 / 0.19	2.30	0.38	0.41	0.28	0.53 / 0.73	0.046
reference: ramped recipe, no edge starts g12_dr175_s0, seed 0	0.58 / 0.29	0.54 / 0.25	0.45 / 0.19	2.44	0.31	0.44	0.26	0.54 / 0.71	0.057
reference: ramped recipe, no edge starts g12_dr175_s1, seed 1	0.60 / 0.30	0.56 / 0.25	0.47 / 0.19	2.33	0.39	0.47	0.20	0.53 / 0.71	0.041

manoeuvres, gave the best controller of the study (§X-I). What it cannot absorb is physical: a light ball, a heavy body, weak motors, and sustained pulls above about 3 N. A practical lesson is that per-world masses in a batched model require recomputing the derived mass properties.

Limitations. Simulation only, with an idealised anisotropic omniwheel friction model and a float32 GPU engine that is not bit-reproducible; the planar model has documented mismatches (soft-contact creep, a 28 % transmission-rate discrepancy). The CPU-study configurations mostly use one seed (the key S6 result is replicated) and two repeats per condition. In the GPU study the clean flat benchmark uses one nominal design and 64 worlds per row, the tile evaluations 16 worlds per design and terrain kind, and most rows are single-seed; most evaluations use the training designs, and two held-out designs and a held-out terrain layout were tested for a few controllers only; there is no CPU re-evaluation of the tile policies; floor materials are plausible, not measured; and the depth sensor assumes a yaw aligned with the command. The perception test is one seed per arm. Yaw control is weak for every controller and the CPU training budgets of $\sim 10^7$ steps were far below GPU-parallel locomotion studies [14].

Next steps. (i) A second seed of the perception arm and the privileged height scan. (ii) Held-out robot designs, and a CPU re-evaluation of the tile policies. (iii) A structure that rewards foresight (approach-speed shaping, an imitation prior, a planner) before drawing conclusions about perception. (iv) A 3-D linearisation to explain the 1 m/s edge. (v) Motor and compute sizing, and a sim-to-real study; no hardware result is claimed here.

XII. Reproducibility

Environment ballbot_gym/envs/ballbot_env.py; model derivation scripts/study/ballbot_model.py; LQR ballbot_gym/controllers/lqr.py; configurations configs/env/*.yaml; training train_ppo.py (CPU) and experiments/warp_ppo/train_dev.py (GPU); GPU eval-

TABLE XLIV

Depth sensor on top of the recommended controller (retainer), corrected engine. Engine: Warp GPU (MuJoCo-Warp 3.17, float32), corrected for the S69 stale-subtree-mass error. Domain randomisation: on (flat column: off). Observation noise: on (flat column: off). Worlds per cell: 176 per terrain kind (1760 per goal or tile cell); 64 per flat cell. Seeds: one seed for the depth arm, three for the blind recommended controller. Protocol: eval_battery.py with the 100 deg arrestors in training and evaluation; the depth arm also uses edge-focused starts.

Policy	Goal success undisturbed, DR 1 / 1.75 / 2.5	Goal success pushes + pulls, DR 1 / 1.75 / 2.5	Tile velocity falls undisturbed / disturbed (DR 1)	Step-up falls (velocity task)	Flat achieved / command
recommended + depth sensor (18x14) + edge starts g20_ret_rs025_depth_s0, seed 0	0.74 / 0.69 / 0.63	0.43 / 0.38 / 0.31	0.35 / 0.54	0.55	0.96 to 0.99
recommended, blind g18_ret_rs025_s0, seed 0	0.71 / 0.68 / 0.60	0.42 / 0.36 / 0.29	0.34 / 0.55	0.61	0.99 to 1.01
recommended, blind g18_ret_rs025_s1, seed 1	0.76 / 0.72 / 0.66	0.43 / 0.39 / 0.32	0.35 / 0.56	0.62	0.98 to 1.00
recommended, blind g18_ret_rs025_s2, seed 2	0.76 / 0.71 / 0.63	0.44 / 0.41 / 0.32	0.36 / 0.55	n/a	0.90 to 0.99

TABLE XLV

Supervised depth-to-height encoder. Engine: Warp GPU (MuJoCo-Warp 3.17, float32). Domain randomisation: on in the data collection (see the caveat). Observation noise: on (depth zone noise 0.5 % of range, 2 % dropout). Worlds per cell: 606k frames from 2048 worlds, 11 designs; validation = 10 % of worlds, split by world. Seeds: 1. Protocol: supervised regression of the height grid ahead and rise/drop hazards from an 18x14 simulated depth image; the driver policy is g2_goal_s0. Caveat: the frames were collected by a policy running on the engine before the S69 fix (stale subtree mass under mass randomisation). The metrics describe depth-to-height prediction, which does not depend on that artefact, but the frame distribution does.

Encoder	R2 (7x5 height grid)	MAE [cm]	rise $\geq$ 2 cm F1	drop $\geq$ 2 cm F1	R2 by forward row (0.1 ... 1.3 m)
mean baseline	-0.00	3.87	0.00	0.00	-0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00
proprio only	0.02	3.97	0.25	0.20	0.04 0.05 0.04 0.02 -0.01 -0.02 -0.01
depth+proprio	0.55	2.19	0.62	0.67	0.13 0.42 0.61 0.69 0.70 0.68 0.64
depth only policy shaped	0.60	1.93	0.70	0.73	0.17 0.47 0.68 0.74 0.74 0.71 0.67

uation eval_tiles2.py, batched LQR lqr_gpu.py; the GPU-study tables, figures and numbers site/build_data.py; benchmarks eval_control.py, eval_falls.py; theory checks analyze_theory.py; videos render_control.py; tables and figures make_tracking_report.py; decision log docs/research/tracking_study_log.md.

References

[1] N. Tadesse, "Reinforcement learning for ballbot navigation: Replication and enhancement of Salehi (2025)," Johns Hopkins University course report, 2025.

[2] A. Salehi, "Reinforcement learning for ballbot navigation in uneven terrain," arXiv:2505.18417, 2025.

[3] T. B. Lauwers, G. A. Kantor, and R. L. Hollis, "A dynamically stable single-wheeled mobile robot with inverse mouse-ball drive," in Proc. IEEE ICRA, 2006, pp. 2884–2889.

[4] U. Nagarajan, G. Kantor, and R. Hollis, "The ballbot: An omnidirectional balancing mobile robot," Int. J. Robotics Research, vol. 33, no. 6, pp. 917–930, 2014.

[5] P. Fankhauser and C. Gwerder, "Modeling and control of a ballbot," Bachelor thesis, ETH Zürich, 2010.

[6] M. Abdelrahim et al., "Modeling and control of a ballbot: A systematic approach," IEEE Access, 2025, doi:10.1109/ACCESS.2025.3597797.

[7] C. Xiao et al., "Design and control of a ballbot drivetrain with high agility, minimal footprint, and high payload," arXiv:2304.02887, 2023.

[8] Y. Zhou, J. Lin, S. Wang, and C. Zhang, "Learning ball-balancing robot through deep reinforcement learning," arXiv:2208.10142, 2022.

[9] C. Mavrogiannis, K. Balasubramanian, S. Poddar, A. Gandra, and S. S. Srinivasa, "Topology-informed model predictive control for anticipatory collision avoidance on a ballbot," 2021.

[10] J. Carius, R. Ranftl, F. Farshidian, and M. Hutter, "Constrained stochastic optimal control with learned importance sampling: A path integral approach," Int. J. Robotics Research, vol. 41, no. 2, 2022.

[11] K. J. Åström and R. M. Murray, Feedback Systems: An Introduction for Scientists and Engineers. Princeton University Press, 2008.

[12] J. Hwangbo et al., "Learning agile and dynamic motor skills for legged robots," Science Robotics, vol. 4, no. 26, 2019.

[13] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," Science Robotics, vol. 5, no. 47, 2020.

[14] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in Proc. CoRL, 2022.

[15] S. Mysore, B. Mabsout, R. Mancuso, and K. Saenko, "Regularizing action policies for smooth control with reinforcement learning," in Proc. IEEE ICRA, 2021.

[16] J. Tobin et al., "Domain randomization for transferring deep neural networks from simulation to the real world," in Proc. IEEE/RSJ IROS, 2017.

[17] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in Proc. IEEE ICRA, 2018.

[18] T. Johannink et al., "Residual reinforcement learning for robot control," in Proc. IEEE ICRA, 2019.

[19] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, "Residual policy learning," arXiv:1812.06298, 2018.

[20] D. Amodei et al., "Concrete problems in AI safety," arXiv:1606.06565, 2016.

[21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[22] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in Proc. ICLR, 2016.

[23] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in Proc. IEEE/RSJ IROS, 2012, pp. 5026–5033.

[24] A. Raffin et al., "Stable-Baselines3: Reliable reinforcement learning implementations," J. Mach. Learn. Res., vol. 22, no. 268, pp. 1–8, 2021.